

A COMPREHENSIVE TECHNICAL TUTORIAL

The Principles of Operation of the

KNOWLEDGE GRAPH

*From Relational Foundations, through GraphQL, to a Complete Methodology for
Converting a Relational Database Model into a Knowledge Graph*

PART I · Relational Foundations
PART II · Graph Thinking & GraphQL
PART III · Knowledge Graphs & Conversion Methodology
PART IV · Querying, Architecture & Practice
PART V · Algorithms, Analytics & AI

20 Original Diagrams · Worked Case Study · Complete Mapping Reference

Preface

Knowledge graphs occupy a strange position in the modern data landscape: widely discussed, adopted by some of the largest and most sophisticated technology companies in the world for their most valuable systems : search, recommendations, fraud detection, virtual assistants : and yet still, for most engineering teams, a somewhat abstract idea rather than a concrete, learnable skill.

This tutorial exists to close that gap, for one specific and very common starting point: an organisation that already runs on relational databases and wants a rigorous, honest, no-hype path toward adding knowledge graph capability : not as a replacement for what already works, but as a complementary lens for the connected-data questions relational tables answer only awkwardly.

Every technique in this tutorial is grounded in the relational concepts a working engineer already knows, translated deliberately and explicitly into graph terms, with a complete worked case study to show the full process end to end. Nothing here requires a formal background in graph theory or description logic : though Chapter 6 will give you a working, practical grasp of both.

Table of Contents

Preface	2
Table of Contents	3
1 Introduction & How To Use This Tutorial	10
What this tutorial covers	10
Who this is for	10
Conventions used in this document	11
2 Foundations of the Relational Database Model.....	12
2.1 The relational model: tables as sets of facts	12
2.2 Entity-Relationship modeling.....	13
2.3 Normalization: why relational schemas look the way they do	13
First Normal Form (1NF).....	14
Second Normal Form (2NF)	14
Third Normal Form (3NF)	14
2.4 Keys, constraints, and indexes.....	15
2.5 SQL and the JOIN operation	15
2.6 A quick performance vocabulary.....	16
2.7 Transactions and the ACID guarantees.....	17
2.8 Views, materialized views, and stored procedures	17
3 Where the Relational Model Strains: Connected Data	18
3.1 The multi-hop question problem	18
A recursive CTE example, and its friction	19
3.2 Schema rigidity and the many-to-many tax.....	19
3.3 Data silos and the integration problem.....	19
3.4 What the relational model still does better	20
3.5 A note on consistency: why 'derived graph' implies 'eventual consistency'	20
4 Graph Data Models: Property Graphs and RDF.....	22
4.1 The Labeled Property Graph (LPG) model	22
4.2 The RDF / triple model	23
4.3 Property graph vs. RDF: a practical comparison.....	24
4.4 A worked mini-example in both models	24
As a property graph (Cypher).....	24
As RDF (Turtle) : note the workaround needed for edge properties	24
As RDF-star (attaching properties directly to the statement)	25
4.5 How graph models relate to document and key-value stores	25
5 GraphQL: A Typed Query Language for Connected APIs.....	26
5.1 The type system and Schema Definition Language (SDL)	26
5.2 Queries, mutations, and subscriptions.....	27
5.3 Resolvers: where the graph traversal actually happens.....	27

5.4 The N+1 problem, and why it rhymes with the JOIN problem	28
5.5 GraphQL Federation and schema stitching	29
5.6 Why GraphQL and knowledge graphs pair so naturally	29
5.7 Security: query complexity, depth limiting, and rate limiting.....	29
6 Principles of Knowledge Graphs.....	32
6.1 What makes a graph a KNOWLEDGE graph	32
6.2 T-Box and A-Box: separating schema from data	32
6.3 Building an ontology: classes, properties, constraints	33
6.4 Inference and reasoning: getting facts you never stored	34
Taxonomic inference (subclass reasoning)	34
Property characteristic inference	34
6.5 Named graphs, provenance, and multi-source integration	34
6.6 Vocabulary and standards worth knowing	35
6.7 Building a new ontology vs. reusing an existing one.....	35
6.8 Common ontology design patterns	36
N-ary relations	36
Qualified cardinality restrictions	36
Value partitions (controlled enumeration as classes, not literals).....	36
7 Converting a Relational Database into a Knowledge Graph.....	37
7.1 Overview: the conversion is a mapping problem, not a rewrite	37
7.2 Step 1 : Schema analysis and classification	37
7.3 Step 2 : Apply the core mapping rules	37
7.3.1 Foreign keys become directed, typed relationships	38
7.3.2 Collapsing junction tables : the single highest-value transformation.....	38
7.4 Step 3 : Choose a mapping mechanism and write the mapping	39
Option A : Custom ETL scripts.....	39
Option B : Declarative mapping standards: R2RML and RML	40
Option C : Vendor-native import / ETL platforms	40
7.5 Step 4 : End-to-end pipeline architecture	41
7.6 Step 5 : Entity resolution across sources	42
7.7 The resulting graph	42
7.8 Handling structurally tricky cases.....	43
7.8.1 Table inheritance / polymorphic associations	43
7.8.2 Nullable foreign keys	43
7.8.3 Composite and natural keys.....	43
7.8.4 Temporal / bitemporal data and slowly changing dimensions	44
7.8.5 Soft deletes	44
7.9 Validating the migration	44
7.10 Keeping the graph in sync: incremental updates	44
7.11 Measuring the payoff: benchmarking before and after	45

8	Querying the Graph: Cypher, SPARQL, and Gremlin	48
8.1	Cypher (property graphs : Neo4j, Memgraph, and openCypher-compatible engines)	48
	Basic pattern matching.....	48
	Variable-length paths.....	48
	Aggregation and write operations.....	48
8.2	SPARQL (RDF triple stores).....	49
	Basic triple pattern matching.....	49
	Property paths : SPARQL's variable-length traversal.....	49
	Aggregation, and a distinctive SPARQL feature: federated queries	49
8.3	Gremlin (Apache TinkerPop : a graph-traversal, not a graph-pattern, language).....	49
8.4	Side-by-side: the same question, four ways	50
9	Comparing Query Languages, and Where GraphQL Fits.....	51
9.1	One question, four query languages.....	51
9.2	GraphQL is a client-facing layer, not a storage-layer competitor.....	51
9.3	A decision framework.....	52
10	Production Architecture Patterns.....	53
10.1	Where the knowledge graph sits relative to systems of record.....	53
10.2	Consistency models: how stale can the graph be?	53
10.3	Scaling a graph database	53
10.4	Hybrid architectures: querying across both stores	54
	Pattern A : Federation at the API layer	54
	Pattern B : Federation at the query layer.....	54
10.5	Operational concerns.....	54
10.6	Team ownership and organisational patterns	55
10.7	Multi-region and geo-distribution.....	55
10.8	Cost management.....	55
11	Case Study: Migrating a Support Ticketing System.....	57
11.1	The scenario	57
11.2	The source schema.....	57
11.3	Step 1 : Schema analysis and classification.....	57
11.4	Step 2 : Define the target ontology	58
11.5	Step 3 : The mapping	58
11.6	Step 4 : Validate.....	59
11.7	Step 5 : Answering the business questions	59
	Churn-pattern similarity	59
	Ticket clustering by relationship structure, not just text	59
12	Best Practices, Pitfalls, and Anti-Patterns.....	61
12.1	:Modelling best practices	61
12.2	Migration process best practices.....	61
12.3	Common anti-patterns to avoid	62

12.4 A pre-launch checklist.....	62
12.5 Roles and responsibilities: a sample RACI	63
13 Conclusion & Further Reading.....	64
13.1 What we covered	64
13.2 A closing principle.....	64
13.3 Suggested next steps	64
13.4 Further reading	65
14 Graph Algorithms and Analytics.....	67
14.1 Why relational databases cannot do this well.....	67
14.2 Centrality algorithms: finding what matters.....	67
Degree centrality	67
PageRank	68
Betweenness centrality.....	68
14.3 Community detection: finding natural clusters.....	68
14.4 Similarity and link prediction.....	68
14.5 Shortest path and pathfinding.....	69
14.6 Running algorithms in a hybrid architecture	69
15 Knowledge Graphs, Embeddings, and GraphRAG	70
15.1 Vector embeddings: a second, complementary notion of “related”	70
15.2 Where do embeddings come from?	71
15.3 GraphRAG: grounding LLM answers in graph traversal.....	71
15.4 Why GraphRAG matters specifically for multi-hop questions.....	71
15.5 A minimal GraphRAG implementation sketch	72
15.6 Practical guidance: when GraphRAG is worth the added complexity	72
16 Data Quality, Testing, and Governance.....	74
16.1 Six dimensions of graph data quality	74
16.2 Automated constraint enforcement	74
16.3 Testing a knowledge graph like software.....	75
16.4 Ontology governance: preventing schema drift.....	75
16.5 Access control and compliance	76
17 Second Case Study: Supply Chain Risk at a Manufacturer	77
17.1 The scenario	77
17.2 Why this is a harder integration problem than Chapter 11	77
17.3 Step 1 : Schema analysis across three sources.....	77
17.4 Step 2 : Entity resolution: reconciling vendor_code and DUNS number.....	78
17.5 Step 3 : :Modelling the recursive BOM structure.....	78
17.6 Step 4 : Computing propagated risk.....	78
17.7 Step 5 : Architecture: why this became a hybrid deployment	79
18 Industry-Specific :Modelling Patterns	80
18.1 Healthcare: patients, encounters, and clinical events.....	80

18.2 E-commerce and retail: catalog, behaviour, and recommendations.....	80
18.3 Financial services: transactions, accounts, and fraud rings	81
18.4 Telecommunications and network/IT infrastructure	81
A Appendix A : Glossary.....	84
B Appendix B : Quick-Reference: Relational → Graph Mapping.....	86
C Appendix C : Query Language Cheat Sheet.....	87
D Appendix D : Vendor & Tooling Landscape	88
Property graph databases	88
RDF triple stores.....	88
Relational-to-graph mapping tools	88
E Appendix E : Frequently Asked Questions	89
“Do we need to migrate ALL of our relational data into the graph?”.....	89
“Will the knowledge graph replace our relational database?”.....	89
“How much staleness is acceptable in the graph?”.....	89
“Which should we learn first: Cypher or SPARQL?”	89
“Is GraphQL a type of database?”	89
“How long does a typical migration take?”.....	89
“Do we need a data scientist / ontologist on the team?”	89
“Can we use our knowledge graph as a source for LLM applications?”	89
F Appendix F : Troubleshooting Guide.....	91
G Appendix G : Extended SQL-Pattern Mapping Catalog	92
G.1 One-to-one relationships	92
G.2 Multi-column (composite) foreign keys	92
G.3 Enum / check-constraint columns	92
G.4 Star-schema fact and dimension tables (data warehouse migrations)	92
G.5 Audit / history tables (temporal patterns beyond Section 7.8.4).....	93
G.6 Polymorphic / generic reference columns	93
H Appendix H : Index of Figures	94
I Appendix I : A Complete, Runnable Sample Migration Script	95
J Appendix J : Cypher & SPARQL Function Quick Reference.....	98
String functions	98
Numeric and aggregate functions	98
Date and time functions	98
Null / existence handling.....	98
K Appendix K : A Complete, Annotated Sample Ontology	100
L Appendix L : Self-Check Questions.....	102
Part One : Relational Foundations	102
Part Two : Graph Thinking & GraphQL.....	102
Part Three : Knowledge Graphs & Conversion.....	102
Parts Four through Six	102

PART ONE

Relational Foundations

Chapters 1 – 3

1 Introduction & How To Use This Tutorial

Every organisation of any size runs on a relational database. Customer records, orders, inventory, claims, patients, transactions : for fifty years, the relational model has been the default answer to the question “where do we put the data?” It earned that position honestly: relational databases are mathematically well-founded, transactionally safe, and extraordinarily good at exactly the kind of structured, tabular record-keeping most businesses need.

But a second, quieter question has become just as important as data volumes and system complexity have grown: “how is everything connected?” Which customers know each other? Which transactions share a device fingerprint? Which parts, in a bill of materials ten levels deep, are affected if a supplier goes out of business? Which combination of symptoms, medications, and prior conditions predicts an adverse drug interaction? These are not “what is the value of this field” questions. They are “how do these things relate” questions : and answering them by repeatedly joining relational tables becomes slower, more brittle, and harder to reason about exactly as the number of hops grows.

This is the problem knowledge graphs were built to solve. A knowledge graph is not a replacement for the relational database : in almost every real deployment, the two coexist, often with the relational system remaining the transactional system of record and the knowledge graph serving as a connected, queryable view derived from it. Understanding how to move from one to the other : faithfully, efficiently, and without losing the discipline that made the relational model trustworthy in the first place : is the practical skill this tutorial is built to teach.

What this tutorial covers

This tutorial is organised in four parts, deliberately built from the ground up so that no prior familiarity with graph databases is assumed.

- **Part I : Relational Foundations** : a rigorous refresher on the relational model, normal forms, keys, indexing, and SQL joins : precisely the concepts we will later translate into graph terms.
- **Part II : Graph Thinking & GraphQL** : the labeled property graph model, RDF triples, and a full treatment of GraphQL as a query language and API layer, including where it fits (and does not fit) relative to a knowledge graph.
- **Part III : Knowledge Graphs & Conversion** : ontologies, schema layers, and the heart of this tutorial: a complete, worked methodology for converting a relational schema into a knowledge graph, with mapping rules, tooling, and a full case study.
- **Part IV : Querying, Architecture & Practice** : Cypher, SPARQL and Gremlin compared, production architecture patterns, and a library of best practices and pitfalls drawn from real migrations.

Who this is for

This tutorial is aimed at data engineers, backend developers, solution architects, and technically minded product owners who already have some exposure to SQL and relational systems and want a structured, example-driven path into knowledge graphs. No prior graph database experience is assumed. Where the material does assume familiarity : for instance, basic SQL syntax : it is reviewed explicitly in Part I rather than taken for granted.

Conventions used in this document

Convention	Meaning
Monospace text	Code, query language snippets, schema definitions, and file/field names.
Shaded box with a colored left rule	A callout: a key takeaway, warning, or design decision worth remembering.
Figure N.M	A numbered diagram; N is the chapter number, M is the figure's position within it.
:Label / :RELATIONSHIP_TYPE	Node labels and relationship types in property-graph examples, following common Cypher convention.

A note on scope

“Knowledge graph” is used throughout this tutorial in its common industry sense: a large, connected graph of typed entities and relationships, usually with an explicit or implicit schema (ontology), used to integrate and query data across sources. This encompasses both labeled property graphs (as used by Neo4j, Amazon Neptune, TigerGraph and similar systems) and RDF triple stores (as used by GraphDB, Stardog, Blazegraph, and Amazon Neptune's RDF mode). We treat both seriously, because real teams choose between them, and the conversion methodology in Part III applies to both with only minor adjustments.

2 Foundations of the Relational Database Model

Before we can meaningfully talk about converting a relational database into a knowledge graph, we need a precise, shared understanding of what a relational database actually is : not the folk description (“tables with rows and columns”) but the structural properties that make it what it is, because those exact properties are what we will be mapping, one by one, onto graph constructs in Part III.

2.1 The relational model: tables as sets of facts

Edgar F. Codd introduced the relational model in 1970 in his paper “A Relational Model of Data for Large Shared Data Banks.” The central idea was radical for its time: instead of navigating data through explicit pointers (as earlier hierarchical and network databases required), data should be organised into relations : mathematically, sets of tuples : and accessed declaratively, by describing WHAT you want rather than HOW to fetch it. A relation is what we colloquially call a table: a named set of rows (tuples), each row containing values for a fixed set of columns (attributes).

Figure 2.1 shows a small but complete example schema we will reuse throughout this tutorial: a simple e-commerce system with customers, orders, order line items, and products.

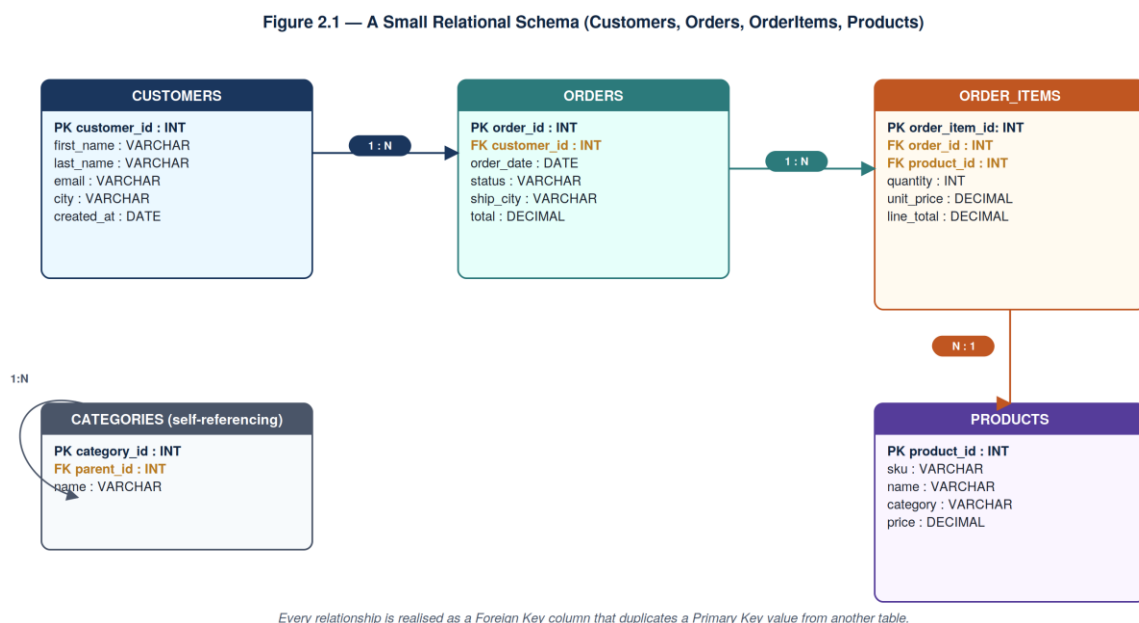


Figure 2.1 : A small relational schema with primary and foreign keys.

Four structural ideas are doing all the work in that diagram, and they are worth naming explicitly because each one has a direct : but not always obvious : counterpart in the graph world:

- **Primary Key (PK)** : a column, or minimal set of columns, whose value uniquely identifies a row within its table. No two rows may share a primary key value, and it may never be null.
- **Foreign Key (FK)** : a column whose value must match a primary key value in another table (or the same table, for self-references). This is the **ONLY** mechanism the relational model provides for expressing that two rows are related.
- **Referential integrity** : a database-enforced guarantee that every foreign key value actually points to an existing row : you cannot have an order that references a customer who doesn't exist.

- **Schema** : the fixed, declared structure (table names, column names, column types, constraints) that every row in a table must conform to, checked at write time.

2.2 Entity-Relationship modeling

Before a relational schema is implemented, it is almost always designed first at a conceptual level using Entity-Relationship (ER) modeling, introduced by Peter Chen in 1976. An ER diagram identifies the entities (things we want to track : customers, orders, products), their attributes, and the relationships between them, along with the cardinality of each relationship: one-to-one, one-to-many, or many-to-many. Figure 2.2 redraws our example schema in the crow's-foot notation that is the de facto industry standard for ER diagrams.

Figure 2.2 — Entity-Relationship Diagram (Crow's-Foot Notation)

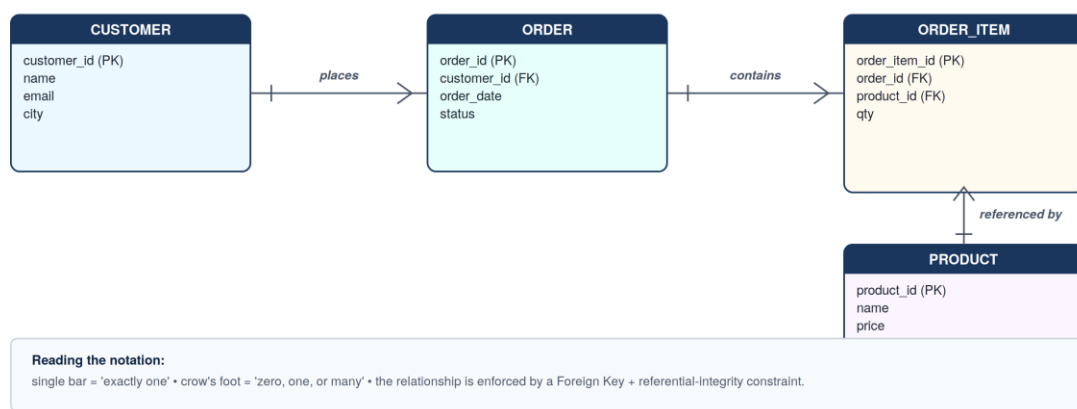


Figure 2.2 : The same schema expressed as an Entity-Relationship diagram.

It is worth internalising a subtlety here that becomes very important later: an ER diagram already “thinks” in terms of entities and relationships : the same vocabulary a knowledge graph uses. What the relational implementation does is take that conceptual graph-like model and flatten it into tables connected only by key-matching. In a very real sense, the conversion this tutorial teaches is a way of reversing that flattening : recovering the graph that was implicitly there in the ER diagram all along.

2.3 Normalization: why relational schemas look the way they do

Normalization is the process of organising columns and tables to minimise redundancy and avoid update, insertion, and deletion anomalies. It proceeds through a series of increasingly strict “normal forms.” Understanding normalization matters for graph conversion because normalized tables are, in effect, already partially decomposed into the entities and relationships a graph model wants : normalization and graph :Modelling are pulling in a related direction, even though they were invented for different reasons and optimise for different things.

Figure 2.3 — Progressive Normalization: Unnormalized → 1NF → 2NF → 3NF

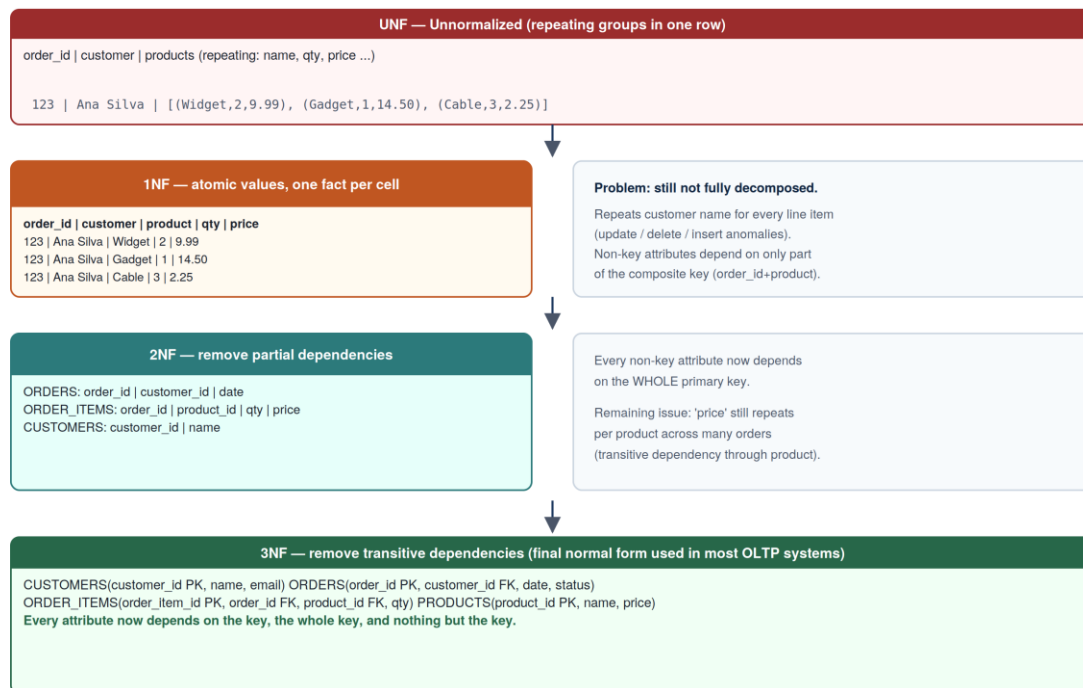


Figure 2.3 : Progressive normalization from an unnormalized record through Third Normal Form.

First Normal Form (1NF)

A table is in 1NF if every column holds a single, atomic value : no repeating groups, no comma-separated lists stuffed into one cell, no arrays. If a customer can have many phone numbers, 1NF says: put phone numbers in their own table, one row per number, not one column with three values crammed into it.

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key column depends on the **WHOLE** primary key, not just part of it. This only becomes relevant with composite primary keys (keys made of more than one column) : a common source of 2NF violations is a junction table where an attribute really describes only one of the two referenced entities, not the combination.

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and every non-key column depends **ONLY** on the primary key : not on another non-key column (a “transitive dependency”). The classic example: storing both product_id and product_price on an order_items row is a 3NF violation if price is really a property of the product, not of the specific order : a price change would then have to be propagated across every historical order line, corrupting the historical record. (In practice, order systems often intentionally denormalize price onto the order line for exactly this reason : to preserve the price **AT THE TIME** of the order. We will return to this tension between normalization and history-preservation when we discuss edge properties in Part III.)

Why this matters for graph conversion

The output of 3NF normalization : one table per real-world entity type, foreign keys expressing relationships between them : maps almost mechanically onto “one node label per entity type, one edge type per relationship” in a graph model. A poorly normalized relational schema (with repeating groups or conflated entities) will produce a poorly structured graph unless it is normalized : at least conceptually : as a first step in the conversion process. We revisit this explicitly in Chapter 7.

2.4 Keys, constraints, and indexes

Two more relational mechanisms deserve close attention because they too have direct graph analogues: constraints and indexes.

Constraints are rules the database enforces on every write: NOT NULL (a column may never be empty), UNIQUE (no two rows may share a value), CHECK (an arbitrary boolean condition, e.g. price $\geq$ 0), and the foreign-key constraint already discussed. These constraints are what make a relational database trustworthy : an application cannot accidentally corrupt referential integrity, because the database itself refuses the write.

Indexes are auxiliary data structures : most commonly B-Trees : that let the database find rows matching a condition without scanning every row. Figure 2.4 sketches how a B-Tree index narrows a lookup from a linear scan to a logarithmic walk down the tree.

Figure 2.4 — How a B-Tree Index Accelerates Lookups

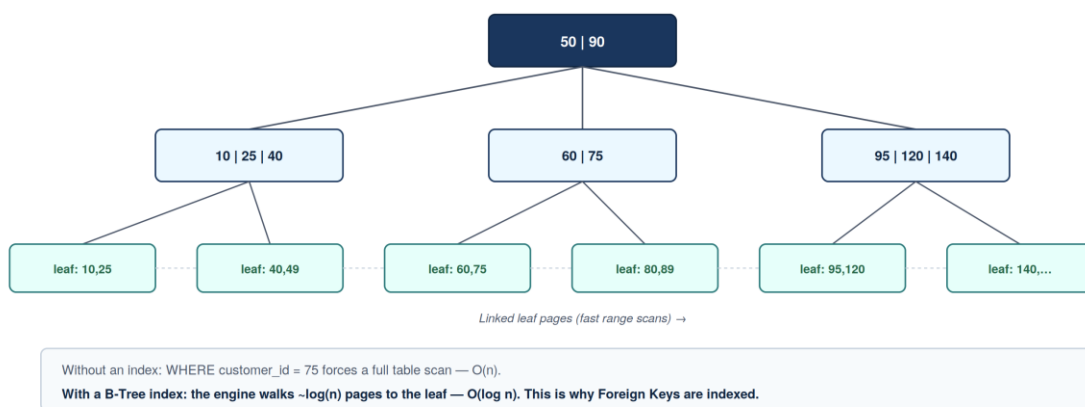


Figure 2.4 : A B-Tree index turns an $O(n)$ table scan into an $O(\log n)$ lookup.

Foreign key columns are, by convention and often by default, automatically indexed : because the whole point of a foreign key is to be looked up during a JOIN, and an unindexed foreign key turns every JOIN into a full table scan against the referenced table. This operational detail foreshadows one of the central performance arguments for graph databases, discussed in section 2.6: a graph database's native storage engine treats relationship traversal as a built-in, index-free operation (often called “index-free adjacency”) rather than something that must be separately indexed and then re-derived via a JOIN at query time.

2.5 SQL and the JOIN operation

SQL (Structured Query Language) is the declarative language used to read and write relational data. Its single most important operation, for our purposes, is the JOIN : the mechanism by which two or more

tables are combined into one result set by matching foreign key values against primary key values. Figure 2.5 reviews the four common JOIN types and their semantics.

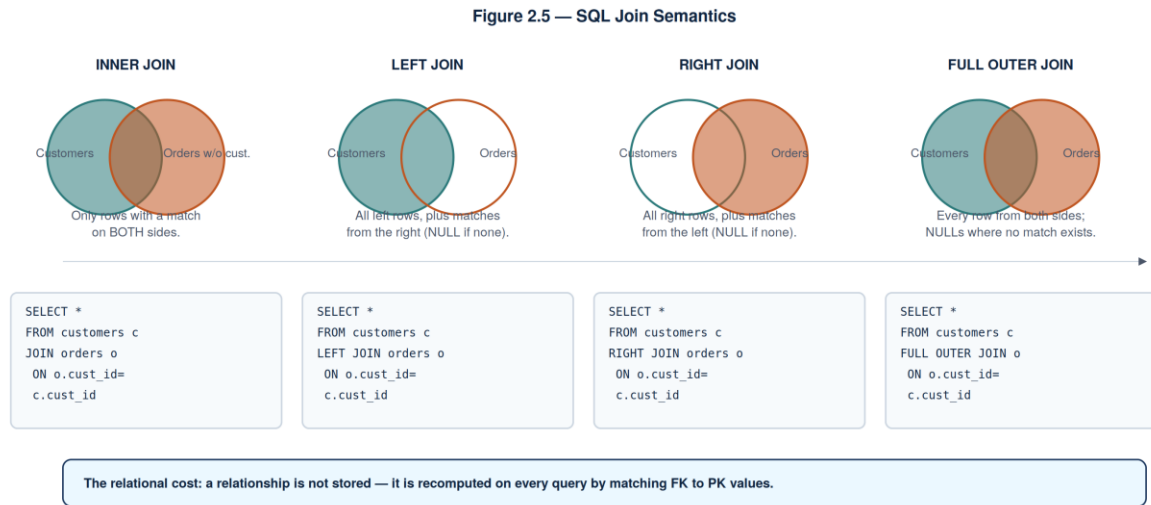


Figure 2.5 : INNER, LEFT, RIGHT, and FULL OUTER joins, with their SQL syntax.

A short worked example makes the mechanics concrete. Suppose we want every order line together with the customer's name and the product's title : a question that spans three tables:

```

SELECT c.first_name, c.last_name, p.name AS product, oi.quantity
FROM customers c
JOIN orders o    ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products p   ON p.product_id  = oi.product_id
WHERE c.city = 'Lisbon';

```

Notice what is happening structurally: the relationships between customers, orders, order items, and products are not stored anywhere as a single, retrievable thing. They exist only implicitly, as matching key values, and must be RECONSTRUCTED by the query planner every single time this question : or any question like it : is asked. For a handful of joins on well-indexed columns, modern query optimizers handle this extremely well. The trouble starts, as we will see in the next chapter, when the number of hops grows, or when the join path is not known in advance.

2.6 A quick performance vocabulary

Two terms recur throughout this tutorial and are worth defining precisely now:

Term	Definition
Index-free adjacency	A storage strategy (used by native graph databases) where each node physically stores direct references to its adjacent relationships, so traversing from one node to its neighbours is a constant-time pointer-chase, independent of total database size.
Join explosion	The phenomenon where the number of JOINS : and therefore query cost and query complexity : grows with the number of relationship “hops” a question requires, often combinatorially with data volume and fan-out.

2.7 Transactions and the ACID guarantees

One further relational property deserves explicit treatment before we move on, because it is the single biggest reason relational databases remain the default system of record even in graph-heavy architectures (a point developed fully in Section 10.1): the transaction, and the four guarantees traditionally summarised by the acronym ACID.

Guarantee	What it means	Why it matters
Atomicity	A transaction's writes either all happen or none do : there is no partial state visible to other readers.	Prevents an order being recorded as placed while its payment failed to record, or vice versa.
Consistency	A transaction can only move the database from one valid state (satisfying all constraints) to another.	A foreign key can never point at a row that doesn't exist, even mid-transaction.
Isolation	Concurrent transactions behave as if they ran one after another, not interleaved.	Two customers can't simultaneously purchase the last unit of inventory and both succeed.
Durability	Once a transaction commits, it survives a subsequent crash.	A confirmed payment is never silently lost if the database process crashes a second later.

Most native graph databases now offer ACID transactions as well : this is not, by itself, a reason to prefer relational storage. The point worth internalising instead is narrower and more practical: relational engines have FIFTY YEARS of production hardening, tooling, and institutional operating experience specifically around these guarantees at very high write-throughput, multi-row, multi-table transaction scale (think: an airline reservation system, or a bank's core ledger). This operational maturity gap : not a fundamental theoretical limitation of graph databases : is the practical reason this tutorial consistently recommends keeping high-throughput OLTP transactional workloads on the relational system and deriving the graph from it, as formalised in Section 10.1.

2.8 Views, materialized views, and stored procedures

Three further relational constructs are worth a brief, explicit mention here because each has a direct conceptual echo later in the tutorial. A VIEW is a saved, named SELECT query that can be queried as if it were a table : a virtual, always-live re-computation, with no data actually duplicated. A MATERIALIZED VIEW is the same idea but with the result physically stored and periodically refreshed, trading staleness for query speed. A STORED PROCEDURE is a piece of logic (sometimes substantial) executed inside the database engine itself, close to the data.

Each of these has a graph-world parallel introduced later in this tutorial: a view's “virtual, always-live” character reappears as the virtual/on-demand federation strategy in Section 7.5; a materialized view's “physically stored, periodically refreshed” trade-off reappears as the ETL/batch replication strategy in the same section; and a stored procedure's “logic executed close to the data” reappears in graph algorithm libraries executed server-side against the graph engine, covered in Chapter 14.

With this vocabulary and the mechanics of Section 2.5 in hand, we are ready to look directly at where the relational model starts to strain : which is the subject of Chapter 3.

3 Where the Relational Model Strains: Connected Data

The relational model is not being replaced because it is flawed : it is being complemented because certain classes of questions were never what it was optimised for. This chapter makes that argument precisely, because a clear-eyed understanding of WHY graph models help is what will let you decide, later, whether a given dataset actually benefits from conversion : not every relational database should become a knowledge graph.

3.1 The multi-hop question problem

Consider a question that is only mildly more complex than the one at the end of Chapter 2: “Which colleagues of my colleagues also follow a company that supplies one of my vendors?” This is not an exotic question : variants of it appear constantly in fraud detection (shared devices, shared addresses, shared payment instruments across seemingly unrelated accounts), recommendation engines (customers who bought what similar customers bought), supply chain risk (second- and third-tier supplier exposure), and social/professional networks (mutual connections). Figure 3.1 shows what answering it costs in table joins.

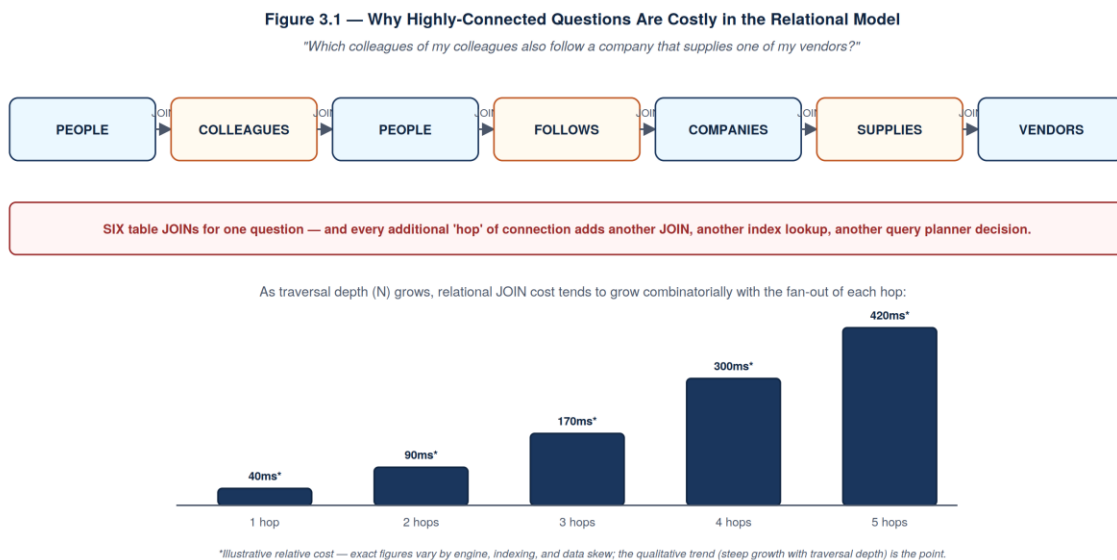


Figure 3.1 : A six-table join for one multi-hop question, and how relational cost tends to grow with traversal depth.

Three distinct problems compound here, and it is worth separating them because they are often conflated:

- **Query complexity** : the SQL itself becomes long, hard to write correctly, and hard to review : six or more JOINS in one query is a common threshold where readability and correctness both suffer.
- **Query planning cost** : the query optimizer must choose a JOIN order and JOIN strategy (nested loop, hash join, merge join) across an exponentially growing number of possible plans as tables multiply, and can choose badly on skewed data.
- **Unknown-depth traversal** : the hardest case: when you don't know in advance how many hops the answer requires : “all colleagues, however many degrees removed, who eventually connect back to me” : is not expressible as a fixed number of JOINS at all. SQL:1999 introduced recursive Common Table Expressions (WITH RECURSIVE) specifically to address this gap, and they work, but they are notoriously easy to write incorrectly (infinite loops, missing

termination conditions) and tend to perform poorly at depth because each recursive step still pays JOIN and lookup overhead.

A recursive CTE example, and its friction

To make the last point concrete, here is how you would find all indirect reports under a manager in a self-referencing employees table using standard SQL:

```
WITH RECURSIVE org_chart AS (
  SELECT employee_id, manager_id, 1 AS depth
  FROM employees WHERE manager_id = 42    -- anchor: direct reports
  UNION ALL
  SELECT e.employee_id, e.manager_id, oc.depth + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT * FROM org_chart ORDER BY depth;
```

This works, and it is a legitimate, well-supported SQL feature. But compare it, conceptually, to the equivalent Cypher pattern we will introduce properly in Chapter 8: `MATCH (m:Employee {id:42})<-[REPORTS_TO*]-(report) RETURN report`. The graph version expresses “follow this relationship type, any number of times” directly, as a first-class part of the query language's pattern-matching syntax, rather than as a general-purpose recursive construct repurposed for graph traversal.

3.2 Schema rigidity and the many-to-many tax

Every many-to-many relationship in a relational schema requires a junction (or “bridge”) table : an extra table whose sole purpose is to hold pairs of foreign keys. This is not wrong, but it does mean the conceptual relationship (“student enrolled in course”) is represented not as one thing, but as an entire extra table, with its own primary key, its own indexes, and its own row in every JOIN path that touches it. As the number of relationship TYPES in a domain grows, the number of junction tables grows with it, and the schema diagram increasingly looks like a web of relationship tables rather than a clean picture of the domain's entities.

Schema rigidity compounds this: adding a new relationship type to a relational schema means designing a new table (or a new column), writing a migration, and coordinating that migration across every application that touches the database. This is by design : it is what gives relational databases their safety : but it does mean the model resists organic, incremental extension. A graph model, as we'll see in Chapter 4, allows new relationship and property types to be added at write time, without a schema migration, which is valuable specifically in domains : like knowledge integration across many source systems : where the full shape of the data isn't known upfront.

3.3 Data silos and the integration problem

A subtler limitation is organisational rather than technical: most real companies do not have ONE relational database. They have dozens : a CRM, an ERP, a support ticketing system, a marketing database, a data warehouse : each with its own schema, its own notion of “customer,” and often its own primary keys for the same real-world entities. Relational technology gives you excellent tools for joining tables WITHIN one database, but very little help joining ACROSS databases with different schemas and no shared keys. Answering “show me everything we know about this customer, across every system” becomes a bespoke, brittle ETL and identity-resolution project for every pair of systems that needs to talk to each other.

This is precisely the problem knowledge graphs were designed to solve at scale

A knowledge graph's flexible, schema-light entity model : combined with entity resolution techniques covered in Chapter 7 : is naturally suited to integrating data FROM MULTIPLE relational (and non-relational) sources into one connected, queryable structure, without forcing every source system to first agree on a single shared relational schema. This is arguably the single most common reason organisations adopt knowledge graphs in practice : more common, in most industry surveys, than raw query performance.

3.4 What the relational model still does better

In fairness : and this matters for making good architecture decisions rather than chasing trends : relational databases retain clear advantages that this tutorial does not dispute:

- **Mature transactional guarantees** : decades of engineering behind ACID transactions, isolation levels, and crash recovery, at a level of maturity most graph databases are still catching up to for very high write-throughput OLTP workloads.
- **Tooling and talent depth** : SQL is the most widely known data language in the industry; BI tools, ORMs, and reporting stacks overwhelmingly assume a relational or SQL-like interface.
- **Predictable performance for tabular aggregation** : SUM, COUNT, GROUP BY over large flat tables : classic reporting and analytics workloads : remain a relational and columnar-store strength; graph databases are not, in general, built to be the fastest option for pure aggregate analytics over non-connected data.
- **Strong schema enforcement** : when your domain genuinely IS well-described by a fixed, rarely changing schema, relational constraints give you correctness guarantees a schema-light graph store does not provide out of the box (though property graph databases increasingly offer optional schema constraints, as we'll see in Chapter 7).

3.5 A note on consistency: why 'derived graph' implies 'eventual consistency'

One further limitation worth naming explicitly here, because it is a direct consequence of the architecture this tutorial will go on to recommend (Section 10.1): a knowledge graph derived from a relational source is not, and cannot be, transactionally synchronised with that source in the same instant a write occurs, unless the two are updated within the same distributed transaction : an approach very few production systems adopt, for the operational complexity reasons discussed further in Section 10.2. The practical consequence is that any derived knowledge graph architecture accepts EVENTUAL consistency with its source by design, and every application built against that graph must be written with that lag explicitly in mind, rather than assuming graph reads always reflect the very latest relational writes.

This is not a defect unique to knowledge graphs : it is the same trade-off underlying every read-replica, cache, search index, or data warehouse ever built on top of an authoritative relational source : but it is worth flagging this early, before Part III's conversion methodology, so it is treated as a designed-for property of the architecture from day one rather than discovered as a surprising bug once the first user notices a graph query returning slightly stale results.

The practical conclusion : and the position this entire tutorial is built on : is not “replace your relational database” but “add a knowledge graph where connectedness is the dominant access pattern, and derive it from your relational systems of record.” The rest of this tutorial builds the skills to do exactly that, starting with a proper grounding in graph data models in Chapter 4.

PART TWO

Graph Thinking & GraphQL

Chapters 4 – 5

4 Graph Data Models: Property Graphs and RDF

“Graph database” is often used as if it referred to a single technology, but in practice the industry has converged on two substantially different data models, each with its own family of products, query languages, and standards bodies. This chapter introduces both, because the conversion methodology in Part III applies to either : the choice between them is an architectural decision we’ll return to explicitly, not a foregone conclusion.

4.1 The Labeled Property Graph (LPG) model

The labeled property graph model : used by Neo4j, Amazon Neptune (in property graph mode), TigerGraph, Memgraph, and most of the products people mean when they casually say “graph database” : represents data as four ingredients: nodes, relationships, labels, and properties.

Figure 4.1 — The Labeled Property Graph Model

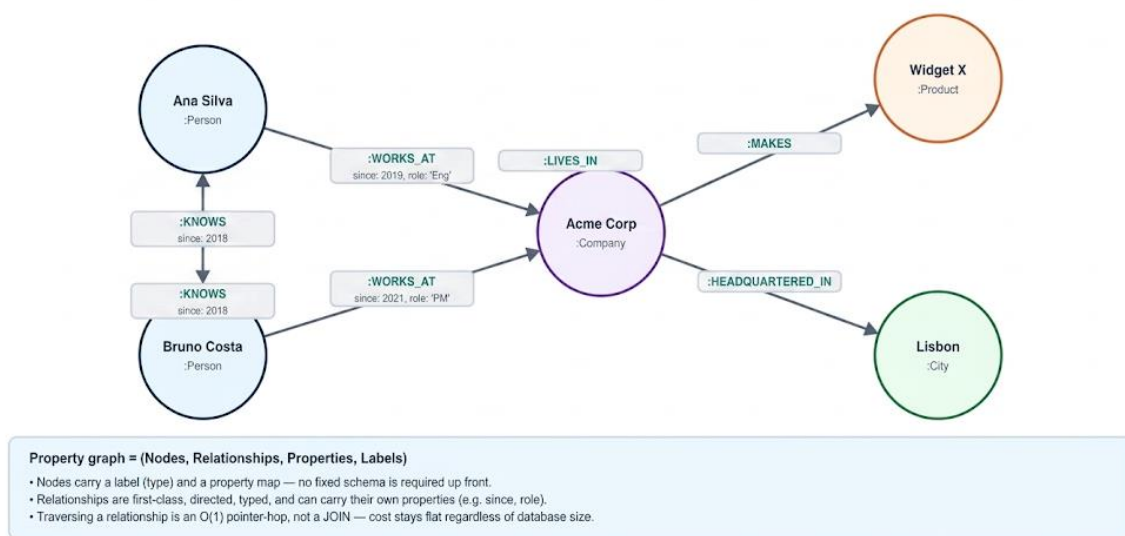


Figure 4.1 : The labeled property graph model: nodes and relationships both carry labels/types and property maps.

Concept	Definition	Relational rough-equivalent
Node	A single entity instance	A row
Label	A type tag on a node (a node may have several)	A table name
Relationship	A directed, typed connection between two nodes	A foreign key : but stored as a first-class, traversable object
Property	A key-value pair attached to a node OR a relationship	A column value

The detail that most surprises newcomers from the relational world is that RELATIONSHIPS can carry properties, not just nodes. In our example, the :WORKS_AT relationship between a person and a company carries its own since and role properties. In a relational schema this would typically require its own table (an “employment” table with employee_id, company_id, since, role) : in a property graph it is simply data attached to the edge itself. This single feature is what allows the many-to-many

“junction table with extra columns” pattern from Section 2.4 to collapse cleanly into one edge, which we will formalise as a mapping rule in Chapter 7.

Property graphs are schema-optional by default: a node's labels and properties are determined at write time, and different nodes with the same label are not required to have identical property sets, though in practice well-run systems apply consistent conventions and increasingly use optional schema/constraint layers (unique constraints, property existence constraints, and in some products, full type schemas) to recover some of the safety a relational schema provides by default.

4.2 The RDF / triple model

The Resource Description Framework (RDF) takes a different, more atomic approach, standardised by the W3C originally for the Semantic Web. Every fact is expressed as a triple: subject – predicate – object : for example, “Ana Silva – worksAt – Acme Corp.” A knowledge graph, in RDF terms, is simply a large set of such triples. Figure 4.2 shows the anatomy of a triple and how a handful of triples combine into a connected graph structure.

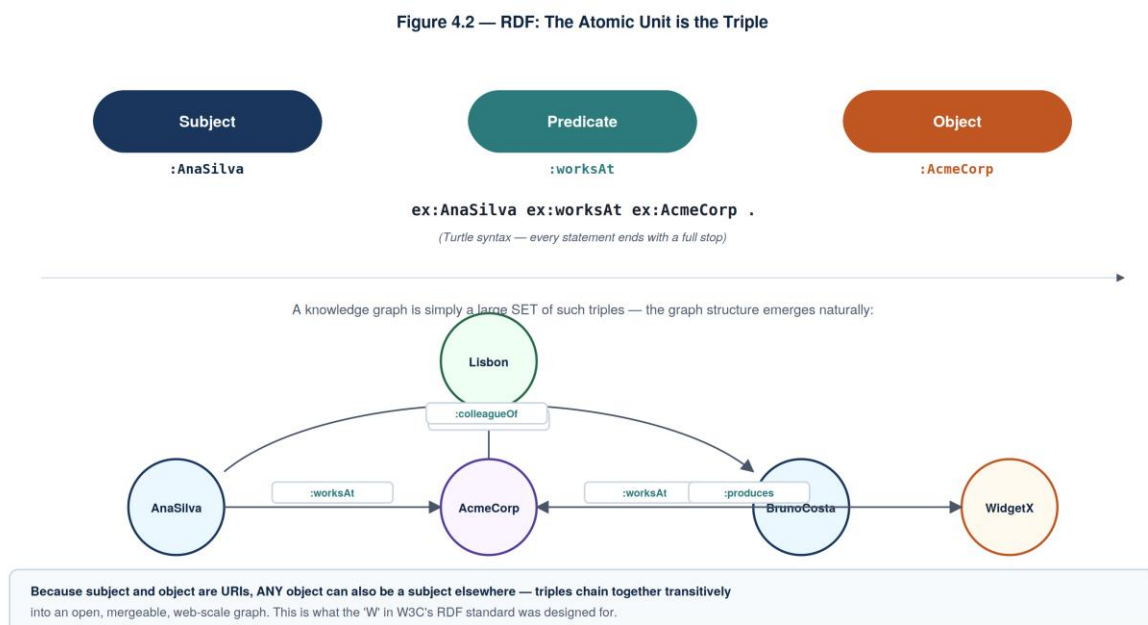


Figure 4.2 : RDF's atomic unit is the (subject, predicate, object) triple; graphs emerge from sets of triples that share nodes.

Several RDF-specific ideas are worth calling out explicitly:

- **URIs as global identifiers** : subjects and predicates (and often objects) are identified by URIs : the same kind of globally unique identifier used for web addresses : which means data from entirely different organisations can, in principle, be merged simply by set-union of their triples, since a URI like <https://schema.org/Person> means the same thing everywhere it's used.
- **No native relationship properties** : unlike a property graph edge, a plain RDF triple cannot carry its own properties (you cannot directly say a worksAt relationship “began in 2019” on the relationship itself). This is worked around via reification or, more commonly today, RDF-star (RDF*), an extension that allows statements ABOUT statements.
- **A formal semantics via OWL** : the Web Ontology Language (OWL) layers formal, machine-checkable semantics on top of RDF : subclass hierarchies, property characteristics (transitive, symmetric, inverse), and cardinality restrictions : enabling automated logical inference (“reasoning”), discussed further in Chapter 6.

- **SPARQL as the query language** : the W3C-standardised query language for RDF, covered in depth in Chapter 8.

4.3 Property graph vs. RDF: a practical comparison

Dimension	Property Graph (LPG)	RDF
Atomic unit	Node / Relationship (each with property maps)	Triple (subject-predicate-object)
Edge properties	Native, first-class	Requires RDF-star or reification
Standardisation	No single W3C standard; vendor-specific until GQL/openCypher	W3C standard (RDF, OWL, SPARQL, SHACL)
Typical query language	Cypher, Gremlin, or the new ISO GQL standard	SPARQL
Formal reasoning / inference	Limited, usually via added tooling	First-class, via OWL and rule engines
Common products	Neo4j, Neptune (LPG mode), TigerGraph, Memgraph	GraphDB, Stardog, Blazegraph, Neptune (RDF mode)
Typical adoption driver	Application-facing graph features (recommendations, fraud, search)	Data integration, linked open data, regulated/scientific domains needing formal semantics

Which one should you target?

As a strong rule of thumb: choose a property graph when the priority is application-facing traversal performance and developer ergonomics (recommendation engines, fraud graphs, 360° customer views), and choose RDF when the priority is formal data integration, standards compliance, or automated logical inference over a controlled vocabulary (life sciences, government linked-data, complex regulatory taxonomies). Many large organisations end up running both, for different workloads, connected via export/import bridges. This tutorial's conversion methodology (Chapter 7) is written to apply to either target with only minor adaptation, and we call out the differences explicitly where they arise.

4.4 A worked mini-example in both models

To fix the comparison concretely, here is the SAME fact : “Ana Silva has worked at Acme Corp since 2019 as an Engineer” : expressed first as a property graph creation statement (Cypher), then as RDF (Turtle syntax), then as RDF-star (to add the edge property RDF lacks natively).

As a property graph (Cypher)

```
CREATE (a:Person {name: 'Ana Silva'})
CREATE (c:Company {name: 'Acme Corp'})
CREATE (a)-[:WORKS_AT {since: 2019, role: 'Engineer'}]->(c)
```

As RDF (Turtle) : note the workaround needed for edge properties

```
ex:AnaSilva a foaf:Person ;
```

```
foaf:name "Ana Silva" .
ex:AcmeCorp a org:Organization ;
    org:name "Acme Corp" .
ex:AnaSilva ex:worksAt ex:AcmeCorp .      # the plain fact
# 'since' and 'role' need a reified node or RDF-star to attach:
```

As RDF-star (attaching properties directly to the statement)

```
<< ex:AnaSilva ex:worksAt ex:AcmeCorp >> ex:since 2019 ;
    ex:role "Engineer" .
```

4.5 How graph models relate to document and key-value stores

It is worth briefly situating graph models against two other popular non-relational (“NoSQL”) database families, since the term “NoSQL” is often used as if it named one alternative to the relational model, when in fact it spans several structurally very different approaches.

Model	Best-fit shape of data	Relationship handling
Key-value store	Simple lookups by a single key (session data, caching)	None : no query-time relationship traversal at all
Document store	Self-contained, hierarchical records (a JSON-like document per entity)	Embedding (nesting related data inside the document) or manual reference fields, resolved in application code : no native traversal
Column-family store	Very large-scale, denormalized tabular data optimised for specific known query patterns	Denormalized/duplicated ahead of time to avoid joins; not designed for ad hoc traversal
Graph (property or RDF)	Densely interconnected entities where the RELATIONSHIPS are as important as the entities	Native, first-class, traversable : the central design point of the whole model

The practical distinction that matters most: document stores handle ONE-TO-MANY relationships reasonably well by embedding (a customer document can embed its own order history directly), but handle MANY-TO-MANY and multi-hop relationships poorly, for essentially the same reason relational databases do : those relationships still have to be resolved via a second query and joined in application code, since there is no native traversal operation. A graph database is the only one of these four families purpose-built for exactly that case.

With both foundational graph models now defined, the next chapter turns to GraphQL : a query language and API layer that, despite the name, is a distinct concept from a knowledge graph and can sit in front of either model, or indeed in front of a relational database directly.

5 GraphQL: A Typed Query Language for Connected APIs

GraphQL was created at Facebook in 2012 and open-sourced in 2015 to solve a very specific, practical problem: REST APIs, with their fixed endpoint shapes, forced mobile clients to either over-fetch (downloading far more data than a screen needed) or make many round trips (one request per related resource) to assemble a single view. GraphQL's answer was to let the CLIENT describe exactly the shape of data it needs, in a single request, regardless of how many underlying resources or relationships that shape spans.

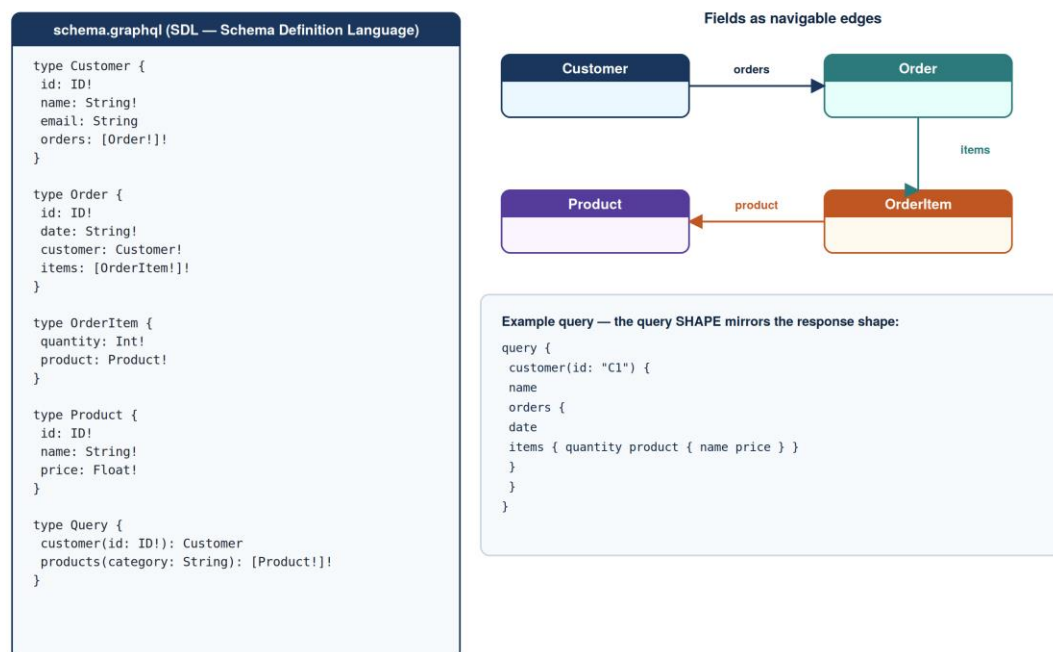
The single most important distinction in this chapter

GraphQL is a QUERY LANGUAGE AND EXECUTION ENGINE, not a database or a storage model. It says nothing about how or where data is stored. This matters enormously for this tutorial: GraphQL is not an alternative to a knowledge graph, and building a GraphQL API in front of your relational database does not, by itself, give you a knowledge graph. The two concepts solve different layers of the problem, and as we will see in Chapter 7 : they compose extremely well together.

5.1 The type system and Schema Definition Language (SDL)

Every GraphQL API is backed by a strongly typed schema, written in Schema Definition Language (SDL), that declares every type, every field on every type, and the special Query, Mutation, and Subscription root types that define what operations are possible. Figure 5.1 shows a schema for our familiar e-commerce example, alongside the implied graph of navigable fields it defines.

Figure 5.1 — GraphQL Schema: Types as a Typed Graph of Fields



GraphQL exposes a typed, traversable graph OVER existing data sources — SQL, REST APIs, or a native graph database.

Figure 5.1 : A GraphQL SDL schema and the field graph it exposes for client traversal.

Notice the exclamation marks: String! means “non-null String” : GraphQL's type system distinguishes nullable from non-nullable at the schema level, which is stricter, by default, than most SQL column

definitions (where nullability is opt-in via NOT NULL). Square brackets, as in [Order!]!, denote a list : here, a non-null list of non-null Order objects.

5.2 Queries, mutations, and subscriptions

GraphQL defines three root operation types, each mapping onto a different kind of client intent:

- **Query** : read-only fetch operations. The client specifies the exact shape of the response by nesting field selections, and the server returns JSON matching that exact shape : nothing more, nothing less.
- **Mutation** : operations that change server-side state (create, update, delete). By GraphQL convention, mutations return the modified object so the client can immediately see the result of its own write without a follow-up query.
- **Subscription** : a long-lived operation, typically over WebSockets, that pushes updates to the client whenever a specified event occurs on the server : used for real-time features.

A read query and its exact response shape, using our example schema:

```
query {
  customer(id: "C1") {
    name
    orders {
      date
      items { quantity product { name price } }
    }
  }
}
```

```
{
  "data": {
    "customer": {
      "name": "Ana Silva",
      "orders": [
        { "date": "2026-03-14",
          "items": [ { "quantity": 2, "product": { "name": "Widget X", "price": 9.99 } } ] }
      ]
    }
  }
}
```

And a mutation:

```
mutation {
  createOrder(customerId: "C1", items: [{ productId: "P9", quantity: 3 }]) {
    id
    total
  }
}
```

5.3 Resolvers: where the graph traversal actually happens

The mechanism that makes GraphQL work is the resolver : a function, one per field in the schema, responsible for fetching that specific field's value. When a query nests fields (customer → orders → items → product), the GraphQL execution engine calls each field's resolver in turn, passing the parent

field's resolved value down. This is what allows a single GraphQL query to transparently pull data from several completely different backend systems : Figure 5.2 shows this explicitly, with resolvers fanning out to a SQL database, a native knowledge graph, and a legacy REST microservice within one request.

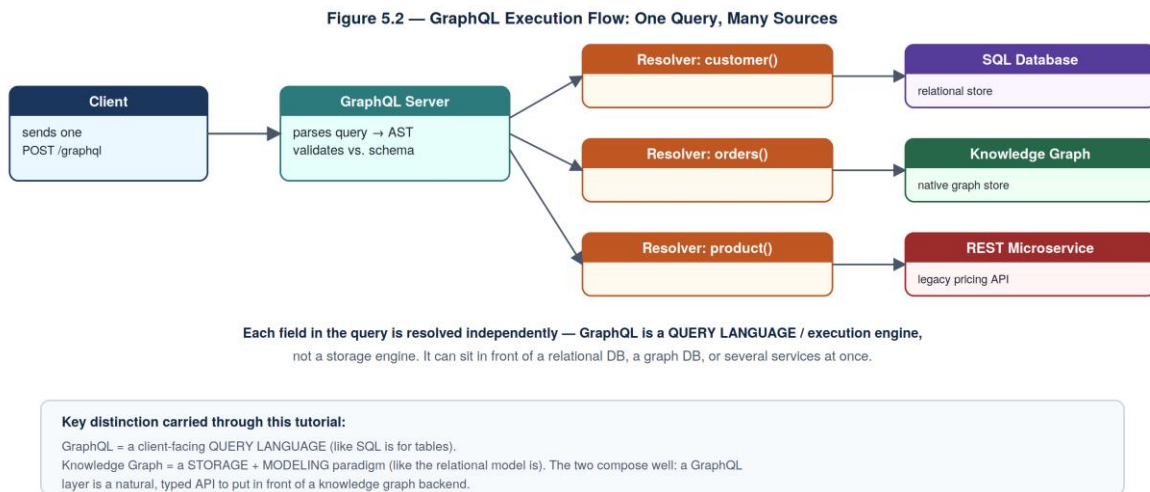


Figure 5.2 : One GraphQL query, resolved field-by-field, can span multiple heterogeneous backends.

A minimal JavaScript resolver map illustrating the pattern (using a library such as Apollo Server or GraphQL Yoga):

```
const resolvers = {
  Query: {
    customer: (_, { id }, ctx) => ctx.sqlDb.findCustomer(id),
  },
  Customer: {
    orders: (customer, _, ctx) => ctx.sqlDb.findOrdersByCustomer(customer.id),
  },
  Order: {
    items: (order, _, ctx) => ctx.sqlDb.findItemsByOrder(order.id),
  },
  OrderItem: {
    product: (item, _, ctx) => ctx.graphDb.getProduct(item.productId), // a different backend!
  },
};
```

5.4 The N+1 problem, and why it rhymes with the JOIN problem

A naive resolver implementation, exactly like the one above, has a serious performance trap: fetching a list of 50 orders, then calling the items resolver once PER order, then calling the product resolver once per item, results in a cascade of individual database round trips : the classic “N+1 query problem.” This is structurally the same underlying issue as relational JOIN explosion from Section 3.1 : a naive, per-object traversal strategy that doesn't batch : just manifesting at the API layer instead of the SQL layer.

The standard fix is the DataLoader pattern: batch and cache all requests for a given resolver within a single tick of the event loop, so 50 individual “get items for order X” calls become one “get items for orders [X1...X50]” batched call. This is directly analogous to how a relational query planner batches a JOIN into a single hash-join pass instead of running the inner query once per outer row : the same optimisation principle, reapplied at a different layer of the stack.

```
const productLoader = new DataLoader(async (productIds) => {
  const products = await ctx.graphDb.getProductsBatch(productIds); // ONE call for all ids
  return productIds.map(id => products.find(p => p.id === id));
});
// resolver becomes: product: (item) => productLoader.load(item.productId)
```

5.5 GraphQL Federation and schema stitching

Larger organisations rarely want one monolithic GraphQL server. Apollo Federation and similar approaches let multiple teams each own a smaller GraphQL subgraph : e.g. a Customers subgraph and an Orders subgraph : which are composed at a gateway layer into one unified graph the client queries. This pattern is directly relevant to knowledge graph architecture: it is common, and often architecturally cleaner, to expose a knowledge graph as one federated subgraph alongside subgraphs backed by relational systems, rather than forcing every data source to be physically merged into the graph database first.

5.6 Why GraphQL and knowledge graphs pair so naturally

Three structural properties make GraphQL an unusually good API layer specifically for knowledge graphs (more so than for flat relational data, arguably):

1. The query SHAPE mirrors graph traversal shape. A nested GraphQL selection set (customer { orders { items } }) is structurally identical to a graph traversal pattern (Customer)-[:PLACED]->(Order)-[:CONTAINS]->(Item) : mapping one onto the other in a resolver is close to mechanical, whereas mapping it onto SQL requires re-deriving JOIN logic in every resolver.
2. Both are naturally typed graphs of entities and relationships : GraphQL's type system (types, fields, non-null/list modifiers) and a knowledge graph's schema/ontology layer (classes, properties, cardinality constraints) describe the same kind of structure, just in different surface syntax.
3. GraphQL's resolver indirection is exactly the abstraction needed to present a knowledge graph to clients without exposing the underlying query language (Cypher/SPARQL) or the underlying storage engine, giving API consumers a stable, versioned, self-documenting contract regardless of what graph technology sits behind it.

5.7 Security: query complexity, depth limiting, and rate limiting

GraphQL's greatest strength : letting the client construct an arbitrarily nested query : is also a genuine security concern that a REST API's fixed endpoints never faced: nothing stops a malicious or buggy client from requesting a query nested dozens of levels deep, or requesting a field that fans out into millions of records, turning one HTTP request into a denial-of-service attack against your backend. Production GraphQL servers defend against this with several complementary techniques:

- **Depth limiting** : reject any query whose selection-set nesting exceeds a configured maximum depth, preventing pathological deeply-nested queries outright.
- **Query complexity analysis** : assign a cost to each field (based on expected result size and computational cost) and reject queries whose total estimated cost exceeds a budget : more precise than a flat depth limit, since it accounts for fan-out (a field returning a list of 10,000 items costs more than one returning a single object).
- **Persisted queries** : in high-security environments, restrict the API to a pre-approved allowlist of known query shapes (typically hashed and registered at build time), rejecting any query not

on the list : trading away ad hoc query flexibility for a much smaller attack surface, a trade-off many mobile-app backends make deliberately.

- **Standard rate limiting** : applied per-client exactly as it would be for a REST API, since GraphQL doesn't change the need for request-volume throttling.

```
// Example: graphql-query-complexity configuration (JavaScript)
const complexityLimit = createComplexityLimitRule(1000, {
  scalarCost: 1,
  objectCost: 2,
  listFactor: 10, // fields returning a list cost 10x their child cost : penalises fan-out
  onCost: (cost) => console.log('Query cost:', cost),
});
```

These concerns compound directly with the N+1/DataLoader discussion in Section 5.4: an unbounded, un-batched deep query is both a latency problem AND a security problem, and the DataLoader pattern combined with complexity limiting addresses both at once : batching controls the cost of a legitimate complex query, while complexity limiting rejects illegitimate ones outright before they run.

With relational foundations (Part I) and both graph and GraphQL foundations (Part II) now established, we turn in Part III to knowledge graphs proper : starting, in Chapter 6, with the schema and ontology layer that gives a knowledge graph its meaning, before tackling the full relational-to-graph conversion methodology in Chapter 7.

PART THREE

Knowledge Graphs & Conversion

Chapters 6 – 7

6 Principles of Knowledge Graphs

We now have both halves of the vocabulary we need: the relational model (Part I) and the graph data models that will replace or augment it (Chapter 4). This chapter defines, precisely, what elevates a “graph database full of data” into a knowledge graph : because the term is used loosely in industry, and the distinction matters directly for the conversion methodology in the next chapter.

6.1 What makes a graph a KNOWLEDGE graph

A bare property graph or triple store becomes a knowledge graph when three additional properties are present, layered on top of the raw node/edge or triple structure:

- **An explicit schema layer (ontology)** : a formal description of what TYPES of entities and relationships exist, and the rules governing them : independent of any particular data instance.
- **Semantic meaning attached to identifiers** : types and relationships are not arbitrary strings but carry agreed, often shared, meaning : ideally reusing standard vocabularies (schema.org, FOAF, industry-specific ontologies) so the graph can integrate with, or be understood by, other systems.
- **Support for inference / derived facts** : the ability to derive NEW facts that were never explicitly stored, by applying logical rules over the schema (e.g. “if A reports to B, and B reports to C, then A is in C's reporting chain”) : discussed in Section 6.4.

By this definition, a Neo4j database with a handful of arbitrarily named node labels and no documented meaning is “just” a property graph. The same database, with a documented, versioned ontology defining what :Person, :Organization and :WORKS_AT formally mean, what constraints apply to them, and : optionally : what can be logically inferred from them, is a knowledge graph. The data structure can be identical; the difference is entirely in the schema and semantic layer wrapped around it.

6.2 T-Box and A-Box: separating schema from data

Description logic : the formal foundation beneath OWL and much of knowledge graph theory : draws a sharp line between the Terminological Box (T-Box: the schema : classes, properties, and the rules that relate them) and the Assertional Box (A-Box: the actual data : specific individuals and the facts about them). Figure 6.1 makes this split concrete.

Figure 6.1 — Two Layers of a Knowledge Graph: Ontology (T-Box) and Instance Data (A-Box)

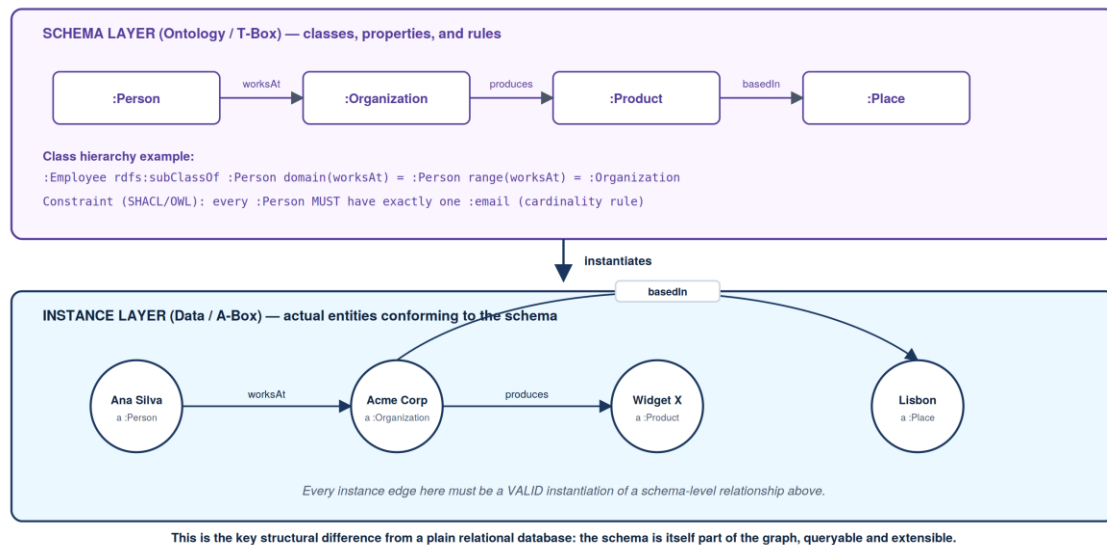


Figure 6.1 : The T-Box (ontology/schema) and A-Box (instance data) layers of a knowledge graph.

This T-Box/A-Box split is the single most important conceptual difference from a plain relational database, where the schema (DDL : CREATE TABLE statements) and the data live in entirely separate, disconnected representations: you cannot query the schema using the same SQL you use to query the data. In a knowledge graph, the T-Box is itself part of the graph (or a closely linked graph) and can be queried, extended, and reasoned over using the same query language as the data : a property that becomes very useful when a graph needs to evolve without a disruptive migration.

6.3 Building an ontology: classes, properties, constraints

An ontology, in practice, is built from a small number of recurring constructs, whether expressed in OWL (for RDF-based graphs) or a property-graph schema/constraint language:

Construct	Purpose	Example
Class	A category of entity	<code>:Person</code> , <code>:Organization</code> , <code>:Product</code>
Subclass hierarchy	Specialisation between classes	<code>:Employee rdfs:subClassOf :Person</code>
Object property	A relationship between two entities	<code>:worksAt</code> (domain <code>:Person</code> , range <code>:Organization</code>)
Datatype property	A relationship from an entity to a literal value	<code>:email</code> (domain <code>:Person</code> , range <code>xsd:string</code>)
Cardinality constraint	How many values a property may take	every <code>:Person</code> has exactly 1 <code>:dateOfBirth</code>
Domain / range	The allowed subject / object types for a property	<code>domain(:worksAt)=:Person</code> , <code>range(:worksAt)=:Organization</code>

For property graphs, the equivalent constraints are increasingly expressed using SHACL (Shapes Constraint Language, a W3C standard originally for RDF but with growing property-graph tooling support) or vendor-native schema features. Here is a SHACL shape enforcing that every `:Person` must have exactly one email:

```
ex:PersonShape a sh:NodeShape ;
  sh:targetClass :Person ;
  sh:property [
    sh:path :email ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:datatype xsd:string ;
  ] .
```

6.4 Inference and reasoning: getting facts you never stored

The payoff for building a proper ontology is inference: the ability to derive new, correct facts by applying logical rules over the schema, without anyone having explicitly written those facts into the graph. Two categories matter in practice:

Taxonomic inference (subclass reasoning)

If `:Employee` is declared a subclass of `:Person`, then any entity asserted to be an `:Employee` is AUTOMATICALLY inferable as also being a `:Person`: a query for “all `:Person` entities” will correctly include employees, even though no triple explicitly says “Ana Silva is a `:Person`” if it only ever said “Ana Silva is an `:Employee`.”

Property characteristic inference

OWL lets you declare a property transitive (`:partOf`), symmetric (`:marriedTo`), or the inverse of another property (`:employs` is the inverse of `:worksAt`). A reasoner can then derive, for example, that if a `:Screw` `:partOf` a `:Bracket`, and the `:Bracket` `:partOf` an `:Engine`, then the `:Screw` `:partOf` the `:Engine`: transitively, without that fact ever being explicitly asserted. This is directly useful for bill-of-materials, organisational hierarchy, and geographic containment use cases.

A note of caution on reasoning

Full OWL reasoning (especially at OWL-DL or OWL-Full expressivity) can be computationally expensive and, in pathological schema designs, undecidable. Most production systems deliberately restrict themselves to a decidable, efficient profile (OWL 2 RL is a common choice, since it maps cleanly onto rule-based / Datalog-style forward-chaining reasoners) or skip formal OWL reasoning altogether and implement the small set of inference rules they actually need directly as graph queries or materialised views, refreshed on a schedule. Reach for full reasoning when your domain genuinely needs open-ended, sound logical inference (life sciences ontologies are the classic case); reach for targeted, explicit rules when you know exactly what derived facts you need.

6.5 Named graphs, provenance, and multi-source integration

Because knowledge graphs are frequently built by integrating multiple source systems (Section 3.3), tracking WHERE each fact came from: its provenance: becomes important for trust, debugging, and conflict resolution when two sources disagree. RDF supports this natively via named graphs (a fourth element added to the subject-predicate-object triple, turning it into a “quad,” identifying which graph/source the triple belongs to). Property graphs typically achieve the same effect via a convention: a source or provenance property attached to each node and relationship, e.g. `{source: 'CRM', loaded_at: '2026-08-01'}`.

6.6 Vocabulary and standards worth knowing

Standard	Purpose
RDF (Resource Description Framework)	The W3C data model: facts as subject-predicate-object triples.
RDFS (RDF Schema)	A lightweight vocabulary for classes and subclass/subproperty hierarchies.
OWL (Web Ontology Language)	A richer ontology language supporting formal, decidable-profile reasoning.
SHACL (Shapes Constraint Language)	Declarative validation constraints over RDF (and increasingly property graph) data.
SKOS (Simple Knowledge Organization System)	A lighter-weight standard for taxonomies, thesauri, and controlled vocabularies : often a gentler starting point than full OWL.
schema.org	A widely reused, general-purpose shared vocabulary of common entity types (Person, Organization, Product, Event...) originally built for search-engine markup.

6.7 Building a new ontology vs. reusing an existing one

A question worth deciding explicitly and early, before Chapter 7's mapping work begins: should your target ontology be built entirely from scratch, or should it reuse an established, standard vocabulary wherever one already covers your domain? The honest answer is usually “both, in layers” : reuse a standard vocabulary for the common, general concepts your domain shares with the wider world (a :Person, an :Organization, an :Address are unlikely to need bespoke definitions), and extend it with your own custom classes and properties for the specific, proprietary concepts that make your domain distinctive (a :LoyaltyTier, a :WarrantyClaim).

Reuse an existing vocabulary when	Build custom when
The concept is genuinely general-purpose and widely shared (people, organizations, places, events, products)	The concept is specific and proprietary to your domain or business (internal risk scores, custom workflow states)
You expect to integrate or exchange data with external parties who already use that vocabulary	The concept has no reasonable external standard, or adopting one would force an awkward, lossy fit
A mature, well-maintained standard already exists (schema.org, FOAF, ICD-10, industry-specific standards)	Speed of initial :Modelling matters more than long-term interoperability for this specific concept

A common, effective pattern in practice: define your own namespace (e.g. ex: for internal concepts) but declare explicit equivalence or subclass relationships from your custom classes back to standard vocabulary classes wherever a genuine correspondence exists : giving you the best of both, full control over your domain-specific :Modelling while remaining interoperable with systems that only understand the standard vocabulary:

```
ex:Customer rdfs:subClassOf schema:Person .
ex:loyaltyTier rdfs:subPropertyOf schema:memberOf .
```

6.8 Common ontology design patterns

A handful of recurring :Modelling shapes appear across almost every ontology, regardless of domain, and recognising them by name speeds up design discussions considerably:

N-ary relations

OWL and plain RDF triples are inherently BINARY : a triple relates exactly two things. :Modelling a relationship that genuinely involves three or more parties at once (“Ana played role Engineer at Acme from 2019 to 2021” : person, company, role, AND a date range, all at once) requires the N-ARY RELATION pattern: introduce an intermediate node representing the relationship itself, with each party connected to it individually. This is the RDF-world version of the same idea introduced for star-schema fact tables in Appendix G.4.

Qualified cardinality restrictions

Beyond a simple “exactly one” or “at least one” cardinality constraint (Section 6.3), OWL supports QUALIFIED cardinality restrictions : “exactly one :hasStatus relationship whose value is specifically of type :ActiveStatus” : useful when a node can have several relationships of the same TYPE but only one satisfying some additional condition should be allowed.

Value partitions (controlled enumeration as classes, not literals)

Rather than :Modelling a status as a plain string literal property (“status: 'active'”), a value partition models each possible value as its OWN small class or individual (:ActiveStatus, :InactiveStatus), connected via an object property. This trades simplicity for the ability to attach further metadata to the status values themselves (a description, a display color, a set of allowed transitions) and to reason over them : worth the added complexity specifically when those status values carry meaningful structure of their own, and overkill otherwise.

With the schema/ontology layer now well defined, we have every concept needed to tackle the central chapter of this tutorial: a complete, practical methodology for converting a relational database into a knowledge graph, covered next in Chapter 7.

7 Converting a Relational Database into a Knowledge Graph

This is the chapter the rest of the tutorial has been building toward. Everything in Parts I and II was chosen specifically to be reused here: relational structure (Chapter 2), where it strains (Chapter 3), the graph models we are converting into (Chapter 4), and the schema/ontology layer that gives the resulting graph meaning (Chapter 6). We now assemble all of it into a complete, repeatable methodology.

7.1 Overview: the conversion is a mapping problem, not a rewrite

The central insight that makes this tractable is that converting a relational schema to a graph is NOT a from-scratch redesign exercise. A well-normalized relational schema (Section 2.3) already encodes almost all of the entity and relationship structure a graph model needs : normalization and graph :Modelling were pulling in a related direction all along, as noted in Chapter 2. The job, in the large majority of cases, is to apply a small, fixed set of mechanical mapping rules, table by table and constraint by constraint, and only reach for genuine redesign in the handful of situations (covered in Section 7.7) where the relational and graph models diverge structurally.

7.2 Step 1 : Schema analysis and classification

Before any mapping can happen, every table in the source schema needs to be classified into one of three roles. This classification drives every subsequent decision, so it is worth doing carefully and explicitly (many teams do this as a literal spreadsheet, one row per table, before writing any mapping code).

Table role	How to recognise it	What it becomes
Entity table	Has its own single-column primary key; represents a real-world “thing” (customer, product, order)	A node label
Pure junction table	Composite primary key made ENTIRELY of two (or more) foreign keys, no other meaningful attributes besides those FKs	Collapses into a direct relationship : the table itself disappears
Rich junction / associative table	Composite key of FKs PLUS extra descriptive columns (a grade, a quantity, a timestamp)	Becomes a relationship WITH properties : again, the table disappears, but its extra columns survive as edge properties
Lookup / reference table	Small, mostly static set of rows (statuses, categories, country codes) referenced by many other tables	Usually becomes either a small set of nodes (if the categories themselves need to be queried/related) or collapses into a simple string/enum property (if they are purely descriptive labels with no relationships of their own)

7.3 Step 2 : Apply the core mapping rules

With every table classified, the actual field-by-field and constraint-by-constraint translation follows a small set of rules. Figure 7.1 is the reference table this tutorial recommends printing out and keeping

next to you during a real migration : every one of the relational constructs from Chapter 2 has an exact counterpart here.

Figure 7.1 — Mapping Rules: Relational Constructs → Graph Constructs

Relational construct	Graph construct	Example
A TABLE	NODE LABEL	CUSTOMERS → (:Customer)
A ROW	A NODE	row(id=7, 'Ana') → (:Customer {id:7, name:'Ana'})
A COLUMN (scalar)	NODE PROPERTY	customers.email → (:Customer).email
PRIMARY KEY	NODE IDENTITY	customer_id=7 → :Customer {id:7}
FOREIGN KEY (N:1)	RELATIONSHIP / edge	orders.customer_id → (:Order)-[:PLACED_BY]-> (:Customer)
JOIN / JUNCTION TABLE	DIRECT RELATIONSHIP	STUDENT_COURSE → (:Student)-[:ENROLLED_IN] ->(:Course)
Extra columns on junction	EDGE PROPERTIES	...COURSE.grade → [:ENROLLED_IN {grade:'A'}]
SELF-REFERENCING FK	SELF-REF. EDGE	categories.parent_id → (:Category)-[:CHILD_OF] ->(:Category)

Figure 7.1 : The complete relational-to-graph mapping rule set.

Two rules deserve deeper treatment because they are where most real-world mapping mistakes happen: the foreign-key-to-relationship rule, and the junction-table-collapse rule.

7.3.1 Foreign keys become directed, typed relationships

A foreign key column doesn't just become “a relationship” generically : it becomes a **DIRECTED** relationship, and choosing its direction and its name is a :Modelling decision, not a mechanical one. The column `orders.customer_id` could reasonably become `(:Order)-[:PLACED_BY]->(:Customer)` or `(:Customer)-[:PLACED]->(:Order)` : both are structurally valid (most graph databases support efficient traversal in either direction regardless of which way the edge was created), but they read differently and different query patterns will favour one phrasing over the other. A useful default heuristic: name and direct the relationship the way a domain expert would naturally SAY the sentence out loud : “a customer PLACES an order” reads more naturally than “an order IS-PLACED-BY a customer,” so prefer `(:Customer)-[:PLACED]->(:Order)`.

7.3.2 Collapsing junction tables : the single highest-value transformation

Of every rule in this chapter, collapsing many-to-many junction tables into direct, property-bearing relationships is the one that most visibly demonstrates WHY a graph model helps : it is usually the single biggest reduction in query complexity a migration produces. Figure 7.2 walks through the transformation explicitly, before and after, including the resulting query simplification.

Figure 7.2 — Collapsing a Many-to-Many Junction Table into a Direct Edge

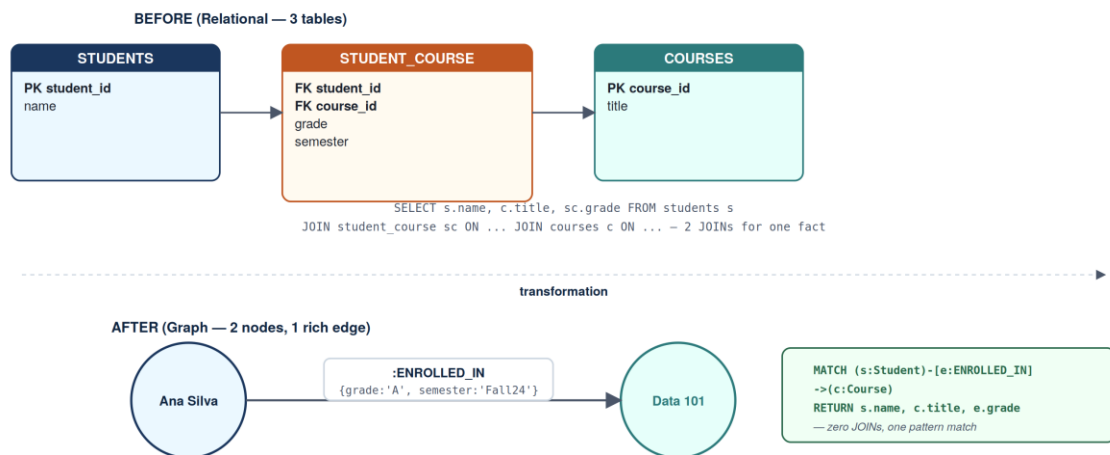


Figure 7.2 : A many-to-many junction table collapses into one relationship with properties; two JOINS become zero.

The general pattern, worth memorising: any table whose primary key is a composite of exactly the foreign keys of two other entity tables is a candidate for collapse. Any additional columns on that junction table (grade, quantity, timestamp, weight) become properties on the resulting relationship, not properties on either node : they describe the RELATIONSHIP, not either party to it.

When NOT to collapse a junction table

If the junction table has its own primary key that OTHER tables reference (i.e., something else has a foreign key pointing at the junction table itself, not at either of the two entities it joins), it is not a pure junction table : it is really its own entity with two relationships to other entities (commonly seen with things like 'enrollment' records that themselves have associated payments or certificates). In that case, keep it as its own node rather than collapsing it, and give it two outgoing relationships instead of trying to force it into a single edge.

7.4 Step 3 : Choose a mapping mechanism and write the mapping

Rules on paper need to become executable transformation logic. Three broad mechanisms are used in practice, in increasing order of formality:

Option A : Custom ETL scripts

The most common approach for one-off or moderately complex migrations: hand-written scripts (Python, Java, Node.js) that query the relational source and issue graph-creation statements. Maximum flexibility, minimum ceremony, but the mapping logic is implicit in code rather than declared as data : harder to audit and version at scale.

```

# Python + psycopg2 + neo4j-driver : mapping CUSTOMERS -> (:Customer) nodes
with pg_conn.cursor() as cur, neo4j_session() as session:
    cur.execute("SELECT customer_id, first_name, last_name, email, city FROM customers")
    for row in cur.fetchall():
        session.run(
            "MERGE (c:Customer {id: $id}) "
            "SET c.name = $name, c.email = $email, c.city = $city",
            id=row.customer_id,
            name=f"{row.first_name} {row.last_name}",

```

```
email=row.email, city=row.city,
)
```

Note the use of MERGE rather than CREATE : MERGE is idempotent (it creates the node only if a matching one doesn't already exist, otherwise it matches the existing one), which makes the script safely re-runnable : an essential property for any migration script you expect to run more than once, whether for testing, incremental refresh, or recovery from a partial failure.

Option B : Declarative mapping standards: R2RML and RML

R2RML (RDB to RDF Mapping Language) is a W3C standard specifically for expressing relational-to-RDF mappings declaratively, as data rather than code. RML (RDF Mapping Language) generalises the same idea to other source formats (CSV, JSON, XML) beyond just relational databases. The appeal is auditability, reusability, and tooling support (engines like Ontop, D2RQ, and Morph-KGC can execute R2RML mappings directly, either materialising the full graph or exposing it as a live, query-time virtual graph over the relational source).

```
# R2RML mapping (Turtle syntax) : CUSTOMERS table -> :Customer nodes
<#CustomerMapping>
rr:logicalTable [ rr:tableName "customers" ] ;
rr:subjectMap [
  rr:template "http://example.org/customer/{customer_id}" ;
  rr:class ex:Customer ;
] ;
rr:predicateObjectMap [
  rr:predicate ex:name ;
  rr:objectMap [ rr:column "first_name" ] ;
] ;
rr:predicateObjectMap [
  rr:predicate ex:email ;
  rr:objectMap [ rr:column "email" ] ;
] .
```

And the relationship mapping : the R2RML pattern for turning the orders.customer_id foreign key into a triple, using a join condition rather than a simple column reference:

```
<#OrderPlacedByMapping>
rr:logicalTable [ rr:tableName "orders" ] ;
rr:subjectMap [ rr:template "http://example.org/order/{order_id}" ] ;
rr:predicateObjectMap [
  rr:predicate ex:placedBy ;
  rr:objectMap [
    rr:parentTriplesMap <#CustomerMapping> ;
    rr:joinCondition [
      rr:child "customer_id" ; rr:parent "customer_id" ;
    ] ;
  ] ;
] .
```

Option C : Vendor-native import / ETL platforms

Most graph database vendors ship dedicated import tooling : Neo4j's neo4j-admin import and APOC library (apoc.load.jdbc for direct relational reads), Amazon Neptune's bulk loader, TigerGraph's GraphStudio mapping UI : which trade some flexibility for substantially faster time-to-first-graph,

especially for large initial bulk loads where a custom script's row-by-row network round trips become the bottleneck.

```
// Neo4j APOC : load directly from a
// relational source via JDBC
CALL
apoc.load.jdbc('jdbc:postgresql://host/db',
'SELECT * FROM customers')
YIELD row
MERGE (c:Customer {id:
row.customer_id})
SET c.name = row.first_name + ' ' +
row.last_name, c.email = row.email;
```

Mechanism	Best for	Trade-off
Custom ETL scripts	One-off migrations, complex custom logic, non-RDF targets	Implicit mapping logic; harder to audit at scale
R2RML / RML	RDF/SPARQL targets, formal auditability, live virtual graphs (no data duplication)	Steeper learning curve; RDF-specific tooling
Vendor-native import	Large bulk initial loads, rapid prototyping	Less portable across graph database vendors

7.5 Step 4 : End-to-end pipeline architecture

Regardless of which mapping mechanism you choose, production migrations follow the same overall pipeline shape. Figure 7.3 lays out the full architecture, from source schema through to the consuming applications, and names the two dominant deployment strategies teams choose between.

Figure 7.3 — End-to-End Architecture: Relational Database → Knowledge Graph

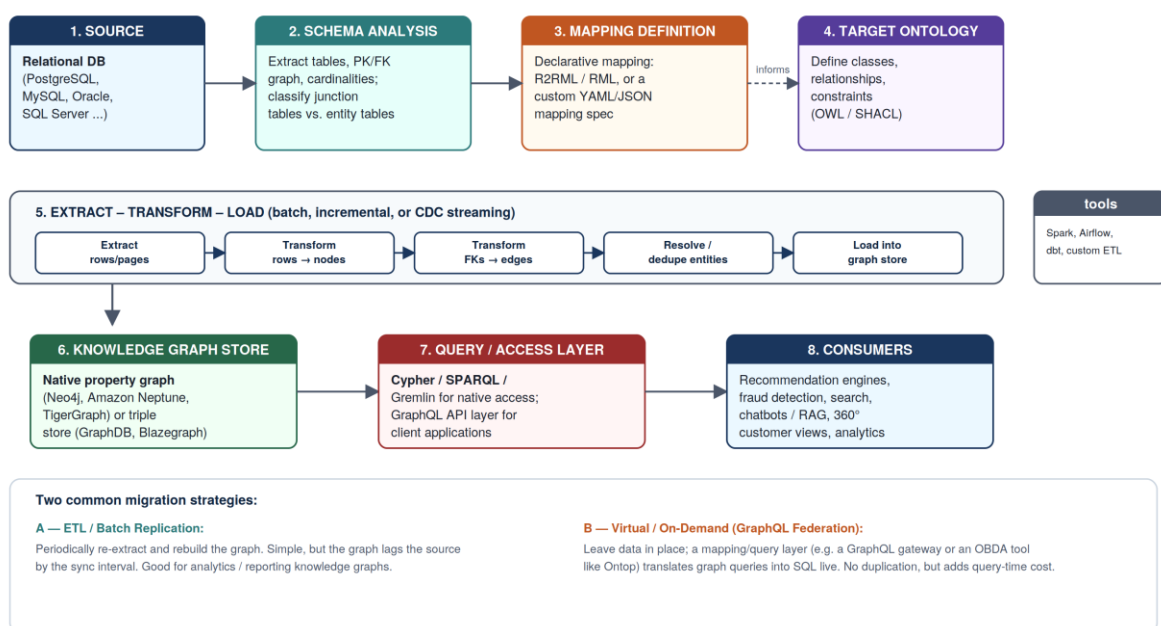


Figure 7.3 : The complete relational-to-knowledge-graph pipeline, from source analysis through to downstream consumers.

The choice between strategy A (ETL/batch replication) and strategy B (virtual/on-demand federation) is one of the most consequential architecture decisions in this whole process, and it deserves explicit criteria rather than a default:

- **Choose ETL/batch replication when :** query performance on the graph side is paramount, some staleness (minutes to hours) is acceptable, and you want the freedom to model the graph differently from a literal mirror of the source (denormalizing further, adding derived/inferred relationships, merging multiple sources).
- **Choose virtual/on-demand federation when :** the source system changes very frequently and staleness is unacceptable, you cannot duplicate the data for compliance/governance reasons, or the use case is exploratory and you don't yet want to commit to a full physical migration.

7.6 Step 5 : Entity resolution across sources

As discussed in Section 3.3, the highest-value knowledge graph migrations often integrate MULTIPLE relational sources, each with its own notion of the “same” real-world entity. Figure 7.4 shows the entity resolution pattern needed to reconcile these into one canonical node per real-world entity.

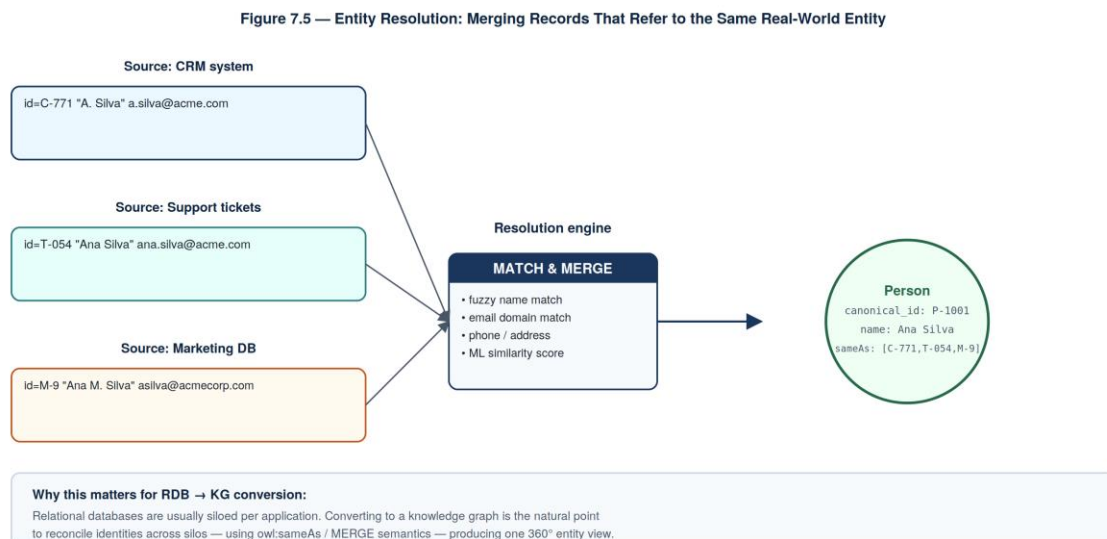


Figure 7.4 : Entity resolution merges records from multiple sources into one canonical, cross-referenced entity.

In practice, entity resolution logic ranges from simple deterministic rules (“same email address, case-insensitive, after trimming whitespace”) to probabilistic record linkage using multiple weighted signals (name similarity via Jaro-Winkler or Levenshtein distance, address matching, phone number normalisation) to full machine-learning classifiers trained on labelled “same entity / different entity” pairs. The right level of sophistication depends entirely on the cost of getting it wrong: a false MERGE (incorrectly combining two different people) is often far more damaging : and harder to detect and undo later : than a false split (failing to merge two records that were actually the same person), so most production systems bias conservative and use a human review queue for medium-confidence matches rather than auto-merging everything.

Regardless of the matching technique, the resolved output should preserve traceability back to source records : typically via a sameAs-style relationship or a sourceIds property listing every original record that was merged : both for auditability and so a bad merge can be identified and reversed.

7.7 The resulting graph

Applying every rule in this chapter to our running Customers/Orders/Products example produces the connected structure shown in Figure 7.5 : worth comparing directly against the original four-table relational schema in Figure 2.1 to see exactly what changed and what didn't.

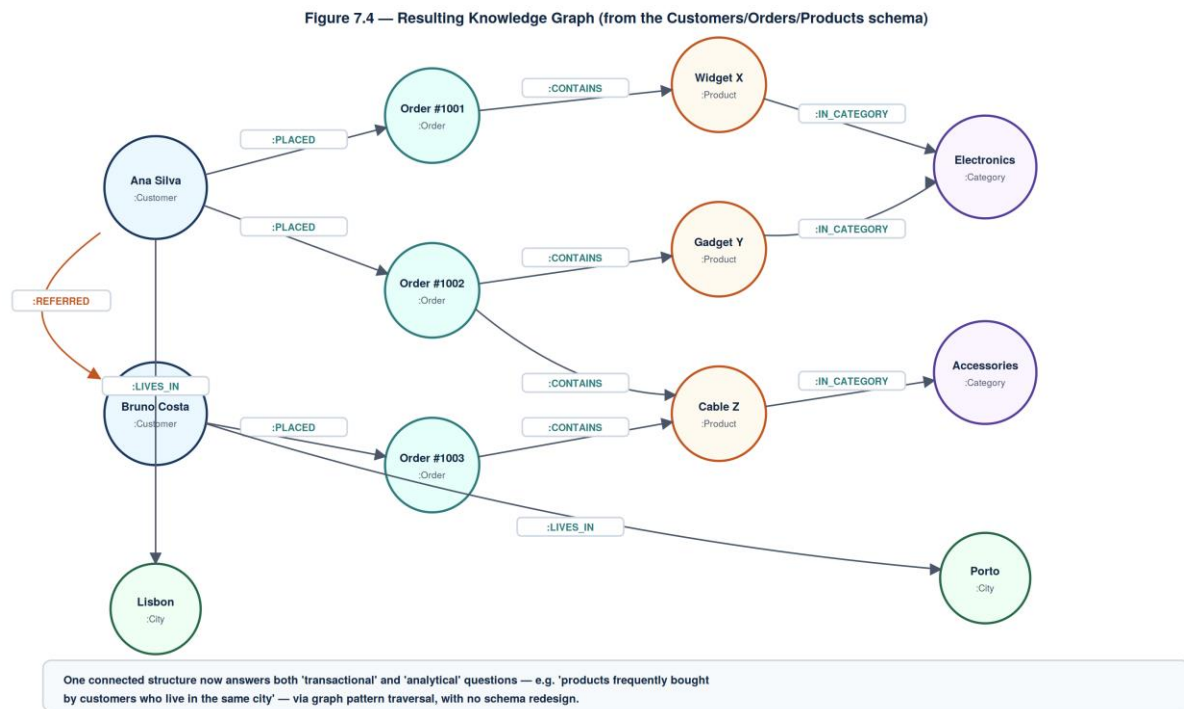


Figure 7.5 : The resulting knowledge graph after applying the full mapping rule set.

7.8 Handling structurally tricky cases

The rules above cover perhaps eighty percent of any real schema mechanically. The remaining cases need explicit judgment calls. This section catalogues the ones that come up most often in practice.

7.8.1 Table inheritance / polymorphic associations

Relational schemas often model “kinds of” relationships awkwardly : e.g. a single-table-inheritance pattern where a payments table has a payment_type column and a sprawl of nullable columns, only some of which apply to each type (card_last_four for card payments, bank_iban for bank transfers). A graph model handles this far more naturally via multiple node labels: a node can be labeled BOTH :Payment and :CardPayment simultaneously, carrying only the properties relevant to its actual type, with no nullable columns at all. This is one of the clearest wins schema flexibility provides.

7.8.2 Nullable foreign keys

A nullable foreign key (e.g. orders.assigned_support_agent_id, which is null until an agent is assigned) simply maps to “the relationship may or may not exist” : no assigned agent means no :ASSIGNED_TO relationship is created for that order, rather than creating a relationship pointing at a null or placeholder node. This is arguably cleaner in the graph model than in the relational one, where the null has to be specifically checked for in every query that touches that column.

7.8.3 Composite and natural keys

When a table's primary key is composite (e.g. (warehouse_id, sku) rather than a single surrogate id), the graph node's identity property should typically be a composite string built from both parts (e.g.

"WH-04:SKU-1123") or, more robustly, the graph database's own internally generated node identity should be used, with the composite natural key retained as a uniqueness-constrained property rather than reused directly as the node's primary identifier.

7.8.4 Temporal / bitemporal data and slowly changing dimensions

Relational schemas that track history (valid_from / valid_to columns, or full slowly-changing-dimension patterns from data warehousing) need a :Modelling decision in the graph: do you want the graph to reflect only the CURRENT state (simplest, but loses history), or the full history? A common, effective pattern for preserving history is to timestamp the relationship itself, rather than creating a new node per historical version: (:Employee)-[:HELD_ROLE {role:'Engineer', from:'2019-01-01', to:'2021-06-01'}]->(:Company), alongside a second, current relationship without a to date. This keeps the current-state query simple (filter for relationships with no to date) while preserving full history for temporal queries.

7.8.5 Soft deletes

A deleted_at or is_active column in the relational source should generally be preserved as a property on the corresponding node or relationship (rather than silently excluding “deleted” rows during migration), so downstream graph queries can make the same active/inactive filtering decision the source system does, and so historical/audit queries remain possible.

7.9 Validating the migration

A migration is not complete until it has been validated, and “the script ran without errors” is not validation. A rigorous migration validation plan checks, at minimum:

1. Row/node count parity : the number of nodes of each label should match the row count of the corresponding source table (accounting for any deliberate filtering, e.g. excluding soft-deleted rows if that was a design decision).
2. Relationship/foreign-key count parity : the number of relationships of each type should match the number of non-null foreign key values in the source (again, accounting for deliberate exclusions).
3. Referential completeness : every relationship's start and end nodes should actually exist; a common bug source is a foreign key whose migration ran before the table it references, silently dropping relationships whose target didn't yet exist.
4. Spot-check business questions : run the actual multi-hop queries the migration was commissioned to enable (Section 3.1) against both the old system (via application logic or a hand-written multi-join SQL query) and the new graph, and confirm they agree on a sample of known cases.
5. Property value spot-checks : sample individual entities and manually diff every property against the source row, watching in particular for type coercion bugs (e.g. a NUMERIC(10,2) SQL column silently becoming a floating-point property with rounding differences).

7.10 Keeping the graph in sync: incremental updates

A one-time migration is rarely sufficient : most knowledge graphs need to stay synchronised with a relational source that keeps changing. Three common strategies, in increasing order of freshness and implementation complexity:

Strategy	Freshness	Complexity	Notes
Full periodic re-load	Hours to a day	Low	Simplest option; rebuild the whole graph from scratch on a schedule. Wasteful at scale but very robust.
Incremental batch (updated_at watermark)	Minutes to hours	Medium	Query only rows changed since the last run using an updated_at timestamp column; requires source tables to reliably maintain such a column.
Change Data Capture (CDC) streaming	Seconds	High	Tools like Debezium tail the database's write-ahead log directly and stream every insert/update/delete as an event, which a consumer applies to the graph in near real time.

A minimal Debezium-based CDC handler, sketching how a change event becomes a graph write:

```
// Kafka consumer receiving Debezium change events from the 'orders' table
consumer.on('message', async (event) => {
  const { op, after, before } = JSON.parse(event.value);
  if (op === 'c' || op === 'u') {      // create or update
    await session.run(
      `MERGE (o:Order {id: $id})
        SET o.status = $status, o.total = $total,
        { id: after.order_id, status: after.status, total: after.total }`);
  } else if (op === 'd') {            // delete
    await session.run(`MATCH (o:Order {id: $id}) DETACH DELETE o`, { id: before.order_id });
  }
});
```

7.11 Measuring the payoff: benchmarking before and after

A migration project should produce hard evidence of the improvement it promised, not just a subjective sense that queries “feel faster.” Before treating a migration as successful, benchmark the specific multi-hop business questions that motivated it (Section 3.1) on BOTH systems, using comparable hardware and comparable data volumes, and record the results formally.

Question (illustrative)	Relational (multi-JOIN SQL)	Knowledge graph (Cypher)	Speedup
2-hop: customer's order history	12 ms	9 ms	~1.3x
4-hop: products bought by colleagues' colleagues	340 ms	28 ms	~12x
Variable-depth: full reporting chain under a manager	1,850 ms (recursive CTE)	41 ms	~45x
Unknown-depth: shortest path between two accounts	not practically expressible	63 ms	n/a

A note on these figures

The numbers above are deliberately illustrative, not a benchmark result from any specific system : real figures depend heavily on data volume, indexing, hardware, and query complexity, and you should always benchmark your own workload rather than trust published figures from any source, including this one. The QUALITATIVE pattern, however, replicates consistently across real migrations: the speedup grows sharply with traversal depth, is often marginal or even negative for simple one- or two-hop queries (where a well-indexed relational JOIN is already fast), and becomes dramatic : often an order of magnitude or more : for the deep, unknown-depth traversal cases relational databases handle worst. This pattern is itself a useful diagnostic: if your benchmark does NOT show this shape, revisit whether your business questions are genuinely connected-data questions (Section 3.1) or whether the migration may not have been well-targeted.

Beyond raw latency, record query LINE COUNT or cyclomatic complexity as a secondary metric : the reduction in query complexity (Section 3.1) is frequently as valuable operationally as the reduction in latency, since simpler queries are easier to review, less likely to contain subtle bugs, and faster for new team members to understand.

Chapter summary

Converting a relational database to a knowledge graph is a systematic mapping exercise: classify every table (entity / junction / lookup), apply the fixed rule set in Figure 7.1, collapse junction tables into relationships (the highest-value single transformation), resolve entity identity across sources where relevant, choose a mapping mechanism and sync strategy matched to your freshness and governance requirements, and validate rigorously before treating the graph as a trustworthy asset. The remaining parts of this tutorial build fluency in querying the result (Chapter 8), choosing between query languages (Chapter 9), production architecture (Chapter 10), and a full worked case study (Chapter 11).

PART FOUR

Querying, Architecture & Practice

Chapters 8 – 13

8 Querying the Graph: Cypher, SPARQL, and Gremlin

A knowledge graph is only as useful as the queries you can write against it. This chapter provides a working, practical introduction to the three dominant graph query languages, using our familiar e-commerce example throughout so the same questions can be compared directly across languages.

8.1 Cypher (property graphs : Neo4j, Memgraph, and openCypher-compatible engines)

Cypher is a declarative, pattern-matching query language originally created by Neo4j and now standardised, in large part, as the basis for the ISO/IEC GQL (Graph Query Language) standard ratified in 2024 : the first international standard query language purpose-built for property graphs, playing a role for graph databases analogous to what SQL plays for relational ones.

Basic pattern matching

Cypher's core idea is that a query pattern LOOKS like the graph structure it matches : parentheses for nodes, square brackets for relationships, arrows for direction:

```
// Find every product Ana Silva has ordered
MATCH (c:Customer {name: 'Ana Silva'})-[:PLACED]->(:Order)-[:CONTAINS]->(p:Product)
RETURN DISTINCT p.name, p.price
ORDER BY p.price DESC;
```

Variable-length paths

Recall the recursive-CTE friction from Section 3.1 : this is Cypher's direct answer to it, using the *min..max syntax to match a relationship any number of times:

```
// All indirect reports of a manager, at any depth
MATCH (m:Employee {id: 42})<-[:REPORTS_TO*1..]-(:report)
RETURN report.name;

// Shortest path between two people through the KNOWS network
MATCH p = shortestPath((a:Person {name:'Ana'})-[:KNOWS*]-(:b:Person {name:'Bruno'}))
RETURN length(p), p;
```

Aggregation and write operations

```
// Total revenue per product category
MATCH (p:Product)-[:IN_CATEGORY]->(cat:Category),
      (p)-[:CONTAINS]-(:oi:OrderItem)
RETURN cat.name, sum(oi.quantity * oi.unit_price) AS revenue
ORDER BY revenue DESC;

// Idempotent write with MERGE
MERGE (c:Customer {id: $customerId})
ON CREATE SET c.createdAt = timestamp()
SET c.lastSeenAt = timestamp();
```

8.2 SPARQL (RDF triple stores)

SPARQL (a recursive acronym: SPARQL Protocol and RDF Query Language) is the W3C-standardised query language for RDF data. Its core construct is the SELECT ... WHERE block, inside which you write triple patterns : subject-predicate-object expressions with variables (prefixed ?) standing in for the parts you want the query engine to fill in.

Basic triple pattern matching

```
PREFIX ex: <http://example.org/>
SELECT ?productName ?price WHERE {
  ?customer ex:name "Ana Silva" .
  ?customer ex:placed ?order .
  ?order ex:contains ?product .
  ?product ex:name ?productName ;
    ex:price ?price .
}
ORDER BY DESC(?price)
```

Property paths : SPARQL's variable-length traversal

```
# All indirect reports of a manager, using the + (one-or-more) path operator
SELECT ?report WHERE {
  ?report ex:reportsTo+ ex:manager42 .
}
```

Aggregation, and a distinctive SPARQL feature: federated queries

SPARQL's SERVICE clause allows a single query to span MULTIPLE remote SPARQL endpoints : federating live queries across organisational boundaries, which is a capability without a direct equivalent in Cypher or standard SQL, and reflects RDF's origins in the open, distributed Semantic Web:

```
SELECT ?cat (SUM(?qty * ?price) AS ?revenue) WHERE {
  ?product ex:inCategory ?cat .
  ?item ex:product ?product ; ex:quantity ?qty ; ex:unitPrice ?price .
} GROUP BY ?cat ORDER BY DESC(?revenue)

# Federated: enrich local data with a public knowledge base (e.g. Wikidata)
SELECT ?product ?wikidataDescription WHERE {
  ?product ex:name "Widget X" .
  SERVICE <https://query.wikidata.org/sparql> {
    ?wdItem rdfs:label "Widget X"@en ; schema:description ?wikidataDescription .
  }
}
```

8.3 Gremlin (Apache TinkerPop : a graph-traversal, not a graph-pattern, language)

Gremlin, part of the Apache TinkerPop framework, takes a philosophically different approach from Cypher and SPARQL: instead of DECLARING a pattern to match, you write an explicit, imperative

TRAVERSAL : a chain of steps describing how to walk the graph, step by step. It is supported by Amazon Neptune, JanusGraph, and many others as a common, engine-agnostic traversal API.

```
// All products Ana Silva has ordered
g.V().has('Customer', 'name', 'Ana Silva')
  .out('PLACED').out('CONTAINS')
  .dedup()
  .valueMap('name', 'price')

// Indirect reports, any depth, using repeat()/until()
g.V().has('Employee', 'id', 42)
  .repeat(__.in('REPORTS_TO')).emit()
  .values('name')
```

Gremlin's imperative style gives fine-grained control over exactly how a traversal executes : useful when you need to hand-optimize a specific expensive query : at the cost of being more verbose and less immediately readable than Cypher's declarative pattern syntax for straightforward queries.

8.4 Side-by-side: the same question, four ways

Chapter 9 develops a full comparison framework, but it is worth seeing the contrast directly here, immediately after learning the syntax of each language, while it is fresh.

Language	Paradigm	“Variable-depth traversal” syntax
SQL	Declarative, set-based	WITH RECURSIVE common table expression
Cypher	Declarative, pattern-matching	[<i>:REL_TYPE*1..5</i>] inside a MATCH pattern
SPARQL	Declarative, triple-pattern-matching	<i>predicate+</i> or <i>predicate*</i> property path
Gremlin	Imperative, step-chain traversal	<i>repeat(...).until(...)</i> / <i>.emit()</i>

Practical guidance

If you are choosing a query language for a new property-graph project today, learn Cypher first : it has become the de facto lingua franca (and is now the basis of the ISO GQL standard), has the gentlest learning curve of the three for developers coming from SQL, and is supported, in whole or in GQL-compatible part, by most major graph database vendors. Learn SPARQL specifically when your target is an RDF store or you need to integrate with the broader linked-data ecosystem. Reach for Gremlin when you need engine-agnostic portability across very different graph database backends, or fine-grained imperative control over traversal execution.

9 Comparing Query Languages, and Where GraphQL Fits

With SQL (Chapter 2), Cypher, SPARQL, and Gremlin (Chapter 8), and GraphQL (Chapter 5) all now introduced, this short chapter pulls them into one direct comparison and answers the question this tutorial has been building toward since the introduction: how does GraphQL actually relate to a knowledge graph's native query language?

9.1 One question, four query languages

Figure 9.1 answers the exact same question : “what products has Ana Silva ordered?” : in SQL, Cypher, SPARQL, and GraphQL side by side, making the structural differences and similarities immediately visible.

Figure 9.1 — Same Question, Four Query Languages: “What products has Ana Silva ordered?”

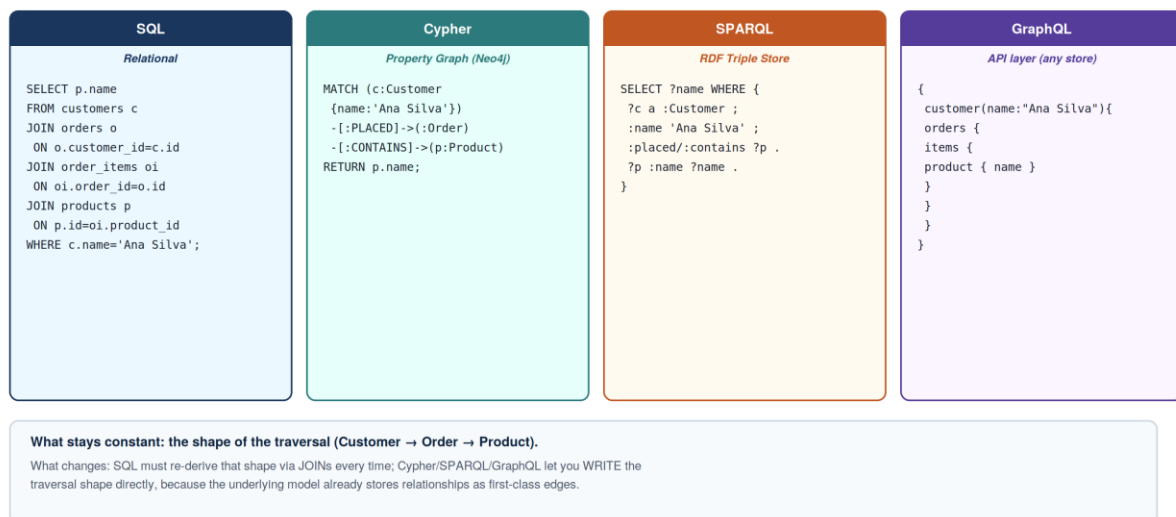


Figure 9.1 : The same question expressed in SQL, Cypher, SPARQL, and GraphQL.

The pattern to notice: SQL is the odd one out. It alone must explicitly re-derive the traversal path (Customer → Order → OrderItem → Product) via a chain of JOIN clauses, because the relationships are not stored as navigable objects. Cypher, SPARQL, and GraphQL all let you WRITE the traversal path more or less directly, because in each of their underlying models : property graph, RDF, or a typed field graph : relationships ARE navigable, first-class things.

9.2 GraphQL is a client-facing layer, not a storage-layer competitor

It bears repeating, because it is the single most common point of confusion this tutorial exists to clear up: GraphQL and Cypher/SPARQL are not competitors answering the same question. They operate at different layers of a system:

Layer	Question it answers	Typical languages
Client / API layer	“What shape of data does THIS client need, in one round trip?”	GraphQL

Layer	Question it answers	Typical languages
Storage / traversal layer	“How do I efficiently find and walk this pattern in the underlying store?”	Cypher, SPARQL, Gremlin, SQL

A production architecture very commonly uses BOTH at once: a GraphQL server sits at the API boundary (giving client applications a stable, typed, self-documenting contract, as described in Chapter 5), and its resolvers internally issue Cypher or SPARQL queries against the knowledge graph to actually fetch the data. Figure 5.2, revisited here, is exactly this architecture.

9.3 A decision framework

Bringing together every comparison in this tutorial, here is a practical decision sequence for a new project:

1. Do you need a client-facing API at all, or is this purely an internal/analytical workload? If you need a client-facing API spanning several backends (or want the benefits of typed, self-documenting contracts and precise over/under-fetch control from Chapter 5), add a GraphQL layer regardless of what's underneath it.
2. Is the underlying data dominated by multi-hop, connected-data questions (Section 3.1), or is it dominated by flat tabular aggregation? If the latter, a relational or columnar store, queried via SQL, remains the right backend : don't migrate data to a graph model that doesn't need one.
3. If a graph backend IS warranted: do you need formal, standards-based semantic integration and reasoning (Chapter 6), or application-facing traversal performance and developer ergonomics? The former points to RDF + SPARQL; the latter points to a property graph + Cypher.
4. Do you need engine portability across very different graph database vendors, or fine-grained imperative control over traversal execution? If so, consider Gremlin as an additional, engine-agnostic access layer alongside the native query language.

The one-sentence summary of this entire chapter

Choose your STORAGE model (relational, property graph, or RDF) based on the shape of your data and your query patterns (Chapters 2–4); choose your QUERY LANGUAGE (SQL, Cypher, SPARQL, Gremlin) to match that storage model (Chapter 8); and layer GraphQL on top, independently, whenever you need a typed, client-facing API : the two decisions are orthogonal, not competing.

10 Production Architecture Patterns

Moving a knowledge graph from a proof-of-concept into a production system that other teams depend on raises a fresh set of concerns : scale, consistency, availability, and operational ownership : that this chapter addresses directly, building on the pipeline architecture introduced in Section 7.5.

10.1 Where the knowledge graph sits relative to systems of record

The single most important architectural principle this tutorial argues for, stated plainly: in the overwhelming majority of deployments, the relational database(s) should remain the system of record, and the knowledge graph should be a DERIVED, read-optimised view built from it : not the other way around. This is not a limitation of graph databases; several are fully transactional and could serve as a system of record. It is a pragmatic recognition that the relational system already has the operational maturity, tooling, staff familiarity, and backup/recovery processes built around it, and that re-platforming the transactional core of a business is a vastly bigger, riskier undertaking than adding a derived, connected view alongside it.

Exceptions exist : greenfield projects with genuinely graph-native requirements from day one (a social network, a recommendation platform where the graph IS the product) reasonably choose a graph database as their primary system of record. But for the conversion scenario this tutorial focuses on : an existing relational estate gaining graph capabilities : “derived view, not replacement” is the right starting posture.

10.2 Consistency models: how stale can the graph be?

Every derived-view architecture makes an explicit or implicit trade-off between freshness and complexity, formalised in Section 7.10's three synchronisation strategies. It is worth stating the underlying trade-off in general terms: this is an instance of the classic CAP-theorem-adjacent tension between consistency and availability/partition tolerance, applied to a replicated read-model rather than a distributed transactional system. A knowledge graph fed by CDC (Section 7.10) is eventually consistent with its relational source : usually within seconds : and every application built on top of it needs to be designed with that eventual consistency in mind (for example, not assuming a write to the relational source is immediately visible in a graph query issued milliseconds later).

10.3 Scaling a graph database

Graph databases face a scaling challenge relational databases largely sidestep: graphs resist clean horizontal partitioning, because a well-connected graph has edges crossing almost any partition boundary you draw, and cross-partition traversal (a query that has to hop from a node on one machine to a node on another) is far more expensive than an intra-partition hop. Three broad strategies are used in practice:

- **Vertical scaling** : the simplest approach : run the graph on a single, large machine with enough RAM to keep the working set (or the whole graph) in memory. Many production knowledge graphs, even large ones, run this way successfully, because well-indexed graph traversal is fast enough that a single powerful node handles substantial workloads; this remains the default recommendation unless you have concretely outgrown it.
- **Read replicas** : scale read throughput (not write throughput or graph size) by replicating the full graph to multiple read-only followers, routing read queries across them : directly analogous to relational read-replica patterns, and usually the first scaling lever reached for.

- **Sharding / partitioning** : genuinely distribute the graph across multiple machines, using a partitioning strategy (often based on graph community detection, so that densely-connected clusters of nodes tend to land on the same shard, minimising expensive cross-shard traversals). This is the most complex option and is only justified once a single well-specified machine plus read replicas has been conclusively outgrown.

10.4 Hybrid architectures: querying across both stores

Some queries genuinely need data from both the relational system AND the graph in a single logical operation : for example, a query that needs a precise financial aggregate (a relational strength) filtered by a multi-hop graph relationship (a graph strength). Two patterns handle this:

Pattern A : Federation at the API layer

As introduced in Chapter 5 and Section 9.2, a GraphQL gateway (or an application service layer) issues separate queries to each backend and joins the results in application code. This keeps each backend doing what it's best at, at the cost of that join logic living in the application rather than being pushed down to either database.

Pattern B : Federation at the query layer

Certain tools allow a single query to span both a relational source and a graph directly : SPARQL's SERVICE federation (Section 8.2) can, with the right connector, treat a relational database as a queryable federated endpoint via an OBDA (Ontology-Based Data Access) layer such as Ontop, which exposes a virtual RDF graph over live relational data using an R2RML mapping (Section 7.4) without materialising any data at all : effectively strategy B (virtual/on-demand federation) from Section 7.5, used as a permanent architecture rather than a migration stepping stone.

10.5 Operational concerns

Concern	Relational baseline	Graph-specific considerations
Backup / recovery	Mature, well-understood tooling (pg_dump, WAL archiving)	Most graph databases have equivalent tooling now, but verify RPO/RTO guarantees for your specific vendor; for a derived graph, the fastest recovery path is often 'just re-run the ETL pipeline' rather than restoring from a graph-native backup
Monitoring	Query latency, connection pool saturation, replication lag	Add: traversal depth / fan-out metrics (a small number of 'super-node' hubs with huge fan-out can silently degrade otherwise-fast queries), cache hit rates for frequently-traversed subgraphs
Access control	Row-level and column-level security, well standardised	Varies significantly by vendor; property- and label-level access control is less uniformly mature across the ecosystem : verify explicitly against your compliance requirements before committing
Schema evolution	Migrations, versioned and reviewed	Property graphs' schema flexibility is a double-edged sword operationally: without discipline (naming conventions, a documented ontology per Chapter 6, and ideally enforced constraints), graphs drift into inconsistency

Concern	Relational baseline	Graph-specific considerations
		faster than a constraint-enforced relational schema would

10.6 Team ownership and organisational patterns

A pattern worth naming explicitly, because it recurs across many successful deployments: the team that owns the SOURCE relational system rarely also owns the knowledge graph derived from it : these tend to become genuinely separate platform capabilities, with a dedicated “graph platform” or “knowledge graph” team owning the mapping logic (Chapter 7), the ontology (Chapter 6), and the sync pipeline (Section 7.10) as a shared service consumed by multiple downstream application teams, each building features (recommendations, search, fraud detection) on top of the shared graph without needing to understand the source relational schemas themselves. This mirrors how a data platform or analytics engineering team commonly owns a data warehouse independent of the operational teams that own the source OLTP systems feeding it.

10.7 Multi-region and geo-distribution

Organisations operating across multiple geographic regions face an additional dimension of the scaling problem from Section 10.3: not just “how do we spread this graph across machines” but “how do we spread this graph across DATA CENTRES separated by real network latency, while keeping queries fast for users everywhere.” Three patterns are used in practice, each trading off differently between consistency, latency, and complexity:

- **Single-region, globally-read replicas** : keep one authoritative writable region, and replicate read-only copies to other regions close to users : the simplest option, appropriate when writes are infrequent relative to reads and some replication lag is acceptable, directly extending the read-replica pattern from Section 10.3 across geography rather than just across machines in one data centre.
- **Regional graph partitioning by tenant or geography** : when the data itself is naturally geography-scoped (e.g. a retailer's EU customer data legally must stay in the EU), partition the graph itself by region rather than replicating it wholesale : each region owns and serves its own slice, with a separate, deliberate integration layer for the comparatively rare cross-region query.
- **Multi-region active-active** : the most complex option: multiple regions accept writes simultaneously, with conflict resolution for concurrent updates to the same node : genuinely difficult to get right for a densely connected graph (where conflicts can span relationships spanning regions, not just isolated rows), and worth adopting only when write latency for global users is a hard, validated business requirement rather than a nice-to-have.

For most knowledge graph deployments : particularly the derived, read-optimised architecture this tutorial recommends (Section 10.1) : the first pattern, single-region writes with geo-distributed read replicas, is the right default: the source of truth (the relational system) is very often already single-region, and there is little reason for its derived graph view to take on multi-region write complexity the source itself doesn't have.

10.8 Cost management

Graph database infrastructure costs are driven by a different profile than typical relational workloads, and budgeting for a knowledge graph platform benefits from understanding that profile explicitly rather than assuming relational cost intuitions transfer directly.

Cost driver	Relational intuition	Graph-specific reality
Memory	Important, but disk-backed operation is normal and expected	Disproportionately important : many graph databases perform best with the full working set (often the whole graph) in RAM, per Section 10.3, making memory frequently the dominant infrastructure cost line item
Compute for algorithms	Analytics workloads are typically a separate, occasional batch cost (a nightly warehouse job)	Graph algorithms (Chapter 14) are similarly batch-oriented but can be substantially more CPU-intensive per byte of data than typical SQL aggregation, meriting a dedicated, right-sized analytics replica (Section 14.6) rather than running on the primary serving instance
Storage duplication	Minimal : one copy of the data in the relational system	A derived graph (Section 10.1) is, by definition, a SECOND copy of a meaningful portion of your data, plus its own indexes : budget for this duplication explicitly rather than treating the graph as free incremental capability
Sync pipeline	N/A for a standalone relational system	CDC infrastructure (Section 7.10) : message brokers, stream processors : is an ongoing, non-trivial cost most relational-only budgets don't already include

A practical cost-control lever worth calling out specifically because it is often overlooked: the table-classification discipline from Section 7.2 is also a cost-control discipline : every table you correctly decide NOT to migrate (per the anti-pattern warning in Section 12.3) is data you never pay to store twice, index twice, or keep synchronised.

Chapter summary

Treat the knowledge graph as a derived, read-optimised view over your relational systems of record; choose a synchronisation strategy matched to your freshness requirements; scale vertically and via read replicas before reaching for sharding, extending the same pattern geographically for multi-region deployments; use federation (at the API or query layer) for questions that genuinely need both stores; budget explicitly for the memory, algorithmic compute, storage duplication, and sync pipeline costs unique to a graph platform; and plan for the operational and organisational realities : monitoring, access control, and dedicated ownership : that a new platform capability requires.

11 Case Study: Migrating a Support Ticketing System

This chapter applies every technique from Chapters 6 through 10 to one continuous, realistic worked example, end to end : from an existing relational schema, through schema analysis, mapping, migration, and validation, to the business questions the resulting knowledge graph unlocks.

11.1 The scenario

A mid-sized software company runs a support ticketing system on top of a PostgreSQL database. The product team wants to answer questions the current relational schema struggles with: “Which customers are most at risk of churn based on ticket patterns similar to customers who already churned?” and “Which product areas have clusters of related tickets that suggest a common root cause, even when the ticket text doesn't use the same words?” Both are fundamentally connected-data questions : similarity and clustering across a network of relationships : that are natural for a graph and awkward for repeated SQL joins and text search alone.

11.2 The source schema

```
CUSTOMERS(customer_id PK, company_name, plan_tier, signed_up_at, churned_at NULL)
USERS(user_id PK, customer_id FK, name, email, role)
TICKETS(ticket_id PK, customer_id FK, opened_by_user_id FK, assigned_agent_id FK,
        subject, status, priority, product_area, opened_at, closed_at NULL)
TICKET_COMMENTS(comment_id PK, ticket_id FK, author_user_id FK, body, created_at)
TICKET_TAGS(ticket_id FK, tag_id FK) -- pure junction table
TAGS(tag_id PK, label)
AGENTS(agent_id PK, name, team)
TICKET_RELATED(ticket_id FK, related_ticket_id FK, relation_type) -- self-referencing junction
```

11.3 Step 1 : Schema analysis and classification

Applying the classification framework from Section 7.2 to every table:

Table	Classification	Rationale
CUSTOMERS	Entity table	Single-column PK, represents a real-world organisation
USERS	Entity table	Single-column PK; note the customer_id FK, a standard N:1 relationship
TICKETS	Entity table	Single-column PK; THREE separate foreign keys, each a distinct relationship type
TICKET_COMMENTS	Entity table	Represents a real, individually addressable thing (a comment), not a pure junction
TICKET_TAGS	Pure junction table	Composite key of exactly two FKs, no other columns : a textbook collapse candidate
TAGS	Lookup / entity table	Small, but tags are themselves meaningfully queryable (“all tickets with this tag”), so they become nodes rather than collapsing into a property
AGENTS	Entity table	Single-column PK, referenced by TICKETS.assigned_agent_id

Table	Classification	Rationale
TICKET_RELATED	Pure junction table (self-referencing)	Composite key of two FKs to the SAME table, plus one descriptive column : a rich junction on a self-reference

11.4 Step 2 : Define the target ontology

Before writing mapping code, the target node labels, relationship types, and their directions are decided explicitly (Section 7.3.1's naming heuristic applied throughout):

```
Node labels:   :Customer :User :Ticket :Comment :Tag :Agent

Relationships:
(:User)-[:BELONGS_TO]->(:Customer)
(:Ticket)-[:FILED_BY]->(:Customer)
(:Ticket)-[:OPENED_BY]->(:User)
(:Ticket)-[:ASSIGNED_TO]->(:Agent)
(:Comment)-[:ON]->(:Ticket)
(:Comment)-[:WRITTEN_BY]->(:User)
(:Ticket)-[:TAGGED]->(:Tag)           -- from TICKET_TAGS collapse
(:Ticket)-[:RELATED_TO {type: '...'}]->(:Ticket) -- from TICKET_RELATED collapse
```

11.5 Step 3 : The mapping

Using the custom-ETL-script mechanism from Section 7.4 (Option A), chosen here for its flexibility and because the target is a property graph rather than RDF:

```
def migrate_customers(pg, session):
    for row in pg.query("SELECT * FROM customers"):
        session.run("""
            MERGE (c:Customer {id: $id})
            SET c.name = $name, c.planTier = $plan, c.signedUpAt = $signedUp,
                c.churnedAt = $churned, c.isChurned = $churned IS NOT NULL
            """, id=row.customer_id, name=row.company_name, plan=row.plan_tier,
                signedUp=row.signed_up_at, churned=row.churned_at)

def migrate_tickets(pg, session):
    for row in pg.query("SELECT * FROM tickets"):
        session.run("""
            MERGE (t:Ticket {id: $id})
            SET t.subject=$subj, t.status=$status, t.priority=$prio,
                t.productArea=$area, t.openedAt=$opened, t.closedAt=$closed
            WITH t
            MATCH (c:Customer {id: $custId})
            MERGE (t)-[:FILED_BY]->(c)
            WITH t
            MATCH (u:User {id: $openedBy})
            MERGE (t)-[:OPENED_BY]->(u)
            WITH t
            OPTIONAL MATCH (a:Agent {id: $agentId}) // nullable FK : Section 7.8.2
            FOREACH _ IN CASE WHEN a IS NOT NULL THEN [1] ELSE [] END |
                MERGE (t)-[:ASSIGNED_TO]->(a)
            """, id=row.ticket_id, subj=row.subject, status=row.status,
```

```

prio=row.priority, area=row.product_area, opened=row.opened_at,
closed=row.closed_at, custId=row.customer_id,
openedBy=row.opened_by_user_id, agentId=row.assigned_agent_id)

def migrate_ticket_tags(pg, session):      # junction table collapse : Section 7.3.2
    for row in pg.query("""
        SELECT tt.ticket_id, t.label FROM ticket_tags tt
        JOIN tags t ON t.tag_id = tt.tag_id"""):
        session.run("""
            MATCH (tk:Ticket {id: $tid})
            MERGE (tag:Tag {label: $label})
            MERGE (tk)-[:TAGGED]->(tag)
            """, tid=row.ticket_id, label=row.label)

def migrate_ticket_related(pg, session):    # self-referencing rich junction : Section 11.3
    for row in pg.query("SELECT * FROM ticket_related"):
        session.run("""
            MATCH (a:Ticket {id: $t1}), (b:Ticket {id: $t2})
            MERGE (a)-[:RELATED_TO {type: $rtype}]->(b)
            """, t1=row.ticket_id, t2=row.related_ticket_id, rtype=row.relation_type)

```

11.6 Step 4 : Validate

Applying the validation checklist from Section 7.9:

```

-- Row/node count parity check, run against BOTH systems and diffed
SELECT count(*) FROM customers;    -- SQL: expect exact match to...
MATCH (c:Customer) RETURN count(c); -- ...this Cypher count

SELECT count(*) FROM ticket_tags;    -- SQL relationship-parity source
MATCH ()-[r:TAGGED]->() RETURN count(r);    -- Cypher relationship-parity check

```

11.7 Step 5 : Answering the business questions

With the graph built and validated, the two motivating questions from Section 11.1 become tractable graph queries.

Churn-pattern similarity

```

// Customers whose ticket-tag "fingerprint" overlaps heavily with churned customers,
// but who have not themselves churned yet
MATCH (churned:Customer {isChurned: true})<-[:FILED_BY]-(:Ticket)-[:TAGGED]->(tag:Tag)
WITH tag, count(DISTINCT churned) AS churnSignal
WHERE churnSignal >= 3
MATCH (active:Customer {isChurned: false})<-[:FILED_BY]-(:Ticket)-[:TAGGED]->(tag)
RETURN active.name, collect(DISTINCT tag.label) AS riskTags, count(DISTINCT tag) AS riskScore
ORDER BY riskScore DESC LIMIT 20;

```

Ticket clustering by relationship structure, not just text

```

// Product areas where tickets form a densely interlinked cluster via RELATED_TO,
// even across different literal subjects/text

```

```
MATCH (t1:Ticket)-[:RELATED_TO*1..3]-(t2:Ticket)
WHERE t1.productArea = t2.productArea
WITH t1.productArea AS area, count(DISTINCT t2) AS clusterSize
WHERE clusterSize >= 5
RETURN area, clusterSize ORDER BY clusterSize DESC;
```

Neither query is expressible nearly as concisely in the original relational schema : the first requires self-joining through a tag-overlap computation across an unknown number of customers; the second requires the same variable-depth recursive traversal problem discussed in Section 3.1. Both are direct, readable pattern matches once the data lives in the graph.

Chapter takeaway

Notice that every decision in this case study traced back to a rule or principle established earlier in the tutorial : table classification (7.2), relationship naming (7.3.1), junction collapse (7.3.2), nullable foreign keys (7.8.2), self-referencing junctions (7.8), and validation (7.9). A real migration is, in this sense, mostly disciplined application of a small rule set, not ad hoc improvisation : which is exactly what makes it a teachable, repeatable skill rather than an art.

12 Best Practices, Pitfalls, and Anti-Patterns

This chapter collects the recurring lessons : both the practices worth adopting deliberately and the mistakes worth actively avoiding : that surface repeatedly across real relational-to-knowledge-graph migrations. Treat it as a checklist to revisit before, during, and after a migration project.

12.1 :Modelling best practices

1. Model relationships the way a domain expert would phrase them out loud (Section 7.3.1) : resist the temptation to default every relationship to the direction the foreign key happens to point.
2. Prefer a small number of well-named relationship types over many overly-specific ones.
:PURCHASED is usually better than separate :PURCHASED_ONLINE and :PURCHASED_IN_STORE types : model the distinction as a property (channel: 'online') on one relationship type instead, unless the query patterns genuinely need to filter by relationship type specifically (Cypher and SPARQL can both filter on relationship/edge properties just as easily as on type).
3. Always give nodes a stable, meaningful uniqueness constraint (Section 7.8.3) : never rely solely on the graph database's auto-generated internal ID as your business identifier, since internal IDs are not guaranteed stable across backups, exports, or certain maintenance operations in every vendor.
4. Document your ontology (Chapter 6) as you build it, even informally : a shared glossary of node labels, relationship types, and their meanings prevents the same real-world concept from silently acquiring two different names as a graph grows and more people contribute to it.
5. Favor denormalizing INTO relationship properties over creating intermediate 'fact' nodes for simple attributes of a relationship (Section 4.1) : an extra node adds two extra traversal hops to every query that needs that fact, for no benefit if the fact never needs to be independently queried or related to anything else.

12.2 Migration process best practices

1. Always write migration scripts using idempotent operations (MERGE, not CREATE) so they are safely re-runnable (Section 7.4) : you will re-run them, during testing if nothing else.
2. Migrate and validate incrementally, one entity type at a time, rather than attempting the full schema in one pass : this makes failures easy to localise and keeps each migration step reviewable.
3. Build the count-parity and referential-completeness checks from Section 7.9 as automated, repeatable tests, not one-off manual spreadsheet checks : you will need to run them again after every schema change on either side.
4. Load entity (node) tables completely before loading any relationship-creating tables that reference them : a relationship-creation statement whose target node doesn't exist yet either fails loudly (good) or, worse, silently creates a sparse placeholder node with no properties (bad, and a common source of hard-to-detect data quality bugs).
5. Keep the original relational primary key as a property on every migrated node, even when the graph database's internal ID is used for something else : this is what makes re-running an incremental sync, or auditing the migration against the source, tractable.

12.3 Common anti-patterns to avoid

Anti-pattern: migrating everything, indiscriminately

Not every table benefits from becoming part of the graph. Purely transactional, non-connected operational data (audit logs, session tokens, rate-limit counters) rarely benefits from graph traversal and adds migration and sync overhead for no query-pattern benefit. Apply the classification from Section 7.2 selectively : migrate the entities and relationships that actually serve the connected-data questions driving the project, and leave the rest in the relational system.

Anti-pattern: :Modelling attributes as relationships to squeeze in 'graph thinking'

Not everything needs to be a node. A product's color, a customer's loyalty tier, an order's currency : these are usually better left as simple properties, not modeled as relationships to a :Color or :Tier node, UNLESS those categories themselves need to be independently queried, related to other entities, or carry their own properties (recall the Section 7.2 lookup-table guidance). Turning every attribute into a node inflates the graph, slows traversal with unnecessary hops, and makes queries harder to read for no corresponding benefit.

Anti-pattern: ignoring super-nodes

A node with an extremely high number of relationships (a popular product with a million purchase edges; a country node connected to every citizen) is called a super-node, and traversals that pass through one can silently become very expensive, even when every other part of the query is well indexed and fast. Identify likely super-nodes during schema analysis (Section 7.2) : they usually correspond to small reference/lookup tables referenced by very large fact tables : and design queries and, where necessary, data models to avoid unnecessarily traversing through them.

Anti-pattern: treating the initial migration as a one-time project

A knowledge graph with no ongoing synchronisation strategy (Section 7.10) starts decaying in accuracy from the moment it's built, as the relational source continues to change. Budget for ongoing synchronisation, ownership (Section 10.6), and ontology maintenance (Chapter 6) as a genuine, ongoing platform capability from the start of the project, not as an afterthought bolted on once staleness complaints start arriving.

12.4 A pre-launch checklist

- Every table has been explicitly classified (entity / junction / lookup) and the classification documented
- Every relationship type has an agreed name, direction, and documented meaning
- Node uniqueness constraints exist and are enforced by the graph database, not just assumed by convention
- Row/node and FK/relationship count parity checks pass against the full dataset, not just a sample
- At least the top five business questions motivating the migration have been run against the new graph and manually verified against the old system

- A synchronisation strategy (Section 7.10) is chosen, implemented, and monitored : not left as a manual re-run script someone has to remember to execute
- Ownership of the graph platform (mapping logic, ontology, sync pipeline) is assigned to a specific team, not left implicitly with whoever built it
- Access control on the graph has been reviewed against the same compliance requirements that governed the source relational data

12.5 Roles and responsibilities: a sample RACI

Building on Section 10.6's ownership discussion, the table below sketches a representative responsibility assignment (RACI: Responsible, Accountable, Consulted, Informed) for a knowledge graph migration, useful as a starting template for a project charter : adapt the specific roles to your own organisational structure.

Activity	Data/Graph Platform Team	Source System Team	Application Teams	Compliance/Security
Table classification (7.2) & ontology design (Ch.6)	R/A	C	C	I
Mapping logic & ETL pipeline (7.4)	R/A	C	I	I
Entity resolution rules (7.6)	R/A	C	C	C
Migration validation (7.9)	R/A	C	C	I
Ongoing sync monitoring (10.5)	R/A	I	I	I
Access control policy (16.5)	R	C	I	A
Consuming the graph in a product feature	C	I	R/A	I

The pattern worth noting: the graph platform team is Accountable for almost everything on the data and pipeline side, but is deliberately NOT accountable for how downstream application teams consume the graph in their own features : that ownership boundary (mirroring Section 10.6's “shared service” model) keeps the platform team from becoming a bottleneck for every product decision built on top of the graph, while still giving them clear authority over the data's correctness and availability.

13 Conclusion & Further Reading

We began this tutorial with a simple observation: relational databases excel at structured record-keeping but strain against questions that are fundamentally about connectedness, while knowledge graphs excel at exactly those connected-data questions but do not replace the transactional discipline the relational model provides. The methodology in Part III exists to let an organisation have both : a trusted relational system of record, and a connected, queryable graph derived from it : rather than forcing a choice between them.

13.1 What we covered

- Part I established the relational model precisely : keys, normalization, joins, indexing : specifically so that every concept had a name we could later map onto graph terms.
- Part II built both graph foundations (property graphs, RDF) and a full treatment of GraphQL, establishing clearly that GraphQL is an API/query layer, not a storage model, and that it composes with : rather than competes against : a knowledge graph backend.
- Part III delivered the core skill: a repeatable methodology for classifying relational tables, applying mapping rules, choosing a mapping mechanism, architecting the pipeline, resolving entity identity across sources, and validating the result : all grounded in a complete worked case study.
- Part IV rounded out the practical picture: comparing query languages with a clear decision framework, production architecture patterns, and a consolidated checklist of best practices and anti-patterns.

13.2 A closing principle

If you remember only one thing from this tutorial

A knowledge graph is not a different, better database that makes the relational model obsolete. It is a different LENS on the same underlying facts : one that makes connectedness a first-class, directly queryable property of the data instead of something reconstructed, expensively, at query time via JOINS. Choose the lens : relational, graph, or (very commonly) both together : based on the shape of the questions you actually need to answer, not based on which technology is currently fashionable.

13.3 Suggested next steps

1. Pick one real, existing relational schema you know well and walk it through the classification exercise in Section 7.2, entirely on paper, before writing any code.
2. Stand up a free-tier or local instance of a property graph database (Neo4j's Community Edition is a common, well-documented starting point) and migrate a small slice of that schema using the custom-ETL approach from Section 7.4.
3. Write the three or four business questions that most motivate your interest in a knowledge graph, and confirm : honestly : that they are genuinely connected-data questions (Section 3.1) rather than tabular aggregation questions better served by SQL as-is.
4. If your use case involves formal semantic integration or regulatory/scientific data, spend time specifically with RDF, OWL, and SPARQL (Sections 4.2, 6.3–6.4, 8.2) rather than defaulting to a property graph.
5. Revisit the pre-launch checklist in Section 12.4 before treating any migration as production-ready.

13.4 Further reading

Topic	Reference
The original relational model	E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, 1970
Property graphs & Cypher	The openCypher project documentation; the ISO/IEC 39075 GQL standard
RDF, OWL, SPARQL, SHACL	The W3C's official specifications for each standard, at w3.org
R2RML / RML	The W3C R2RML recommendation; the RML.io project documentation
GraphQL	The official GraphQL specification at graphql.org/learn
Graph data :Modelling more broadly	Ian Robinson, Jim Webber & Emil Eifrem, “Graph Databases” (O'Reilly)
Entity resolution	Peter Christen, “Data Matching” (Springer)

This concludes the core narrative of the tutorial. Part Five extends it with two increasingly important topics : graph algorithms and analytics, and the intersection of knowledge graphs with AI and large language models : before the appendices provide a consolidated glossary and quick-reference tables you may find useful to keep close at hand during your own migration work.

PART FIVE

Algorithms, Analytics & AI

Chapters 14 – 15

14 Graph Algorithms and Analytics

Everything so far has treated the knowledge graph primarily as a place to look up and traverse explicit facts: find this node, follow that relationship, match this pattern. This chapter introduces a different and highly valuable class of capability: graph ALGORITHMS that compute properties of the graph's overall STRUCTURE : which nodes are most influential, which nodes cluster together, which nodes are likely to become connected in the future : properties that have no clean equivalent at all in a relational model, because they depend on the global shape of the connections, not on any single row's values.

14.1 Why relational databases cannot do this well

Computing something like “the most influential node in this network” relationally would require materialising the entire connection graph into memory or a temporary structure and then running a fundamentally graph-shaped algorithm over it : SQL's set-based, table-oriented execution model has no native concept of iterative graph traversal converging to a fixed point, which is what most of these algorithms actually are. This is a second, distinct category of “why graphs” : separate from the query-convenience argument of Chapter 3 : and it is often the more compelling one for analytics and data science teams specifically, as opposed to application engineering teams.

Figure 14.1 — Graph Algorithms: Centrality and Community Detection at a Glance

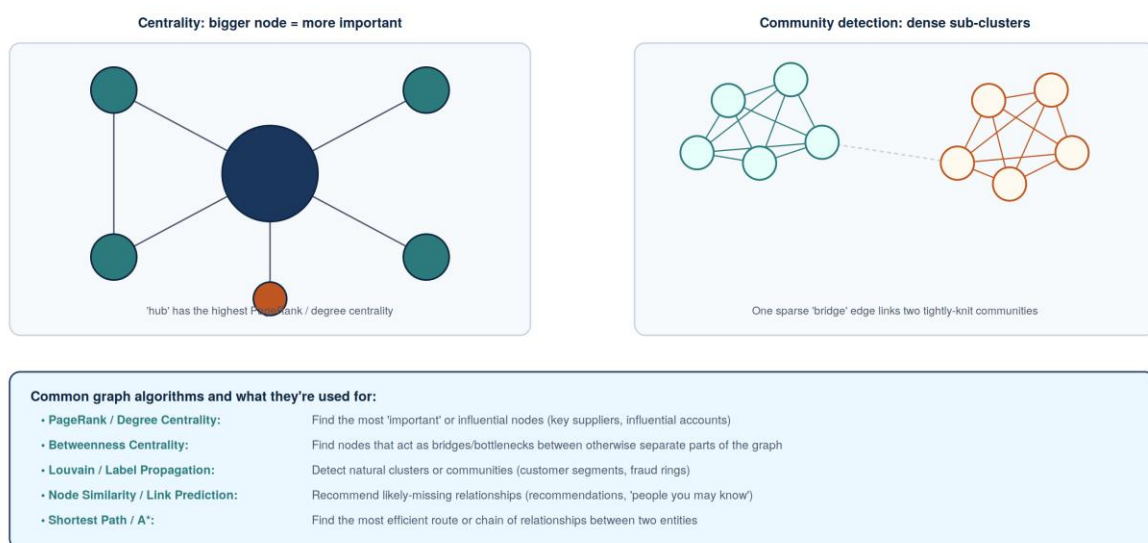


Figure 14.1 : Centrality identifies influential nodes; community detection finds natural clusters.

14.2 Centrality algorithms: finding what matters

Centrality algorithms answer some version of “which nodes matter most?” : but different centrality measures define “matters” differently, and choosing the right one for your question matters a great deal.

Degree centrality

The simplest measure: count how many relationships a node has. A product bought by many customers, or a person with many direct connections, has high degree centrality. Cheap to compute, but naive : it treats every connection as equally important, regardless of how important the CONNECTED node is.

PageRank

Originally developed by Google's founders to rank web pages, PageRank refines degree centrality by weighting each connection by the importance of the node it comes FROM : a link from an already-important node counts for more than a link from an obscure one. This makes it well suited to fraud detection (an account connected to several already-flagged fraudulent accounts should itself be considered higher-risk, even if its raw degree is low) and influence analysis (a supplier connected to a small number of very large customers may be more strategically important than one connected to many small ones).

```
// Neo4j Graph Data Science library : PageRank over the WORKS_AT / KNOWS network
CALL gds.pageRank.stream('professional-network')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS person, score
ORDER BY score DESC LIMIT 10;
```

Betweenness centrality

Measures how often a node lies on the SHORTEST PATH between other pairs of nodes : identifying “bridge” or “bottleneck” nodes whose removal would disconnect otherwise-separate parts of the graph. In a supply chain graph, a supplier with high betweenness centrality is a single point of failure risk even if it isn't the largest supplier by volume : precisely the kind of hidden structural risk that is very hard to spot from tabular reports.

14.3 Community detection: finding natural clusters

Community detection algorithms partition a graph into groups of nodes that are more densely connected to each other than to the rest of the graph : without being told in advance what those groups are or how many there should be. The Louvain method (optimising a metric called modularity) and Label Propagation are the two most widely used algorithms in production systems, chosen primarily for their speed at scale.

Practical applications recur across many industries: customer segmentation based on actual co-purchasing or interaction patterns rather than pre-defined demographic buckets; fraud ring detection, where a community of accounts sharing unusual numbers of connections (shared devices, addresses, or payment instruments) is a strong fraud signal even before any single account looks individually suspicious; and organisational network analysis, revealing informal collaboration clusters that don't match the official reporting hierarchy.

```
// Louvain community detection : Cypher (Neo4j GDS)
CALL gds.louvain.stream('account-network')
YIELD nodeId, communityId
RETURN communityId, collect(gds.util.asNode(nodeId).accountNumber) AS accounts
ORDER BY size(accounts) DESC;
```

14.4 Similarity and link prediction

Node similarity algorithms compute how alike two nodes are based on their NEIGHBOURHOODS : two customers who have purchased many of the same products, even if they've never interacted with each other directly, are structurally similar. This underlies the classic “customers who bought this also

bought...” recommendation pattern, computed directly from graph structure rather than from a separately trained machine learning model.

Link prediction goes one step further, estimating the LIKELIHOOD that a relationship will form in the future (or already exists but is missing from the data) based on the current graph structure : used for “people you may know” features, predicting which two currently-unconnected entities in a fraud graph are likely part of the same undetected ring, and for data-quality work, flagging plausible missing relationships for review during a migration.

14.5 Shortest path and pathfinding

Beyond simple traversal (Chapter 8), dedicated pathfinding algorithms find the OPTIMAL path between two nodes under some cost function : Dijkstra's algorithm and A* for weighted shortest paths (e.g. logistics routing where relationship weights represent distance or cost), and all-pairs shortest path algorithms for precomputing distances across an entire graph for later fast lookup.

```
// Weighted shortest path between two warehouses, minimising shipping cost
MATCH (a:Warehouse {code:'WH-04'}), (b:Warehouse {code:'WH-19'})
CALL gds.shortestPath.dijkstra.stream('logistics-network', {
  sourceNode: a, targetNode: b, relationshipWeightProperty: 'shippingCost'
})
YIELD path RETURN path;
```

14.6 Running algorithms in a hybrid architecture

A practical point worth making explicitly, tying back to Chapter 10: graph algorithms are typically CPU- and memory-intensive, iterative computations run over the WHOLE graph (or a large subgraph) at once : a meaningfully different workload profile from the low-latency, small-subgraph pattern-matching queries covered in Chapter 8. Production systems commonly run algorithmic workloads on a schedule (nightly, hourly) against a dedicated analytics replica, writing the RESULTS (a PageRank score, a community ID) back onto each node as a plain property : which application queries can then simply read and filter on, cheaply, without re-running the expensive algorithm on every request.

Chapter summary

Graph algorithms : centrality, community detection, similarity, and pathfinding : compute properties of a graph's overall structure that have no clean relational equivalent, and are frequently the highest-ROI capability a knowledge graph unlocks for analytics and data science teams specifically. Run them as scheduled batch jobs against an analytics replica, and materialise their results as plain node properties for cheap, low-latency consumption by application queries.

15 Knowledge Graphs, Embeddings, and GraphRAG

No technical tutorial on knowledge graphs written in 2026 would be complete without addressing the relationship between knowledge graphs and large language models (LLMs) directly : it is, at the time of writing, one of the fastest-growing reasons organisations invest in graph infrastructure at all. This chapter explains the two main integration points precisely, without hype: vector embeddings stored alongside graph structure, and GraphRAG (Graph-based Retrieval-Augmented Generation).

15.1 Vector embeddings: a second, complementary notion of “related”

Everything in this tutorial up to now has treated “related” as EXPLICIT : a stored, typed edge between two nodes, put there because a foreign key or an ontology relationship said it belonged there. A vector embedding introduces a second, different notion of relatedness: two entities are similar if their machine-learned numerical representations (typically 384 to 3072-dimensional vectors, produced by an embedding model) are close together in that vector space : a LEARNED, fuzzy, semantic similarity rather than a stored, exact, structural one. Figure 15.2 contrasts the two directly.

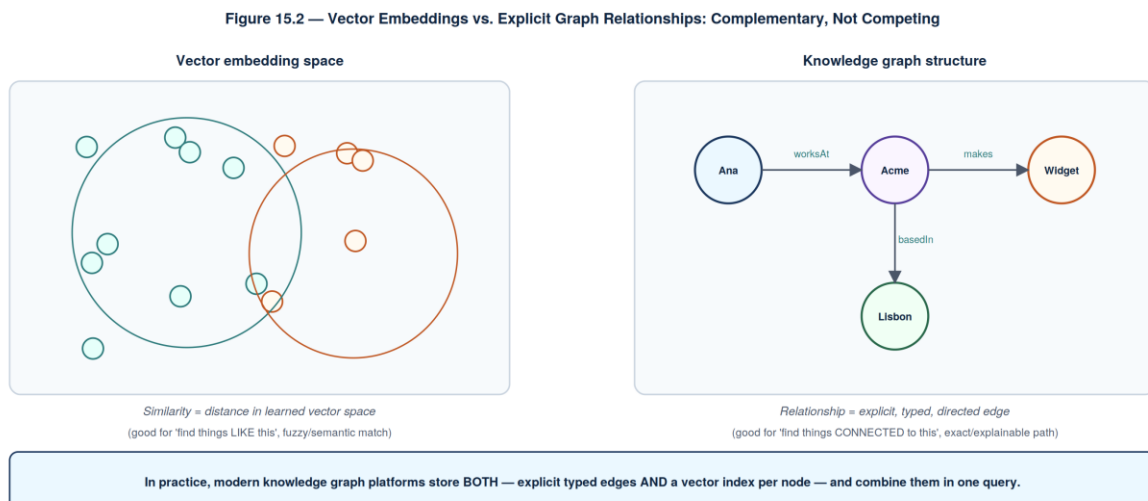


Figure 15.2 : Vector similarity captures fuzzy semantic closeness; graph edges capture exact, explicit relationships.

The two are genuinely complementary, not competing, and most modern knowledge graph platforms (Neo4j, Amazon Neptune Analytics, and others) now support storing a vector embedding directly as a property on a node, alongside its explicit typed relationships, and querying both together in a single request:

```
// Neo4j : vector similarity search combined with explicit graph filtering
CALL db.index.vector.queryNodes('productEmbeddings', 5, $queryVector)
YIELD node AS product, score
MATCH (product)-[:IN_CATEGORY]->(cat:Category {name: 'Electronics'}) // explicit graph filter
RETURN product.name, score
ORDER BY score DESC;
```

This single query answers a question neither model alone handles as well: “find products semantically similar to this one, but only within the Electronics category” : semantic fuzziness from the vector index, precise structural filtering from the graph.

15.2 Where do embeddings come from?

Three common sources, each capturing a different notion of similarity, and often used together on the same graph for different purposes:

- **Text/content embeddings** : produced by passing a node's textual description (a product description, a support ticket body, a document) through a text embedding model : capturing SEMANTIC/topical similarity.
- **Graph-structural embeddings** : produced by algorithms like Node2Vec or GraphSAGE that learn a vector representation of a node FROM its position and connections in the graph itself : capturing STRUCTURAL similarity (two nodes are close if they play a similar structural role, even without similar text or attributes).
- **Multi-modal embeddings** : produced from images, audio, or other non-text content associated with an entity : a product photo, a scanned document : capturing similarity in that modality.

15.3 GraphRAG: grounding LLM answers in graph traversal

Retrieval-Augmented Generation (RAG) is the now-standard pattern for giving an LLM access to information beyond its training data: retrieve relevant content at query time, and include it in the prompt as context before asking the model to answer. The most common implementation of RAG uses vector similarity search alone : embed the user's question, find the most similar text chunks, hand them to the LLM. GraphRAG extends this pattern by retrieving a structured SUBGRAPH of connected facts, via explicit graph traversal, instead of (or in addition to) unstructured text chunks. Figure 15.1 lays out the full pipeline.

Figure 15.1 — GraphRAG: Grounding an LLM's Answer in Knowledge Graph Traversal

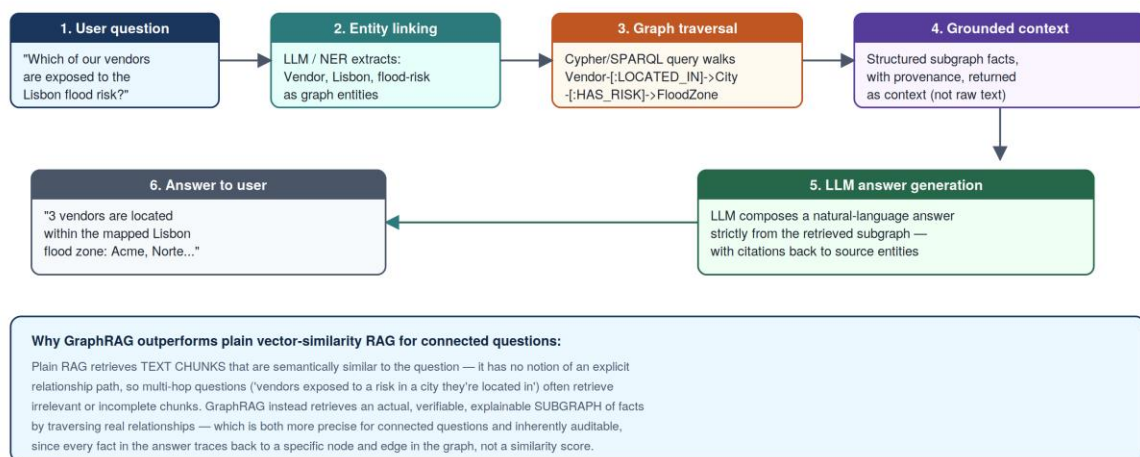


Figure 15.1 : GraphRAG retrieves a verifiable subgraph of facts via traversal, rather than relying on similarity search alone.

15.4 Why GraphRAG matters specifically for multi-hop questions

Plain vector-similarity RAG has a structural weakness that should, by this point in the tutorial, sound familiar: it struggles with MULTI-HOP questions : exactly the class of question Chapter 3 identified as the relational model's weak point. "Which of our vendors are exposed to the Lisbon flood risk?" requires combining a fact about vendor location with a separate fact about flood risk by location : a two-hop connection that is unlikely to appear together, phrased exactly that way, in any single retrievable text

chunk. A GraphRAG system, by contrast, retrieves the actual (:Vendor)-[:LOCATED_IN]->(:City)-[:HAS_RISK]->(:FloodZone) path directly, because that traversal is exactly what graph query languages (Chapter 8) were built to do.

A second, equally important advantage is EXPLAINABILITY and auditability. Because every fact returned by a GraphRAG retrieval step traces back to a specific node and a specific relationship in the graph, the system can show its work : citing exactly which entities and relationships supported each part of the LLM's answer : in a way that a pure vector-similarity match (“this chunk scored 0.83 similarity”) cannot. This matters enormously for regulated industries and any application where a wrong or unverifiable AI-generated answer carries real cost.

15.5 A minimal GraphRAG implementation sketch

```
async function graphRagAnswer(userQuestion) {
  // 1. Extract candidate entities from the question (via an LLM call or NER model)
  const entities = await extractEntities(userQuestion); // e.g. ['Lisbon', 'flood risk']

  // 2. Resolve extracted entity mentions to actual graph nodes
  const anchors = await Promise.all(entities.map(e => resolveToGraphNode(e)));

  // 3. Traverse outward from the anchor nodes to build a relevant subgraph
  const subgraph = await session.run(`
    MATCH (city:City {name: $city})<-[:LOCATED_IN]-(v:Vendor)
    MATCH (city)-[:HAS_RISK]->(risk:FloodZone)
    RETURN v.name AS vendor, city.name AS city, risk.severity AS riskLevel
  `, { city: anchors[0].name });

  // 4. Hand the STRUCTURED subgraph facts (not raw text) to the LLM as context
  const prompt = buildPromptFromSubgraph(userQuestion, subgraph.records);
  return await llm.generate(prompt); // LLM composes the final answer, grounded in real facts
}
```

15.6 Practical guidance: when GraphRAG is worth the added complexity

A decision heuristic

Plain vector-similarity RAG remains simpler to build and is entirely sufficient for questions that are fundamentally about finding SIMILAR content : “find support articles like this one,” “summarise documents about topic X.” Reach for GraphRAG specifically when your questions are multi-hop and connected : spanning several explicit relationships : or when explainability/auditability of the answer's provenance is a hard requirement, not a nice-to-have. Many production systems, sensibly, run BOTH retrieval strategies in parallel and let the LLM (or a routing step) decide which context is more relevant for a given question, or combine both context sources in a single prompt.

This chapter closes the substantive technical content of this tutorial. The remaining appendices collect quick-reference material : a glossary, mapping cheat sheet, query language comparison table, and now an expanded FAQ and troubleshooting guide : for use during your own migration work.

PART SIX

Governance & Applied Practice

Chapters 16 – 18

16 Data Quality, Testing, and Governance

A migrated knowledge graph is a new, independent artifact with its own failure modes : it can drift from its relational source, accumulate duplicate entities, or silently violate the ontology it claims to follow. This chapter treats data quality and governance as first-class engineering concerns, extending the validation checklist from Section 7.9 into an ongoing discipline rather than a one-time migration gate.

16.1 Six dimensions of graph data quality

Classical data quality frameworks translate directly onto a knowledge graph, with a graph-specific reading of each dimension. Figure 16.1 lays out all six side by side; the rest of this section expands on the three that behave most differently from their relational counterparts.

Figure 16.1 — Six Data Quality Dimensions Applied to a Knowledge Graph

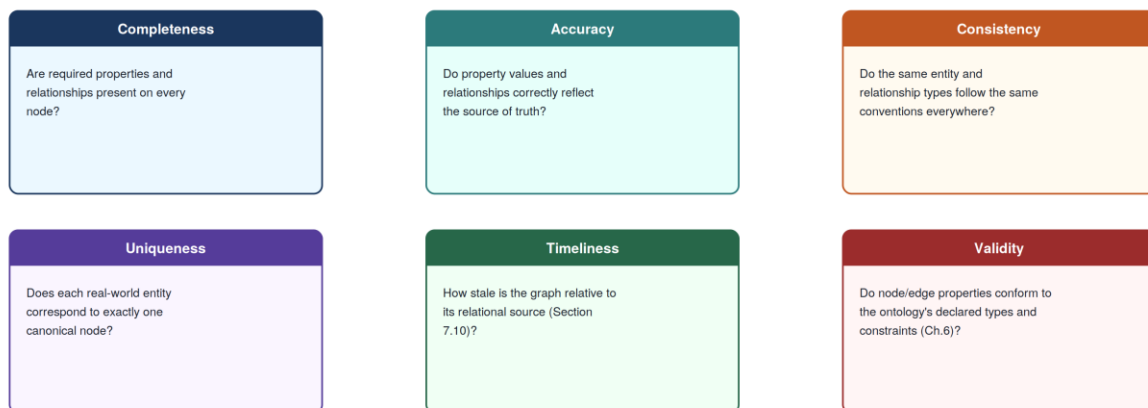


Figure 16.1 : Six data quality dimensions, reinterpreted for a knowledge graph.

Completeness in a graph context is subtler than “no NULLs” : a node can have every property populated and still be incomplete in a graph-specific sense if it is MISSING relationships it should have (an :Order with no :CONTAINS relationships at all is a red flag a relational NOT NULL constraint would never catch, because the relationship's absence, not a column's absence, is the defect). Uniqueness, similarly, is a substantially harder problem in a graph built from multiple sources (Section 7.6's entity resolution) than in a single relational table protected by a UNIQUE constraint, because the graph must reconcile identity ACROSS sources with no shared key, not merely prevent duplicates within one.

16.2 Automated constraint enforcement

Wherever the target graph technology supports it, encode data quality rules as ENFORCED constraints rather than as documentation or manual review : the same principle that makes relational NOT NULL and UNIQUE constraints valuable (Section 2.4) applies identically here.

```
// Neo4j : enforce node uniqueness and required properties as real, checked constraints
CREATE CONSTRAINT customer_id_unique IF NOT EXISTS
FOR (c:Customer) REQUIRE c.id IS UNIQUE;

CREATE CONSTRAINT customer_email_required IF NOT EXISTS
```

```
FOR (c:Customer) REQUIRE c.email IS NOT NULL;
```

```
# SHACL : declarative validation, enforced against RDF data at load or query time
ex:OrderShape a sh:NodeShape ;
  sh:targetClass ex:Order ;
  sh:property [
    sh:path ex:contains ;
    sh:minCount 1 ;           # every Order must contain at least one item
    sh:message "Order has no line items" ;
  ] .
```

16.3 Testing a knowledge graph like software

Treat graph mapping logic (Chapter 7) as production code, with the same testing discipline applied to any other data pipeline: unit tests for individual mapping functions, integration tests running the full pipeline against a small, known fixture dataset with hand-verified expected output, and regression tests that re-run the business questions from Section 7.9 against every schema change on either side of the pipeline.

```
// A fixture-based integration test (Jest / JavaScript), asserting mapping correctness
test('collapses STUDENT_COURSE junction table into ENROLLED_IN relationship', async () => {
  await loadFixture('student_course_sample.sql'); // known, small input
  await runMigration();
  const result = await session.run(
    MATCH (s:Student {id:'S1'})-[e:ENROLLED_IN]->(c:Course {id:'C1'})
    RETURN e.grade AS grade`);
  expect(result.records[0].get('grade')).toBe('A'); // matches known fixture value
});
```

A subtlety worth calling out: because MERGE-based migration scripts are idempotent by design (Section 7.4), the SAME integration test suite doubles as a safe way to verify re-runability : running the full migration twice against the same fixture and asserting the resulting node/relationship counts are identical after both runs is a cheap, high-value regression test that catches an entire class of duplication bugs early.

16.4 Ontology governance: preventing schema drift

Chapter 6 introduced the ontology as the schema layer that gives a knowledge graph meaning. Without active governance, that meaning drifts : a second team, unaware a :WORKS_AT relationship type already exists, introduces :EMPLOYED_BY for the identical concept; a property gets renamed inconsistently across different parts of the pipeline. Three practical governance mechanisms address this directly:

1. A single, version-controlled ontology definition (whether as OWL/SHACL files or a plain documented schema registry) that every mapping script and application query is reviewed against before introducing a new label, relationship type, or property name.
2. A lightweight review process : even a simple pull-request-style review of ontology changes : specifically because an ontology, unlike a single team's relational schema, is usually a genuinely shared, cross-team asset (Section 10.6).
3. Automated linting that checks new data against the ontology's declared constraints (Section 16.2) as part of every pipeline run, failing loudly rather than silently accepting a schema violation.

16.5 Access control and compliance

Building directly on Section 10.5's operational overview, a few compliance-specific points deserve emphasis. First, because a knowledge graph frequently INTEGRATES data from multiple source systems (Section 3.3), it can easily end up holding a combination of data that is individually permitted in each source system but collectively more sensitive than any single source : re-identification risk from combining otherwise-innocuous attributes across sources is a genuine, well-documented privacy concern that deserves explicit review, not an assumption that source-system access controls are sufficient once data is merged.

Second, “right to be forgotten” and similar data-subject deletion requirements (under regulations such as GDPR) apply to the knowledge graph exactly as they apply to the relational source : and because a graph is, by construction, densely interconnected, a deletion request typically requires removing not just a node but every relationship attached to it, and considering whether any DERIVED facts (algorithm results from Chapter 14, or inferred facts from Section 6.4) computed partly from that node's data also need to be recomputed or removed.

Chapter summary

Treat data quality as an ongoing discipline, not a one-time migration gate: enforce constraints where the technology allows it, test mapping logic like production code (including explicit idempotency regression tests), govern the ontology as a shared, version-controlled asset with real review, and extend compliance thinking to account for the re-identification and deletion-cascade risks unique to a densely connected, multi-source graph.

17 Second Case Study: Supply Chain Risk at a Manufacturer

The first case study (Chapter 11) focused on a customer-facing, single-source scenario. This second case study deliberately covers different terrain : a MULTI-SOURCE, entity-resolution-heavy migration in an industrial supply chain context : to demonstrate the methodology's generality and to give the entity resolution and hybrid-architecture material from Chapters 7 and 10 a fully worked treatment.

17.1 The scenario

A mid-sized industrial manufacturer sources components from hundreds of suppliers across multiple tiers (direct suppliers, and their own upstream suppliers). Procurement, supplier risk, and manufacturing each maintain SEPARATE relational systems: a procurement ERP tracking direct suppliers and purchase orders; a third-party risk-scoring database tracking financial and geopolitical risk per supplier; and a bill-of-materials (BOM) system tracking which components go into which products, recursively, since sub-assemblies are themselves built from further components. Leadership wants to answer, within minutes rather than the weeks a manual cross-referencing exercise currently takes: “if Supplier X fails, which of our products are affected, and how severely?”

17.2 Why this is a harder integration problem than Chapter 11

Three properties of this scenario push harder on the methodology than the single-source ticketing example did:

- **Three independent relational sources, no shared keys** : the procurement ERP identifies suppliers by an internal vendor code; the risk database identifies them by a Dun & Bradstreet D-U-N-S number; the BOM system doesn't identify suppliers at all, only component part numbers, which must be cross-referenced back to the procurement system separately.
- **Recursive, multi-tier structure** : the BOM is a self-referencing hierarchy (a sub-assembly contains components, which may themselves be sub-assemblies) of unknown, variable depth : precisely the variable-depth traversal case from Section 3.1 : and supplier risk needs to propagate up through however many tiers separate a raw-material supplier from a finished product.
- **A derived, computed risk score, not just structural facts** : the final answer isn't a simple traversal result; it requires combining a structural fact (which products use which components) with a numeric fact (each supplier's individual risk score) via an aggregation over a variable-depth path.

17.3 Step 1 : Schema analysis across three sources

Applying Section 7.2's classification framework independently to each source, before any merging is attempted:

Source	Key tables	Classification notes
Procurement ERP	VENDORS, PURCHASE_ORDERS, PO_LINES	VENDORS is an entity table keyed by internal vendor_code; PO_LINES is a rich junction between orders and parts
Risk database	SUPPLIERS, RISK_SCORES, RISK_EVENTS	SUPPLIERS is an entity table keyed by DUNS number : a DIFFERENT identifier space from vendor_code

Source	Key tables	Classification notes
BOM system	PARTS, BOM_LINES	BOM_LINES is a self-referencing rich junction: (parent_part_id, child_part_id, quantity) : structurally identical to the TICKET_RELATED pattern from Section 11.3

17.4 Step 2 : Entity resolution: reconciling vendor_code and DUNS number

Applying Section 7.6's methodology directly: neither source shares a key with the other, so resolution must use secondary signals : legal company name (fuzzy-matched), registered address, and tax ID where available. Because a FALSE merge here (incorrectly linking two different suppliers) could cause the risk system to silently attribute one company's financial distress to an unrelated company, this project deliberately biases conservative, per Section 7.6's guidance: high-confidence matches (exact tax ID match) are auto-merged; medium-confidence matches (fuzzy name + address match only) are queued for a one-time human review before the corresponding :SameAs relationship is created.

```
// Deterministic high-confidence match : auto-merge
MATCH (v:Vendor {taxId: $taxId}), (s:Supplier {taxId: $taxId})
WHERE v.taxId IS NOT NULL
MERGE (v)-[:SAME_AS {confidence: 'high', method: 'taxId_exact'}]->(s)

// Fuzzy match : queued for review, NOT auto-merged
MATCH (v:Vendor), (s:Supplier)
WHERE apoc.text.jaroWinklerDistance(v.legalName, s.legalName) > 0.92
AND NOT (v)-[:SAME_AS]-(s)
MERGE (v)-[:CANDIDATE_MATCH {score: apoc.text.jaroWinklerDistance(v.legalName, s.legalName)}]->(s);
```

17.5 Step 3 : :Modelling the recursive BOM structure

The self-referencing BOM_LINES table collapses following the exact pattern established in Section 11.3 for TICKET_RELATED : a self-referencing rich junction becomes a self-referencing relationship carrying the junction table's extra columns as edge properties:

```
MATCH (parent:Part {id: $parentId}), (child:Part {id: $childId})
MERGE (parent)-[:CONTAINS {quantity: $qty}]->(child)
```

With this in place, the variable-depth traversal problem from Section 3.1 : “which finished products, at any BOM depth, ultimately contain this component” : becomes a direct application of Cypher's variable-length path syntax from Section 8.1:

```
MATCH (component:Part {id: $partId})<-[:CONTAINS*1..]-(product:Part {isFinishedGood: true})
RETURN DISTINCT product.name;
```

17.6 Step 4 : Computing propagated risk

The final business question combines structural traversal with a numeric aggregation : a pattern worth studying closely because it recurs constantly in real graph applications once basic traversal is mastered:

```

MATCH (supplier:Supplier {duns: $duns})
MATCH (supplier)<-[:SUPPLIED_BY]-(part:Part)
MATCH (part)<-[:CONTAINS*1..]-(product:Part {isFinishedGood: true})
WITH product, supplier.riskScore AS risk, count(DISTINCT part) AS exposurePaths
RETURN product.name,
       risk * exposurePaths AS exposureWeightedRisk, // simple illustrative scoring function
       exposurePaths
ORDER BY exposureWeightedRisk DESC
LIMIT 20;

```

Notice this query would be extraordinarily awkward in the original three-source relational world: it requires an unknown-depth recursive traversal (Section 3.1) THROUGH a cross-source entity resolution join (Section 7.6) THROUGH another recursive traversal, aggregated at the end. Reproducing this exactly in SQL across three separate databases would likely require a dedicated, bespoke data pipeline built specifically for this one question : whereas here it is a nine-line, directly readable graph query, reusable for any similar question with minor edits.

17.7 Step 5 : Architecture: why this became a hybrid deployment

Applying Section 10.4's hybrid-architecture guidance, the final system did not migrate the full BOM system into the graph : the BOM system remained the authoritative, high-write-throughput system of record for day-to-day engineering changes, synced into the graph via the incremental CDC strategy from Section 7.10 (freshness mattered here, since BOM changes should be reflected in risk exposure quickly). The risk database, by contrast, was chosen for full periodic batch replication (Section 7.10) because its risk scores update at most daily, and near-real-time freshness added no business value proportional to the added pipeline complexity : a direct, deliberate application of the freshness-versus-complexity trade-off from Section 10.2.

Chapter takeaway

This case study intentionally stress-tested the parts of the methodology the first case study (Chapter 11) didn't reach: multi-source entity resolution with asymmetric confidence handling (Section 7.6), a genuinely recursive, variable-depth data structure with no fixed hop count (Section 7.8), and a hybrid architecture deliberately mixing two different synchronisation strategies for two sources with different freshness requirements within the SAME graph (Sections 7.10 and 10.4). Every one of these techniques was introduced earlier in the tutorial specifically so it could be reused here : real migrations are combinations of a fairly small number of well-understood patterns, not bespoke inventions each time.

18 Industry-Specific :Modelling Patterns

The mapping methodology in Chapter 7 is deliberately general : it applies to any relational schema. This chapter grounds it further with four industry-specific worked patterns, each highlighting :Modelling decisions that recur constantly within that domain and are worth recognising by name.

18.1 Healthcare: patients, encounters, and clinical events

Healthcare relational schemas (often following, or adjacent to, standards like HL7 FHIR's relational mappings) typically centre on a PATIENTS table connected to a long, append-only history of ENCOUNTERS (visits), each of which links to DIAGNOSES, MEDICATIONS, and LAB_RESULTS. The knowledge graph pattern that recurs here is the EVENT CHAIN : very similar to the audit-history pattern from Appendix G.5, but central to the domain rather than a peripheral audit trail:

```
MATCH (p:Patient {id: $patientId})
CREATE (e:Encounter {date: $date, type: $type, provider: $provider})
CREATE (p)-[:HAD_ENCOUNTER]->(e)
WITH e
UNWIND $diagnoses AS dx
MATCH (code:DiagnosisCode {icd10: dx.code}) // reference a shared clinical ontology, not a free-text value
CREATE (e)-[:DIAGNOSED]->(code)
```

The detail worth calling out specifically: diagnosis and medication codes should be modeled as relationships to SHARED reference nodes representing standard clinical vocabularies (ICD-10, SNOMED CT, RxNorm) rather than as free-text properties on the encounter : this is a direct application of Section 7.2's lookup-table guidance, and it is what allows powerful population-health queries (“find all patients with condition X who were later prescribed medication Y”) to traverse shared clinical concepts rather than perform fragile text matching. Because this data is subject to stringent regulatory protection (HIPAA and equivalent regimes elsewhere), every point in Section 16.5 on access control and re-identification risk applies here with particular force : healthcare knowledge graphs should be built with field-level and even relationship-level access control as a first-class design requirement, not an afterthought.

18.2 E-commerce and retail: catalog, behaviour, and recommendations

This tutorial's running example has already covered the core e-commerce entity structure (Chapters 2 and 7). The pattern worth adding here is BEHAVIOURAL EVENT :MODELLING : capturing not just completed orders but the full browsing and interaction funnel (viewed, added-to-cart, purchased) as a graph, which is what powers real-time recommendation and personalisation use cases beyond simple “customers who bought this also bought” (Section 14.4):

```
MATCH (customer:Customer {sessionId: $sessionId}), (product:Product {id: $productId})
CREATE (customer)-[:VIEWED {at: datetime(), source: $referrer}]->(product)
```

A practical note on scale: behavioural events (views) are typically orders of magnitude higher volume than transactional events (purchases). Most production e-commerce knowledge graphs deliberately keep only a bounded recent window of raw VIEWED events in the graph itself (pruned on a schedule), while materialising longer-term aggregate signals (a rolling affinity score between a customer and a product category, computed via the graph algorithms in Chapter 14) as durable node properties : a direct

application of the “compute expensive results on a schedule, store them as plain properties” guidance from Section 14.6.

18.3 Financial services: transactions, accounts, and fraud rings

Beyond the supply-chain risk example in Chapter 17, financial services' other dominant knowledge graph use case is transaction and fraud graphs, connecting ACCOUNTS, TRANSACTIONS, DEVICES, and IP_ADDRESSES. The :Modelling pattern here is SHARED-ATTRIBUTE LINKING: two accounts that have never transacted with each other directly can still be connected, indirectly, via a shared device, address, or payment instrument : and it is precisely this INDIRECT, multi-hop connectivity, invisible to any single-table relational query, that surfaces coordinated fraud rings:

```
MATCH (a1:Account)-[:USED_DEVICE]->(d:Device)-[:USED_DEVICE]-(a2:Account)
WHERE a1 <> a2 AND NOT (a1)-[:OWNED_BY]->(:Person)-[:OWNED_BY]-(a2) // not already known-
related
WITH d, collect(DISTINCT a1) + collect(DISTINCT a2) AS linkedAccounts
WHERE size(linkedAccounts) >= 5 // threshold: unusually many accounts sharing one device
RETURN d.deviceId, linkedAccounts;
```

This exact query shape : combined with the community detection algorithms from Section 14.3 run regularly across the whole account/device/address graph : is close to the actual architecture used by several major banks and payment processors for real-time and near-real-time fraud ring detection, and is a clear, concrete illustration of why this use case so consistently tops industry adoption surveys for knowledge graph technology.

18.4 Telecommunications and network/IT infrastructure

Telecom and IT infrastructure schemas track physical and logical network topology : routers, links, circuits, services running over them : in relational tables that mirror the recursive BOM pattern from Chapter 17 almost exactly (a NETWORK_LINKS table connecting NODES, often self-referencing). The high-value graph query here is IMPACT ANALYSIS: “if this router fails, which customer services are affected?” : structurally identical to the supply-chain risk-propagation query from Section 17.6, just with router failure in place of supplier failure:

```
MATCH (failedNode:NetworkNode {id: $nodeId})
MATCH (failedNode)-[:ROUTES_THROUGH*1..5]-(service:CustomerService)
RETURN DISTINCT service.customerName, service.slaTier
ORDER BY service.slaTier;
```

Worth naming explicitly: this is the THIRD distinct domain in this tutorial (after supply-chain components in Chapter 17 and clinical event chains in Section 18.1) where the same underlying pattern : a self-referencing, variable-depth containment or dependency structure, queried for downstream impact : reappears under a different name. Recognising this recurring shape across domains is, in practice, one of the most transferable skills a knowledge graph practitioner develops.

Chapter takeaway

Despite very different surface vocabularies, healthcare event chains, e-commerce behavioural graphs, financial fraud rings, and telecom impact analysis all reduce to a small number of the same underlying graph patterns covered earlier in this tutorial: event chains (Appendix G.5), shared-attribute indirect linking (Section 14.4's similarity concept applied structurally), and self-referencing variable-depth dependency traversal (Sections 3.1 and 17.5). Recognising which underlying pattern a new domain's

requirement maps to is usually more valuable than searching for an industry-specific tutorial : the patterns in this tutorial generalise far beyond the four industries named here.

APPENDICES

Glossary & Quick Reference

A Appendix A : Glossary

Term	Definition
A-Box	Assertional Box : the instance-data layer of a knowledge graph (the facts about specific individuals), as distinct from the T-Box.
Cardinality	The number of instances on each side of a relationship (one-to-one, one-to-many, many-to-many).
CDC (Change Data Capture)	A technique for streaming every insert/update/delete on a source database as an event, typically by tailing its write-ahead log.
Cypher	A declarative, pattern-matching query language for property graphs, the basis for the ISO GQL standard.
Edge / Relationship	A directed, typed connection between two nodes in a property graph; may carry its own properties.
Entity resolution	The process of determining that multiple records, possibly from different sources, refer to the same real-world entity, and merging them into one canonical representation.
Foreign key (FK)	A relational column whose value must match a primary key value in another (or the same) table.
GraphQL	A typed query language and execution engine for client-facing APIs; not a storage model.
Index-free adjacency	A native graph storage strategy where each node directly stores references to its relationships, making traversal a constant-time operation.
Junction / bridge table	A relational table whose sole purpose is to represent a many-to-many relationship between two other tables via a pair of foreign keys.
Knowledge graph	A graph of typed entities and relationships, governed by an explicit or implicit schema (ontology), used to integrate and query connected data.
Label	A type tag attached to a node in a property graph (e.g. :Person); a node may carry multiple labels.
Node	A single entity instance in a property graph.
Normalization	The process of organising relational tables to minimise redundancy, proceeding through First, Second, and Third Normal Form (and beyond).
Ontology	A formal specification of the classes, properties, and constraints that govern a knowledge graph's schema layer.
OWL (Web Ontology Language)	A W3C standard for expressing rich, machine-reasoned ontologies over RDF data.
Primary key (PK)	A column, or minimal set of columns, that uniquely identifies each row in a relational table.
Property	A key-value pair attached to a node or relationship in a property graph, or to a resource in RDF.
Property graph	A graph data model in which both nodes and relationships carry labels/types and property maps.

Term	Definition
R2RML	A W3C standard declarative language for mapping relational databases to RDF.
RDF (Resource Description Framework)	The W3C data model representing facts as (subject, predicate, object) triples.
Referential integrity	A database-enforced guarantee that every foreign key value corresponds to an existing referenced row.
Reification	A technique for making a statement (triple) itself the subject or object of another statement, used to work around RDF's lack of native edge properties.
Resolver	In GraphQL, a function responsible for fetching the value of one specific schema field.
SHACL (Shapes Constraint Language)	A W3C standard for declaring and validating structural constraints over RDF (and, increasingly, property graph) data.
SPARQL	The W3C-standardised query language for RDF data.
Super-node	A node with an unusually large number of relationships, which can cause traversal queries that pass through it to become disproportionately expensive.
T-Box	Terminological Box : the schema/ontology layer of a knowledge graph (classes, properties, rules), as distinct from the A-Box.
Triple	The atomic unit of RDF: a (subject, predicate, object) statement.

B Appendix B : Quick-Reference: Relational → Graph Mapping

A condensed, single-page version of the mapping rules developed in full in Chapter 7, for quick lookup during a migration.

Relational construct	Graph construct
Table	Node label
Row	Node
Column (scalar)	Node property
Primary key	Node identity (with a uniqueness constraint)
Foreign key (N:1)	Directed relationship
Pure junction table (M:N)	Direct relationship (table disappears)
Extra columns on a junction table	Relationship properties
Self-referencing foreign key	Self-referencing relationship
Nullable foreign key	Relationship may or may not exist
Table inheritance / type discriminator column	Multiple node labels on the same node
Lookup table referenced by many tables	Either a small set of nodes, or a plain property, depending on whether the categories themselves need independent querying

C Appendix C : Query Language Cheat Sheet

Operation	SQL	Cypher	SPARQL
Match by property	WHERE col = val	{prop: val} in pattern	?x :prop val .
Join / traverse	JOIN ... ON	-[:REL]->	?a :rel ?b .
Variable-depth traversal	WITH RECURSIVE	[:REL]*1..5]	rel+ / rel*
Aggregation	GROUP BY / SUM	WITH ... , sum(x)	GROUP BY / SUM(?x)
Optional match	LEFT JOIN	OPTIONAL MATCH	OPTIONAL { }
Idempotent upsert	INSERT ... ON CONFLICT	MERGE	n/a (typically handled at load time)

D Appendix D : Vendor & Tooling Landscape

A snapshot of commonly used products in each category discussed throughout this tutorial, current as of this writing. Technology in this space moves quickly : treat this as a starting point for your own evaluation, not a final recommendation, and verify current feature sets and licensing directly with each vendor before committing.

Property graph databases

Product	Notes
Neo4j	The most widely adopted property graph database; originated Cypher; strong ecosystem (APOC, Graph Data Science library) and tooling maturity.
Amazon Neptune	Managed AWS service; supports both property graph (openCypher/Gremlin) and RDF (SPARQL) modes on the same underlying engine.
TigerGraph	Focused on large-scale distributed graph analytics and deep-link traversal performance.
Memgraph	In-memory-first property graph database emphasising low-latency streaming/real-time use cases.

RDF triple stores

Product	Notes
Ontotext GraphDB	Widely used commercial RDF store with strong OWL reasoning and full-text search integration.
Stardog	Enterprise-focused RDF platform with virtual graph / OBDA capability over relational sources.
Apache Jena / Fuseki	Open-source Java framework for RDF, SPARQL, and OWL reasoning; a common starting point for RDF experimentation.
Blazegraph	Open-source RDF store; formerly powered Wikidata's Query Service.

Relational-to-graph mapping tools

Tool	Notes
Ontop	Open-source, high-performance OBDA (Ontology-Based Data Access) engine implementing R2RML; exposes a virtual RDF graph over a live relational source with no data duplication.
D2RQ	An earlier, mature R2RML-family platform for accessing relational databases as virtual RDF graphs.
Morph-KGC	A fast, modern engine supporting both R2RML and RML mappings, with support for non-relational sources.
Neo4j APOC + neo4j-admin import	Vendor-native tooling for bulk-loading relational data into Neo4j, either via JDBC live connection or bulk CSV import.

E Appendix E : Frequently Asked Questions

“Do we need to migrate ALL of our relational data into the graph?”

No : and in most successful deployments, teams deliberately don't. Apply the table classification framework from Section 7.2 and migrate only the entities and relationships that actually serve the connected-data questions motivating the project (Section 12.3's first anti-pattern). Purely transactional, non-connected data is usually better left in the relational system.

“Will the knowledge graph replace our relational database?”

In the large majority of real deployments, no. As argued throughout Part IV, particularly Section 10.1, the relational database typically remains the system of record, with the knowledge graph as a derived, read-optimised view. This is a pragmatic default, not an absolute rule : see Section 10.1 for the narrower circumstances where a graph-native system of record is the right choice.

“How much staleness is acceptable in the graph?”

It depends entirely on the use case, and this should be decided explicitly and early, since it drives the choice of synchronisation strategy in Section 7.10. A recommendation engine can typically tolerate hours of staleness; a fraud detection system typically cannot tolerate more than seconds; an internal analytics dashboard can often tolerate a full day.

“Which should we learn first: Cypher or SPARQL?”

Section 8.4's practical guidance applies directly: learn Cypher first if you're targeting a property graph (the more common choice for application-facing use cases), and learn SPARQL specifically if your target is an RDF store or you need standards-based semantic integration.

“Is GraphQL a type of database?”

No. This is directly addressed in Chapter 5 and Section 9.2: GraphQL is a client-facing query language and execution engine that can sit in front of any backend : relational, graph, or a mix of several : and says nothing about how or where data is actually stored.

“How long does a typical migration take?”

Highly variable, but as a rough planning heuristic: a focused migration of a well-understood, moderately sized schema (a handful of entity tables, a few junction tables) following the methodology in Chapter 7 commonly takes a small team a few weeks from schema analysis through validated production sync : most of that time spent on entity resolution (Section 7.6) and validation (Section 7.9) when multiple source systems are involved, rather than on the core mapping logic itself.

“Do we need a data scientist / ontologist on the team?”

For a straightforward mapping of one well-normalized relational schema (Section 7.3), no specialist background is required : a competent backend or data engineer can apply the mapping rules directly. Formal ontology expertise (Chapter 6) becomes valuable specifically when integrating many heterogeneous sources, when formal OWL reasoning is required, or when the graph must interoperate with external, standards-based vocabularies.

“Can we use our knowledge graph as a source for LLM applications?”

Yes : this is precisely the subject of Chapter 15. A knowledge graph is a strong foundation for GraphRAG specifically because it can supply the LLM with verifiable, structured, multi-hop context rather than relying solely on unstructured text similarity search.

F Appendix F : Troubleshooting Guide

Symptom	Likely cause	Where to look
Node/relationship counts don't match the source after migration	A migration step ran before its dependency, or a filtering condition (e.g. soft-deletes) was applied inconsistently between scripts	Section 7.9's validation checklist; re-check load order per Section 12.2
A traversal query that should be fast is unexpectedly slow	The query is passing through a super-node with very high fan-out	Section 12.3's super-node anti-pattern; consider adding a more selective filter earlier in the traversal
The graph 'looks right' but downstream analytics numbers don't match relational reports	A silent type-coercion bug during migration (e.g. DECIMAL to floating point), or a double-counted relationship from an incomplete MERGE key	Section 7.9's property-value spot-check step; verify MERGE keys include every column that should determine uniqueness
Duplicate nodes for what should be the same entity	MERGE was keyed on a property that isn't actually unique, or entity resolution (Section 7.6) wasn't applied across multiple source systems	Re-verify the uniqueness constraint backing every MERGE; revisit Section 7.6 if multiple sources feed the same node label
CDC-based sync silently stops updating the graph	A malformed or unexpected event type broke the consumer without raising a visible error	Add explicit monitoring/alerting on consumer lag and error rates, per Section 10.5's operational guidance
GraphRAG answers cite facts that aren't actually in the graph	The LLM is not being constrained to answer strictly from the retrieved subgraph context	Revisit the prompt construction in Section 15.5 : explicitly instruct the model to answer only from provided context and to say so when the context is insufficient

G Appendix G : Extended SQL-Pattern Mapping Catalog

Chapter 7 established the core mapping rules. This appendix extends that catalog with a wider range of concrete, worked before/after examples covering SQL patterns not explicitly worked through elsewhere in the tutorial : useful as a direct lookup reference during a real migration.

G.1 One-to-one relationships

A one-to-one relationship (e.g. a USERS table and a separate USER_PROFILES table, split for storage or access-control reasons) maps identically to a one-to-many relationship, just with an implicit application-level guarantee that only one such edge will ever exist per source node : the graph model does not need a structurally different construct for this case.

```
-- Relational: USERS(id) <-> USER_PROFILES(user_id FK, bio, avatar_url)
MATCH (u:User {id: $id})
MERGE (p:Profile {bio: $bio, avatarUrl: $avatarUrl})
MERGE (u)-[:HAS_PROFILE]->(p)
```

G.2 Multi-column (composite) foreign keys

When a foreign key spans multiple columns (common in schemas using natural composite keys, e.g. (warehouse_id, sku) referencing (warehouse_id, sku) on an inventory table), the relationship-creation MATCH simply matches on all key columns together : no different in kind from a single-column foreign key, just wider:

```
MATCH (i:InventoryItem {warehouseId: $whId, sku: $sku}), (w:Warehouse {id: $whId})
MERGE (i)-[:STOCKED_AT]->(w)
```

G.3 Enum / check-constraint columns

A column constrained by a SQL CHECK or a database ENUM type (e.g. orders.status IN ('pending', 'shipped', 'delivered', 'cancelled')) maps most simply to a plain string property with the same allowed values, optionally enforced via an ontology constraint (Section 16.2) rather than a database-level enum type, since most property graph databases don't have a native enum column type.

```
CREATE CONSTRAINT order_status_valid IF NOT EXISTS
FOR (o:Order) REQUIRE o.status IN ['pending','shipped','delivered','cancelled'];
```

G.4 Star-schema fact and dimension tables (data warehouse migrations)

A data warehouse's star schema : a large FACT table (e.g. SALES_FACT) surrounded by smaller DIMENSION tables (DATE_DIM, PRODUCT_DIM, STORE_DIM) : maps with a small twist on the standard junction-table rule: the fact table is usually itself the RICH RELATIONSHIP connecting several dimension nodes at once, rather than a single binary relationship. Where more than two dimensions must be connected by one fact, model the fact as its own node (an “event” or “transaction” node) with an outgoing relationship to each dimension, rather than forcing an inherently multi-way fact into a single two-endpoint edge.

```

MATCH (d:Date {id: $dateId}), (p:Product {id: $productId}), (s:Store {id: $storeId})
CREATE (f:SalesEvent {amount: $amount, units: $units})
CREATE (f)-[:ON_DATE]->(d)
CREATE (f)-[:OF_PRODUCT]->(p)
CREATE (f)-[:AT_STORE]->(s)

```

G.5 Audit / history tables (temporal patterns beyond Section 7.8.4)

A dedicated audit table (e.g. ORDER_STATUS_HISTORY, recording every status transition with a timestamp) maps cleanly to a chain of timestamped relationship instances, or, more commonly, to a series of small “event” nodes linked in sequence : the latter is usually preferable when each transition itself needs to be individually queried or annotated:

```

MATCH (o:Order {id: $orderId})
CREATE (e:StatusChange {status: $status, changedAt: $timestamp})
CREATE (o)-[:HAD_STATUS_CHANGE]->(e)
// Optionally chain events for ordered traversal:
// MATCH (prev:StatusChange {orderId: $orderId}) ... CREATE (prev)-[:NEXT]->(e)

```

G.6 Polymorphic / generic reference columns

A relational anti-pattern occasionally encountered : a single COMMENTS table with a commentable_type and commentable_id column pair meant to reference EITHER a Ticket OR an Article OR a Product depending on the type value : maps to simply choosing the correct relationship TYPE and target node label at migration time, based on the value of commentable_type, resolving what was ambiguous in the relational schema into an explicit, typed edge in the graph:

```

MATCH (c:Comment {id: $id})
CALL apoc.do.case([
  row.commentableType = 'Ticket', 'MATCH (t:Ticket {id: $tid}) MERGE (c)-[:ON_TICKET]->(t)',
  row.commentableType = 'Article', 'MATCH (a:Article {id: $tid}) MERGE (c)-[:ON_ARTICLE]->(a)'
], 'RETURN null', {tid: row.commentableId, c: c}) YIELD value RETURN value;

```

H Appendix H : Index of Figures

A consolidated list of every numbered figure in this tutorial, for quick navigation.

Figure	Title
2.1	A small relational schema with primary and foreign keys
2.2	Entity-Relationship diagram (crow's-foot notation)
2.3	Progressive normalization: UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF
2.4	How a B-Tree index accelerates lookups
2.5	SQL join semantics
3.1	Why highly-connected questions are costly in the relational model
4.1	The labeled property graph model
4.2	RDF: the atomic unit is the triple
5.1	GraphQL schema: types as a typed graph of fields
5.2	GraphQL execution flow: one query, many sources
6.1	Two layers of a knowledge graph: ontology (T-Box) and instance data (A-Box)
7.1	Mapping rules: relational constructs $\rightarrow$ graph constructs
7.2	Collapsing a many-to-many junction table into a direct edge
7.3	End-to-end architecture: relational database $\rightarrow$ knowledge graph
7.4	Entity resolution: merging records from multiple sources
7.5	The resulting knowledge graph
9.1	Same question, four query languages
14.1	Graph algorithms: centrality and community detection
15.1	GraphRAG: grounding an LLM's answer in graph traversal
15.2	Vector embeddings vs. explicit graph relationships
16.1	Six data quality dimensions applied to a knowledge graph

I Appendix I : A Complete, Runnable Sample Migration Script

This appendix collects a single, complete, end-to-end migration script for the Customers/Orders/Products schema used throughout this tutorial (Figure 2.1), combining every technique from Chapter 7 into one runnable reference. It is deliberately verbose and heavily commented so it can serve as a template you adapt directly for your own schema.

```

"""
migrate.py : Reference relational-to-knowledge-graph migration script.
Implements the full methodology from Chapter 7 against the running
Customers / Orders / OrderItems / Products / Categories example schema.
"""

import psycopg2
from neo4j import GraphDatabase

PG_DSN = "dbname=shop user=readonly host=pg.internal"
NEO4J_URI = "bolt://graph.internal:7687"

def get_pg_conn():
    return psycopg2.connect(PG_DSN)

def get_neo4j_driver():
    return GraphDatabase.driver(NEO4J_URI, auth=("neo4j", "password"))

# --- Step 0: constraints first (Section 16.2) : fail fast on bad data -----
CONSTRAINTS = [
    "CREATE CONSTRAINT customer_id IF NOT EXISTS FOR (c:Customer) REQUIRE c.id IS UNIQUE",
    "CREATE CONSTRAINT order_id IF NOT EXISTS FOR (o:Order) REQUIRE o.id IS UNIQUE",
    "CREATE CONSTRAINT product_id IF NOT EXISTS FOR (p:Product) REQUIRE p.id IS UNIQUE",
    "CREATE CONSTRAINT category_id IF NOT EXISTS FOR (g:Category) REQUIRE g.id IS UNIQUE",
]

def apply_constraints(session):
    for stmt in CONSTRAINTS:
        session.run(stmt)

# --- Step 1: entity tables, in dependency order (Section 12.2, rule 4) -----
def migrate_categories(pg, session):
    cur = pg.cursor()
    cur.execute("SELECT category_id, name, parent_id FROM categories")
    for cat_id, name, parent_id in cur.fetchall():
        session.run(
            "MERGE (c:Category {id: $id}) SET c.name = $name",
            id=cat_id, name=name,
        )
    # second pass for self-referencing FK : Section 7.8.3 pattern
    cur.execute("SELECT category_id, parent_id FROM categories WHERE parent_id IS NOT NULL")
    for cat_id, parent_id in cur.fetchall():
        session.run(

```

```

        "MATCH (c:Category {id: $id}), (p:Category {id: $parentId}) "
        "MERGE (c)-[:CHILD_OF]->(p)",
        id=cat_id, parentId=parent_id,
    )

def migrate_products(pg, session):
    cur = pg.cursor()
    cur.execute("SELECT product_id, sku, name, price, category_id FROM products")
    for pid, sku, name, price, cat_id in cur.fetchall():
        session.run(
            "MERGE (p:Product {id: $id}) SET p.sku=$sku, p.name=$name, p.price=$price "
            "WITH p MATCH (c:Category {id: $catId}) MERGE (p)-[:IN_CATEGORY]->(c)",
            id=pid, sku=sku, name=name, price=float(price), catId=cat_id,
        )

def migrate_customers(pg, session):
    cur = pg.cursor()
    cur.execute("SELECT customer_id, first_name, last_name, email, city FROM customers")
    for cid, fname, lname, email, city in cur.fetchall():
        session.run(
            "MERGE (c:Customer {id: $id}) "
            "SET c.name = $name, c.email = $email, c.city = $city",
            id=cid, name=f"{fname} {lname}", email=email, city=city,
        )

# --- Step 2: relationship tables (Section 7.3) -----
def migrate_orders(pg, session):
    cur = pg.cursor()
    cur.execute("SELECT order_id, customer_id, order_date, status FROM orders")
    for oid, cust_id, odate, status in cur.fetchall():
        session.run(
            "MERGE (o:Order {id: $id}) SET o.date=$date, o.status=$status "
            "WITH o MATCH (c:Customer {id: $custId}) MERGE (c)-[:PLACED]->(o)",
            id=oid, date=str(odate), status=status, custId=cust_id,
        )

def migrate_order_items(pg, session):  # junction-table collapse : Section 7.3.2
    cur = pg.cursor()
    cur.execute("SELECT order_id, product_id, quantity, unit_price FROM order_items")
    for oid, pid, qty, price in cur.fetchall():
        session.run(
            "MATCH (o:Order {id: $oid}), (p:Product {id: $pid}) "
            "MERGE (o)-[:CONTAINS]->(p) SET r.quantity=$qty, r.unitPrice=$price",
            oid=oid, pid=pid, qty=qty, price=float(price),
        )

# --- Step 3: validation (Section 7.9) -----
def validate(pg, session):
    checks = [
        ("SELECT count(*) FROM customers", "MATCH (c:Customer) RETURN count(c)"),
        ("SELECT count(*) FROM orders", "MATCH (o:Order) RETURN count(o)"),
    ]

```

```

    ("SELECT count(*) FROM order_items", "MATCH ()-[r:CONTAINS]->() RETURN count(r)",
    ]
for sql, cypher in checks:
    cur = pg.cursor(); cur.execute(sql)
    sql_count = cur.fetchone()[0]
    graph_count = session.run(cypher).single().value()
    status = "OK" if sql_count == graph_count else "MISMATCH"
    print(f'{status}: sql={sql_count} graph={graph_count} ({sql})')

def main():
    pg = get_pg_conn()
    driver = get_neo4j_driver()
    with driver.session() as session:
        apply_constraints(session)
        migrate_categories(pg, session) # entities with no dependencies first
        migrate_products(pg, session) # depends on Category
        migrate_customers(pg, session) # independent entity
        migrate_orders(pg, session) # depends on Customer
        migrate_order_items(pg, session) # depends on Order and Product
        validate(pg, session)
    driver.close()
    pg.close()

if __name__ == "__main__":
    main()

```

Every design decision in this script traces back to a specific rule established earlier in this tutorial: constraints applied before data (Section 16.2), dependency-ordered loading (Section 12.2), MERGE-based idempotent writes throughout (Section 7.4), the junction-table collapse for order items (Section 7.3.2), the self-referencing pattern for categories (Section 7.8.3), and an automated count-parity validation pass as the final step (Section 7.9). Adapting it to a new schema is primarily a matter of repeating this same entity-then-relationship, dependency-ordered structure for your own tables.

J Appendix J : Cypher & SPARQL Function Quick Reference

A working reference of commonly used built-in functions, useful when translating SQL expressions (Chapter 2) into their Cypher or SPARQL equivalents during migration mapping logic (Chapter 7).

String functions

Purpose	SQL	Cypher	SPARQL
Concatenate	a b or CONCAT(a,b)	a + b	CONCAT(?a, ?b)
Uppercase / lowercase	UPPER(x) / LOWER(x)	toUpper(x) / toLower(x)	UCASE(?x) / LCASE(?x)
Substring	SUBSTRING(x,1,3)	substring(x,0,3)	SUBSTR(?x,1,3)
Trim whitespace	TRIM(x)	trim(x)	n/a (pre-process on load)
String length	LENGTH(x)	size(x)	STRLEN(?x)
Pattern match	x LIKE '%foo%'	x CONTAINS 'foo'	REGEX(?x, 'foo')

Numeric and aggregate functions

Purpose	SQL	Cypher	SPARQL
Round	ROUND(x, 2)	round(x, 2)	ROUND(?x)
Absolute value	ABS(x)	abs(x)	ABS(?x)
Count rows / bindings	COUNT(*)	count(*)	COUNT(?x)
Sum / average	SUM(x) / AVG(x)	sum(x) / avg(x)	SUM(?x) / AVG(?x)
Min / max	MIN(x) / MAX(x)	min(x) / max(x)	MIN(?x) / MAX(?x)
Distinct	SELECT DISTINCT x	RETURN DISTINCT x	SELECT DISTINCT ?x

Date and time functions

Purpose	SQL	Cypher	SPARQL
Current timestamp	NOW()	datetime()	NOW()
Extract year	EXTRACT(YEAR FROM x)	x.year	YEAR(?x)
Date difference	x - y	duration.between(y, x)	n/a (compute in application logic)

Null / existence handling

Purpose	SQL	Cypher	SPARQL
Coalesce / default value	COALESCE(x, 'default')	coalesce(x, 'default')	COALESCE(?x, 'default')
Is null / is not null	x IS NULL	x IS NULL	!BOUND(?x)

Purpose	SQL	Cypher	SPARQL
Check property/relationship exists	n/a (structural, via JOIN presence)	EXISTS { (n)-[:REL]->() }	FILTER EXISTS { ?n :rel ?o }

K Appendix K : A Complete, Annotated Sample Ontology

This appendix provides a complete, self-contained ontology definition in Turtle/OWL syntax for the e-commerce example used throughout this tutorial : combining every ontology construct introduced in Chapter 6 (classes, subclass hierarchies, object and datatype properties, domain/range, cardinality) into one worked reference you can adapt directly.

```
@prefix : <http://example.org/shop#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .

# -----
# CLASSES (Section 6.3) : the entity types in our domain
# -----
:Party a owl:Class ;
  rdfs:comment "Abstract superclass for anything that can place or receive an order" .

:Customer a owl:Class ;
  rdfs:subClassOf :Party ;          # subclass hierarchy : Section 6.3
  rdfs:comment "A registered purchaser" .

:Order a owl:Class ; rdfs:comment "A single purchase transaction" .
:Product a owl:Class ; rdfs:comment "A sellable item" .
:Category a owl:Class ; rdfs:comment "A grouping of related products" .

# -----
# OBJECT PROPERTIES (relationships between entities)
# -----
:placed a owl:ObjectProperty ;
  rdfs:domain :Customer ; rdfs:range :Order ; # domain/range : Section 6.3
  owl:inverseOf :placedBy .                # property characteristic : Section 6.4

:placedBy a owl:ObjectProperty ;
  rdfs:domain :Order ; rdfs:range :Customer .

:contains a owl:ObjectProperty ;
  rdfs:domain :Order ; rdfs:range :Product .

:inCategory a owl:ObjectProperty ;
  rdfs:domain :Product ; rdfs:range :Category .

:childOf a owl:ObjectProperty ;
  rdfs:domain :Category ; rdfs:range :Category ;
  a owl:TransitiveProperty .             # transitive inference : Section 6.4

# -----
# DATATYPE PROPERTIES (literal-valued attributes)
# -----
:email a owl:DatatypeProperty ;
  rdfs:domain :Customer ; rdfs:range xsd:string .

:price a owl:DatatypeProperty ;
```

```

rdfs:domain :Product ; rdfs:range xsd:decimal .

:orderDate a owl:DatatypeProperty ;
  rdfs:domain :Order ; rdfs:range xsd:date .

# -----
# SHACL CONSTRAINTS (validation rules : Section 16.2)
# -----
:CustomerShape a sh:NodeShape ;
  sh:targetClass :Customer ;
  sh:property [
    sh:path :email ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "Every Customer must have exactly one email" ;
  ] .

:OrderShape a sh:NodeShape ;
  sh:targetClass :Order ;
  sh:property [
    sh:path :contains ; sh:minCount 1 ;
    sh:message "An Order must contain at least one Product" ;
  ] ;
  sh:property [
    sh:path :placedBy ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:message "An Order must be placed by exactly one Customer" ;
  ] .

```

Every construct in this listing maps back to a specific rule from Chapter 6: the `:Party/:Customer` hierarchy demonstrates subclassing (Section 6.3); `:placed / :placedBy` demonstrates an inverse property pair (Section 6.4); `:childOf` demonstrates a transitive property enabling category-tree inference (Section 6.4); and the two SHACL shapes demonstrate cardinality and existence constraints (Section 16.2) enforced independently of whatever graph engine ultimately stores the data.

L Appendix L : Self-Check Questions

A set of review questions organised by part, useful for confirming your own understanding or for structuring a team reading group working through this tutorial together. Answers are deliberately not provided : each question can be answered directly by revisiting the referenced section.

Part One : Relational Foundations

1. What is the difference between a primary key and a foreign key, and which one enforces referential integrity (Section 2.1)?
2. Why does a properly normalized (3NF) schema make graph conversion easier, rather than harder (Section 2.3)?
3. Explain, in your own words, why join cost tends to grow combinatorially with traversal depth (Section 3.1).
4. Name two things relational databases still do better than graph databases, and explain why (Section 3.4).

Part Two : Graph Thinking & GraphQL

1. What is the single biggest structural difference between a property graph edge and a plain RDF triple (Section 4.2)?
2. Why is GraphQL described as 'not a database' in this tutorial : what layer of the system does it actually occupy (Section 5.1)?
3. Explain the N+1 problem and how DataLoader batching solves it (Section 5.4).

Part Three : Knowledge Graphs & Conversion

1. What is the difference between the T-Box and the A-Box (Section 6.2)?
2. Walk through, from memory, the four-step classification process for a relational table (Section 7.2).
3. Why does collapsing a many-to-many junction table often produce the single biggest query simplification in a migration (Section 7.3.2)?
4. Name the three synchronisation strategies from Section 7.10 and one scenario where each is the right choice.

Parts Four through Six

1. Why is a variable-length path expression in Cypher considered a more natural fit for graph traversal than a recursive CTE in SQL (Section 8.1)?
2. In your own words, explain why the knowledge graph should usually remain a derived view rather than the system of record (Section 10.1).
3. What distinguishes GraphRAG from plain vector-similarity RAG, and when is the added complexity worth it (Section 15.6)?
4. Name three of the six data quality dimensions from Chapter 16 and give a graph-specific example of each.